TECHNICAL REPORT

Rethinking the Evaluation of Asynchronous I/O in HPC Systems



Noah Lewis · lewisnoah592@gmail.com

Abstract

Asynchronous I/O is widely used to hide the cost of writing checkpoints and analysis output in high-performance computing (HPC) applications. The application issues its I/O early, and a background engine completes it while the application continues computing. However, much of the evidence for this benefit comes from I/O kernels and benchmarks that emulate the computation phase with a call to `sleep()`. A sleeping thread does not use its core, its memory bandwidth, or the network, so it removes the resource contention that determines whether the I/O can actually be hidden. In this paper, we survey 17 evaluations and benchmarks spanning HDF5, ADIOS, PDC, POSIX-level checkpointing, and deep-learning I/O, and find that `sleep`, an unspecified compute phase, or no compute phase at all is the norm. We then present an open benchmark that drives POSIX, MPI-IO, HDF5, ADIOS2, and PDC with interchangeable compute surrogates calibrated to the same standalone duration, including CPU-bound, memory-bound, and communication-bound kernels, a stencil mini-app, and GPU kernels. We find that the contention does not always appear in the same place. Depending on the I/O library, either the computation slows down while the background engine runs, or the background engine makes little progress and the application later waits for the remaining I/O. In both cases, the run time the application observes is longer than a `sleep`-based evaluation predicts. On a 48-core node writing 120 GiB per run, 21 of 24 configurations take at least 5% longer than `sleep` predicts, and up to 81% longer. In addition, the speedup of the HDF5 asynchronous VOL connector drops from 1.87× with `sleep` to 1.13× with a halo exchange, and non-blocking MPI-IO is slower than blocking MPI-IO in every configuration we measured. Even PDC, whose server runs on a reserved core, runs up to 81% longer than `sleep` predicts, because a polling progress thread in each client shares the core of the rank. Lastly, we provide recommendations for evaluating asynchronous I/O and for interpreting existing results that rely on `sleep`.

1 Introduction

Many high-performance computing (HPC) simulations and analysis codes alternate between computation and Input/Output (I/O) phases. A simulation advances a number of timesteps and then writes a checkpoint or plot file, while an analysis code reads a timestep and then processes it. When I/O is performed synchronously, the computation stalls until the I/O phase completes. Because storage bandwidth has grown more slowly than compute capability, these stalls can become a significant fraction of the total run time.

Asynchronous I/O addresses this problem by separating the issue of an I/O operation from its completion. The application initiates an I/O operation, continues with its next computation phase, and only waits for completion when it needs the buffer again or reaches a synchronization point. Implementations range from POSIX asynchronous I/O (AIO) and non-blocking MPI-IO to background-thread frameworks, such as the HDF5 asynchronous Virtual Object Layer (VOL) connector [24,25], and server-based approaches that use dedicated cores or staging resources [5]. In the ideal case, the observed I/O cost per step falls from t_{io} to a small overhead, and the time per step becomes approximately $\max(t_{comp}, t_{io})$ rather than $t_{comp} + t_{io}$ [20].

Whether this ideal is reached depends on a question that is easy to overlook: *what is the application doing while the I/O runs in the background?* The background I/O engine needs CPU time, memory bandwidth for buffer copies, and network bandwidth to reach the parallel file system (PFS). A real computation phase competes for all three. However, a common practice in the asynchronous I/O literature is to replace the computation phase with a call to `sleep()` of a chosen duration [13,20,25]. This choice is convenient, because it makes experiments cheap and allows the authors to select a compute phase long enough to fully hide the I/O. However, it also removes the contention that could prevent the overlap.

Contention can appear on either side of the overlap. The background engine can take CPU time, memory bandwidth, or MPI resources from the application, which makes the computation run longer. Alternatively, the application can starve the engine, so the I/O does not progress during the computation and the application waits for it at the next synchronization point. Studies that report only the observed I/O time cannot clearly see either effect, and studies that use `sleep` cannot see them at all, because a sleeping thread neither competes for resources nor withholds them. For this reason, we evaluate asynchronous I/O by the quantity the application actually experiences, its end-to-end run time, and attribute any difference to the compute side or the I/O side.

In this paper, our contributions are the following:

- We survey how 17 asynchronous I/O and overlap evaluations spanning HDF5, ADIOS, PDC, POSIX-level checkpointing, and deep-learning I/O treat the computation phase. Only 6 of them use a computation that consumes the resources background I/O needs (Section 3).

- We collect the evidence that bears on the validity of `sleep`-based evaluation from interference studies, real-application evaluations, overlap benchmarks, and the caveats stated by the `sleep`-based studies themselves.
- We present an open benchmark that drives POSIX, MPI-IO, HDF5, and ADIOS2 with interchangeable compute surrogates calibrated to the same standalone duration. We use it to measure how far `sleep`-based estimates of the benefit of asynchronous I/O diverge from estimates that include contention (Section 4).
- We provide recommendations for designing future evaluations and for interpreting published results that rely on `sleep()`.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 describes our survey methodology and its findings. Section 4 presents our benchmark and experimental results. Lastly, Section 5 presents our recommendations and concludes the paper.

2 Related Work

In this section, we review asynchronous I/O frameworks for HPC, describe how influential evaluations of these frameworks emulate the computation phase, and summarize the existing evidence on interference between background I/O and the application.

2.1 Asynchronous I/O frameworks for HPC

Asynchronous read and write operations have long been available through POSIX AIO and non-blocking MPI-IO, and early studies examined overlap strategies at the MPI-IO level [18]. However, these interfaces do not cover non-data operations such as opening and closing files, and they require users to manage dependencies and byte offsets manually [25]. For this reason, high-level libraries have added their own mechanisms. Tang et al. introduced an HDF5 VOL connector that queues operations, tracks their dependencies as a directed acyclic graph (DAG), and executes them on a background thread managed by Argobots [24, 25]. Damaris dedicates one or more cores per node to data management tasks, including asynchronous I/O [5]. In addition, staging-based and server-based systems move data off the compute processes and persist it later.

2.2 Sleep-based compute emulation

Several influential evaluations of asynchronous I/O emulate the computation phase with `sleep()`. We describe them below.

h5bench. The h5bench suite [13] exposes an `EMULATED_COMPUTE_TIME_PER_TIMESTEP` parameter. After each iteration, the write and read kernels sleep for this duration to emulate the application's computation. Because h5bench is widely used as a standard HDF5 benchmark, this choice propagates into many downstream studies.

HDF5 asynchronous VOL. In the TPDS evaluation of the HDF5 asynchronous VOL connector [25], all experiments write five timesteps with a sleep between them, sized so that asynchronous I/O can fully overlap with it. For the Nyx workload, all computation was replaced with sleep, so that the same data structures could be written without running the expensive simulation. The authors argue that the sleep durations, up to 60 s on Cori and 20 s on Summit, are conservative relative to real VPIC compute phases.

Modeling asynchronous I/O. Ravi et al. [20] develop a model in which an asynchronous step costs $\max(t_{comp}, t_{io} - t_{comp}) + t_{\text{transact overhead}}$. Their VPIC-IO and BD-CATS-IO experiments use a 30-second sleep in place of the computation, and the emulated compute length is tuned manually to completely hide the asynchronous I/O time.

Proactive Data Containers. Proactive Data Containers (PDC) move data asynchronously through server processes that run on extra cores [23]. In the CCGrid'18 evaluation of PDC, the multi-timestep VPIC-IO runs "added a sleep time, representing computations between writing data," and the client observes only the last write plus overhead. This yields up to a $5\times$ speedup over HDF5. The follow-up tuning study [22] configures Nyx and BD-CATS-IO with "computation time (longer than I/O time)" between timesteps, but does not describe what that computation is. However, it does state that PDC reserves one core per node for its servers.

ADIOS. The BP5 engine of ADIOS2 can write buffered data in the background after `EndStep`. It does so either naively or guided by `EnterComputationBlock` and `ExitComputationBlock` hints provided by the application [6]. The ADIOS2 skeleton tool, `adios_iotest`, expresses the compute phase of a workflow as a `sleep` command, or a `busy` loop, between I/O steps [1]. The earlier Skel generator produces ADIOS skeletal applications whose source contains "the appropriate ADIOS I/O calls, data generation, and timers" and no computation at all [14]. It is important to note that the ADIOS developers themselves report mixed results for asynchronous writes in real codes. They observed a large gain for WarpX on Perlmutter, while on Frontier "we don't see improvement over synchronous I/O" [19]. In addition, the openPMD-api documentation warns that asynchronous writes "can compete with MPI for communication resources, impacting the *compute* performance of an application" [17].

Deep-learning I/O. DLIO [4] emulates the I/O of scientific deep-learning training. Its per-step `computation_time` is profiled on a specific accelerator, and its shipped configurations carry separate values for V100, A100, and H100 GPUs. This time is replayed by calling `time.sleep()` [2]. Because the DLIO reader workers prefetch the next batches while the main process sleeps, the benchmark cannot observe host-side contention between the input pipeline and the training step.

POSIX-level and checkpointing systems. Systems that flush checkpoints or burst-buffer contents in the background through POSIX are rarely evaluated against a running computation. Gossman et al. evaluate aggregation strategies for the asynchronous flush of VeloC by timing the flush phase alone [7, 16]. Harmonia, an interference-aware burst-buffer

scheduler, uses a synthetic benchmark whose computation phases are "artificial math kernels," and the content of these kernels is not specified [11].

2.3 Evidence on interference and overlap

To the best of our knowledge, no study focuses specifically on sleep-based emulation as a methodological problem. Instead, the relevant evidence is distributed across several lines of work.

Caveats within sleep-based studies. The HDF5 asynchronous VOL paper notes that the background thread can affect application performance if all cores are used by the application, and recommends sparing resources for it [25]. It also explains that starting background execution while the main thread is active can cause the two threads to compete for shared resources, in the worst case serializing them. Ravi et al. limit the scope of their model to the absence of node-level oversubscription [20]. In addition, guidance from the HDF Group recommends that OpenMP applications leave one core per process for the background thread [21]. In each case, the identified risk is one that a sleeping main thread cannot exhibit.

Real-application evaluations. Jain et al. evaluated HDF5 asynchronous I/O in Flash-X on Summit [10]. They note that background threads compete for node resources and with communication, and they observe a measurable overhead from asynchronous I/O. This overhead is outweighed by the reduction in I/O time, which yields up to a $1.5\times$ overall speedup. This figure is far more modest than the I/O-time speedups of roughly 4.5 to $4.9\times$ reported with sleep-based emulation [25], although the two metrics are not directly comparable.

Interference studies. Tseng et al. [27] study interference from background I/O directly. They argue that background I/O competes with the application for CPU time, memory bandwidth, and network bandwidth, and that multicore nodes make the problem worse because many I/O operations compete concurrently. Without accounting for this, asynchronous I/O may perform worse than blocking I/O. Their methodology stresses CPU and memory bandwidth separately and is validated on HACC. Tarraf et al. [26] build on this observation. They note that developers often assume overlap will be sufficient regardless of the configuration, and they evaluate with real computation (WaComM++ and a modified HACC-IO) rather than sleep.

Overlap benchmark methodology. Benchmarks designed to measure overlap generally use computation that occupies the CPU. The non-blocking IMB-IO tests of the Intel MPI Benchmarks run a calibrated CPU-intensive kernel concurrently with the I/O, and compute the overlap as $(t_{pure} + t_{CPU} - t_{ovrl})/\min(t_{pure}, t_{CPU})$ [9]. Similarly, COMB [12] and the MPI overlap suite of Denis and Trahay [3] measure the overlap of communication and computation against busy work. These designs assume that the compute side can be slowed by the background activity, which is not possible with a sleep.

3 Survey Methodology

In this section, we describe how we selected and coded the evaluations in our survey, and we present our findings.

Scope and search. We searched for evaluations that quantify the benefit of asynchronous or overlapped I/O, together with the benchmarks those evaluations are built on. Starting from the HDF5 asynchronous VOL papers [24, 25] and h5bench [13], we followed citations backward and forward. In addition, we searched IEEE Xplore, the ACM Digital Library, and Google Scholar for combinations of "asynchronous I/O," "background thread," "overlap," "checkpoint," and "staging" with the names of the major I/O libraries (HDF5, ADIOS and ADIOS2, PDC, MPI-IO, and POSIX). Because benchmark tools encode their methodology in code as much as in prose, we also inspected the released source code of h5bench, `adios_iotest`, and DLIO.

Inclusion criteria. A source is included if it meets two criteria. First, it reports a quantitative evaluation of asynchronous I/O, or it is a benchmark designed to measure overlap. Second, the treatment of the application's computation phase can be determined from its text or released source code. We identified 26 sources, and 17 of them meet both criteria. These 17 works are listed in Table 1. The other nine sources are library documentation, tutorials, and design papers whose evaluation has no interleaved computation. They provide context in Section 2 but are not coded.

Coding. For each included work, we record the I/O library and five attributes:

1. The *compute phase*, which is `sleep`, a synthetic kernel that consumes resources (busy), a real application, unspecified, or none.
2. Whether the paper *describes* what the compute phase does.
3. Whether it reports an *application-side cost*, such as the slowdown of the compute phase or the end-to-end time, rather than only the observed I/O time.
4. Whether the *core allocation* for background threads or servers is stated.
5. Whether the results are validated on a *real application* with its own computation.

Each attribute is marked as ✓ (yes), ○ (partially or as a caveat), × (no), n/a (not applicable), or ? (not determinable from the sources we examined).

Findings. Table 1 supports four observations.

1. **Sleep is the most common treatment.** Of the 14 works that evaluate I/O (excluding the two MPI communication benchmarks and the generic study of Tseng et al.), 6 use `sleep`, 3 use a compute phase they do not describe, and 2 have no compute phase. Only 3 works (Jain et al., Tarraf et al., and IMB-IO) run the I/O against a computation that can be slowed down by it.
2. **Sleep-based studies report I/O-side metrics.** Five of the six sleep-based works report only I/O time or bandwidth. DLIO reports end-to-end throughput, but with a sleeping compute phase it cannot register a slowdown. Some works state explicitly that the reported time "excludes the computation time between the I/O operations" [24]. None of

Work	Stack	Compute phase	Described	App-side cost	Cores stated	Real app
h5bench [13]	HDF5	sleep	✓	×	?	×
Tang et al. '19 [24]	HDF5 async	unspecified "simulated"	×	×	?	×
Tang et al. '22 [25]	HDF5 async	sleep	✓	×	○	○
Ravi et al. '23 [20]	HDF5 async	sleep (30 s)	✓	×	○	?
Tang et al. '18 [23]	PDC	sleep	✓	×	✓	×
Tang et al. '19b [22]	PDC	unspecified	×	×	✓	○
Skel [14]	ADIOS	none (I/O-only skeleton)	n/a	×	n/a	×
`adios_iotest` [1]	ADIOS2	sleep or busy loop	✓	×	n/a	×
DLIO [2, 4]	DL loaders	sleep (GPU-profiled)	✓	○	?	×
Harmonia [11]	burst buffer	unspecified math kernels	○	?	?	✓
Gossman et al. '21 [7]	VeloC / POSIX	none (flush timed alone)	n/a	×	?	×
Jain et al. '22 [10]	HDF5 async	real (Flash-X)	✓	✓	?	✓
Tseng et al. '21 [27]	generic	CPU/memory stressors	✓	✓	?	✓
Tarraf et al. '24 [26]	various	real (WaComM++, HACC-IO)	✓	✓	?	✓
IMB-IO [9]	MPI-IO	calibrated CPU kernel	✓	✓	n/a	×
COMB [12]	MPI comm.	busy work	✓	✓	n/a	×
Denis & Trahay [3]	MPI comm.	busy work	✓	✓	n/a	×

Table 1. How surveyed evaluations treat the computation phase. Upper block: works whose compute phase cannot slow down (sleep), is unspecified, or is absent. Lower block: works whose compute phase consumes resources. ✓ yes, ○ partially or as a caveat, × no, n/a not applicable, ? not determinable from the sources examined.

the works in the upper block reports the slowdown of the compute phase.

3. **Benchmark tools make sleep the default.** h5bench, `adios_iotest`, and DLIO all expose the compute phase as a sleep duration. Studies that build on these tools inherit this choice, often without stating it.
4. **Evaluations with real computation find smaller benefits and identify contention as the cause.** All works in the lower block report application-side costs. In addition, the works that evaluate I/O attribute the reduced benefit to competition for CPU time, memory bandwidth, or the network [10, 26, 27].

Summary #1

Sleep is the most common way to emulate computation in the evaluation of asynchronous I/O, and the most popular benchmark tools use it by default. Sleep-based studies report I/O-side metrics only, so they cannot observe a slower computation. The few evaluations that use real computation report smaller benefits and attribute the difference to contention.

4 Evaluation

Scope

All results in Section 4.7 are measured on a single node, with one run per configuration. PDC is measured with one server in the main, ratio, size, and GPU experiments, but not at 48 GiB per step, not at compute-to-I/O ratios above 1.25, and not in the core-allocation experiment (beyond the SMT-sibling placement of its clients). Its asynchronous run with the halo exchange did not complete (Section 4.3).

In this section, we describe our research questions, the machine and software used in our experiments, and the asynchronous I/O implementations we evaluate. Sections 4.4 to 4.6 then describe the compute surrogates, the workload, and the metrics, and Section 4.7 presents the results.

4.1 Research questions

Our evaluation is guided by four research questions (RQs):

- **RQ1:** How much longer does an application run than a sleep-based evaluation predicts, when its computation consumes CPU time, memory bandwidth, or network bandwidth?
- **RQ2:** Where does the cost of interference appear, as a slower computation or as exposed I/O, and which resource is responsible for each asynchronous I/O implementation?
- **RQ3:** How do conclusions drawn from sleep-based kernels compare with those from a mini-application that combines computation, memory traffic, and communication?
- **RQ4:** How sensitive are the results to the core allocation and to whether the computation runs on the CPU or the GPU?

4.2 Machine setup

All experiments run on a single dual-socket node. The hardware is described in Table 2, and the software versions and configuration are described in Table 3.

Process placement. Unless stated otherwise, we run 48 MPI ranks. Each rank is bound to one hardware thread on a distinct physical core, and the ranks alternate between the two sockets. Background I/O threads inherit the binding of their rank, and therefore share its hardware thread. We call this configuration *fully subscribed*. For RQ4, we also evaluate two other placements. In the *SMT sibling* placement, each rank is bound to both hardware threads of its core. In the *core reserved* placement, we run 24 ranks, each bound to two physical cores on the same socket. We also ran an unpinned placement, in which the operating system decides where each thread runs, but we exclude it. Without pinning, threads were placed differently during calibration and during the measurement, so the compute phases reached only 30% to 75% of their target length.

Component	Configuration
CPU	2× Intel Xeon Gold 5418Y (Sapphire Rapids), 24 cores each, 2.0 GHz base; 48 cores, 96 hardware threads (SMT on)
Caches	48 KiB L1d and 2 MiB L2 per core; 45 MiB L3 per socket
Memory	256 GB, two NUMA domains (128 GB each); inter-node distance 21
GPU	NVIDIA RTX 5000 Ada Generation, 32 GB, PCIe Gen4 ×16
Storage	4× SanDisk PC SN8000S 4 TB NVMe SSDs (PCIe Gen4 ×4 each) in Linux MD RAID-0 (512 KiB chunk), 16 TB
File system	ext4 (`stripe=512`, `relatime`)
Page cache	bounded per run by a cgroup v2 memory limit (footprint + 24 GiB); `vm.dirty_ratio`=20, `dirty_background_ratio`=10
Other	transparent huge pages `madvise`; no other user jobs during runs

Table 2. Hardware configuration of the test node.

The GPU experiments use 4 ranks that share the single GPU. The binding lists are constructed in Listing 14, lines 14-21.

Storage behavior. Due to the 256 GB of memory on our node, the page cache could absorb most writes and turn the I/O into a memory copy. In contrast, the client of a parallel file system can cache only a small part of a checkpoint. For this reason, we run each experiment in a cgroup v2 scope whose memory limit is the memory footprint of the benchmark plus 24 GiB. Because page-cache pages are charged to the cgroup, this limit bounds the cache available to a run. Once the cache is full, writers stall in memory reclaim until dirty pages have been written back to the RAID array. It is important to note that dirty-page throttling itself remains global, at 20% and 10% of the memory, because the `io` controller is not delegated to user cgroups on this system. In addition, no backend calls `fsync` after each step. The HDF5 asynchronous VOL executes file flushes synchronously by design, so per-step durability would make it blocking. Every run starts with an empty output directory after a `sync`, as shown in Listing 14, lines 40-44, and the memory limit is computed in Listing 15, lines 42-45. As a result, small outputs of up to tens of gigabytes per run complete largely at memory speed, as they would with a burst buffer, while larger outputs are limited by write-back. The size sweep in Section 4.7.6 covers both regimes.

4.3 Asynchronous I/O implementations

Each backend has a blocking and an asynchronous mode behind a common interface, `io_backend`, shown in Listing 1, lines 25-37. All runs initialize MPI with `MPI_THREAD_MULTIPLE` (Listing 2, line 104). We describe each backend below.

- **POSIX.** The blocking mode writes with `pwrite` to a shared file per step, at the offset of the rank. The asynchronous mode copies the buffer to a staging buffer and writes it with POSIX AIO (`aio_write`), which is executed by a glibc helper thread. The write path is shown in Listing 6, lines 59-90, and the completion in Listing 6, lines 44-57.
- **MPI-IO.** The blocking mode writes one shared file per step with the collective `MPI_File_write_at_all`. The asynchronous mode writes a staged copy of the buffer with the non-blocking `MPI_File_iwrite_at_all` (Listing 7, lines 60-77). ROMIO uses collective buffering with 8 aggregators on the node (`cb_config_list=*:8`), which matches the ADIOS2 setting. Because a non-blocking collective write only advances inside MPI calls, the asynchronous mode enables the progress thread of MPICH (`MPIR_CVAR_ASYNC_PROGRESS=1`). Similar to the other background engines, this thread inherits the binding of its rank.
- **HDF5.** Both modes write one shared file with a new dataset per step, using collective `H5Dwrite` and collective metadata. The asynchronous mode uses `H5Dwrite_async` with event sets, which are executed by one Argobots background thread per process with write-buffer copying enabled [25]. Following the guidance of the connector for periodic output [8], we set `HDF5_ASYNC_EXE_DCLOSE=1`, so that background execution starts when the dataset is closed instead of after an idle-detection period. The write path is shown in Listing 8, lines 53-81, the wait in Listing 8, lines 44-51, and the connector selection in Listing 14, lines 27-36.
- **ADIOS2.** The blocking mode uses the BP5 engine with `AsyncWrite` disabled, and the asynchronous mode uses `AsyncWrite=Guided`, in which the compute phase is bracketed with computation-block hints [6]. Both modes use two-level shared-memory aggregation with 8 aggregators. With the default of one aggregator per node, a single process writes all of the data of the node, which limits BP5 to about a third of the bandwidth of POSIX on our single node. The engine setup is shown in Listing 9, lines 27-61, the per-step path in Listing 9, lines 63-73, the hints in Listing 9, lines 85-97, and the aggregation settings in Listing 15, lines 27-32.
- **PDC.** The PDC backend uses synchronous or asynchronous region transfers, with one PDC server per node on a reserved core [23] (Listing 10, lines 48-62). Following the PDC deployment model, we run 1 server on a reserved core and 47 client ranks on the remaining cores. The server cache is capped at 24 GiB, the same budget as the page cache of the other backends, and the cache is flushed before the timed run ends. Each client rank runs a Mercury progress thread during asynchronous transfers, which polls the network with a 1 ms pause between polls. The asynchronous run with the halo exchange hung in two attempts and was stopped after 30 minutes each time (Listing 15, lines 212-244), so PDC has no halo result.

A wait-path defect in the HDF5 asynchronous VOL. In early runs, asynchronous HDF5 was slower than synchronous HDF5 even with no computation, and the waits were quantized in multiples of one second. By tracing system calls, we found that `H5VL_async_request_wait`, which is invoked by `H5ESwait`, sleeps for a full second (`usleep(1000000)`)

Software	Version	Build configuration	Runtime configuration
Ubuntu / Linux	26.04 LTS / 7.0.0-29	distribution kernel	THP `madvise`; cgroup v2 memory limit per run
glibc	2.43	distribution	POSIX AIO: 1 helper thread (`aio_init`)
GCC	15.2.0	distribution	`-O2 -march=native` for the benchmark
MPICH	4.3.2	distribution, `ch4:ucx`, Hydra	`MPI_THREAD_MULTIPLE`; explicit per-rank binding; ROMIO `cb_config_list=*:8`, `cb_nodes=8`; async progress thread for non-blocking MPI-IO
UCX / hwloc	1.20.0 / 2.13.0	distribution	defaults
HDF5	1.14.6	parallel, thread-safe, no compression, Release	MPI-IO VFD; collective data and metadata writes
HDF5 async VOL	git `90fbba9` (2026-02-10)	`ENABLE_WRITE_MEMCPY=ON`; wait back-off patch (Sec. 4.3)	`HDF5_ASYNC_EXE_DCLOSE=1`; event sets, one step in flight
Argobots	1.2	default	one execution stream per process (async VOL)
ADIOS2	2.12.1	MPI on; SST, Python, Fortran, compression off; opt-in fsync patch (inactive)	BP5; `AggregationType=TwoLevelShm`, `NumAggregators=8`; `AsyncWrite=Guided` (async)
PDC	0.7-398 (git `1ad9b312`)	MPI on; Mercury 2.2.0, libfabric 1.18.0	`ofi+tcp`; 1 server on a reserved core; cache 24 GiB
OpenBLAS	0.3.34	`TARGET=SAPPHIRERAPIDS`, `USE_THREAD=0`	single-threaded DGEMM
CUDA	13.3 (nvcc 13.3.73)	`-arch=native` (sm_89)	driver 595.91.07; spin or blocking-sync scheduling
Python	3.14.4	NumPy 2.5.2, Matplotlib 3.11.1	analysis and plotting

Table 3. Software configuration: versions, build options, and runtime settings used in all experiments. Build options are set in Listing 12, lines 22-30 (HDF5), Listing 12, lines 32-41 (async VOL), and Listing 12, lines 43-56 (ADIOS2), and runtime settings in Listing 15, lines 27-32 and Listing 14, lines 27-36.

Surrogate	Resource exercised	Implementation	Lines
sleep (baseline)	none	`nanosleep`	189-192
spin	one core (ALU)	xorshift loop	25-32
DGEMM	CPU, caches	OpenBLAS, 256^2, 1 thread	38-44
STREAM	memory bandwidth	triad, 3×32 MiB/rank	50-58
halo	MPI, memory copies	3-D 6-face exchange	87-96
heat	CPU, memory, MPI	7-pt Jacobi, 128^3/rank	130-145
GPU, host blocks	GPU only	FMA kernel, blocking sync	16-24
GPU, host spins	GPU, one core	FMA kernel, spin sync	16-24

Table 4. Compute surrogates used in place of the computation phase. All are calibrated to the same standalone duration t_{comp}. Lines refer to Listing 3 (CPU) and Listing 4 (GPU).

whenever it fails to reacquire the global lock of HDF5 from the background thread. In addition, it polls the task status every 100 ms. This penalty applies only when the application waits while background I/O is still running. Because this happens more often under contention than with `sleep`, the unpatched connector would exaggerate the effect we study. We therefore reduce both intervals to 1 ms. With this change, asynchronous HDF5 without computation performs on par with synchronous HDF5. The patch is shown in Listing 16, lines 1-22, and it is applied by the build script (Listing 12, lines 32-41).

4.4 Compute surrogates

Table 4 lists the compute surrogates. Each surrogate performs a fixed amount of *work* rather than running for a fixed *time*. A busy loop bounded by time, similar to `sleep`, would end on schedule no matter how much CPU time it received, and therefore could not reveal interference. Before the measured steps, the work of each surrogate is calibrated with all ranks running concurrently and no I/O in flight, so that its standalone duration matches t_{comp}. The calibration is shown in Listing 2, lines 71-89, and it is timed by Listing 2, lines 53-63. Because the work is linear in the number of units, the calibration targets at most 2 s and scales the result to the full t_{comp}. The standalone time t_{comp}^{alone} is then measured directly (Listing 2, lines 148-155). As a result, differences in run time reflect interference rather than a change in the nominal compute length.

The STREAM triad [15] uses arrays that together exceed the L3 cache. The heat mini-app is a 3-D Jacobi stencil with halo exchange, and it stands in for a real simulation. Its sweep is shown in Listing 3, lines 130-145, and its ghost-face exchange in Listing 3, lines 104-128. The halo surrogate reuses the same six-neighbour exchange (Listing 3, lines 77-85) on a 3-D Cartesian decomposition (Listing 3, lines 64-71). For the GPU surrogates, all ranks share the single GPU, and the output field is copied to the host at the end of each compute phase (Listing 4, lines 43-50). The wait mode of the host thread is selected in Listing 4, lines 32-40. Fig. 1 shows the loop that all configurations share. The complete source code of the benchmark, its compute kernels, I/O backends, run scripts, library patches, and analysis scripts is reproduced in Appendix A.

4.5 Workload

The I/O pattern follows the write kernel of h5bench and VPIC-IO. In each timestep, every rank writes a contiguous block of a shared 1-D array of doubles. We vary the block from 16 MiB to 1 GiB per rank, which is 0.75 GiB to 48 GiB per timestep with 48 ranks, and we write 10 timesteps. As a result, a single run writes between 7.5 GiB and 480 GiB. The largest size

```
calibrate units so that kernel(units) takes t_comp
  (all ranks concurrently, no I/O); measure t_alone
backend->init()
for (s = 0; s < steps; s++) {
  a = now();
  backend->compute_begin();      /* ADIOS2 Guided hint */
  kernel_run(k, units);          /* fixed work, not time */
  kernel_output(k, buf);         /* GPU: D2H copy      */
  backend->compute_end();
  b = now();
  backend->issue(s, buf);        /* sync: write + wait;
                                    async: wait(s-1), then enqueue(s) */
  MPI_Barrier();                 /* I/O phase ends on all
     ↪ ranks */
  t_comp[s] = b - a;  t_io[s] = now() - b;
}
backend->wait(); backend->fini();  /* last step drains */
```

Figure 1. The epoch loop of `aio_bench`, simplified from Listing 2, lines 158-187 in the appendix. At most one asynchronous step is in flight, as in h5bench.

is measured only for the two fastest backends, POSIX and ADIOS2, because MPI-IO and HDF5 need 30 to 40 minutes per run at that size. The main experiments use 256 MiB per rank, which is 12 GiB per step and 120 GiB per run.

Unless stated otherwise, t_{comp} is set to $1.25\times$ the synchronous I/O time per step of an I/O-only run with the same backend, size, rank count, and number of steps. This I/O-only run is measured at the start of each experiment (Listing 15, lines 101-106). With this choice, a sleep-based run can hide the I/O completely, which is the configuration used in prior work [20, 25]. It is important to note that the ratio achieved in the experiments differs from 1.25, because the I/O time varies between runs. We therefore report the measured ratio for each backend in Section 4.7.4. In addition, each step ends with a barrier. Without it, ranks that finish their I/O early would charge the imbalance to the next compute phase, because communicating kernels wait for their neighbours.

Each configuration is run once. The main experiment is defined in Listing 15, lines 125-132, the size sweep in Listing 15, lines 135-142 and Listing 15, lines 189-196, and each configuration is executed by Listing 15, lines 60-77. Repeated runs are left to future work (Section 4.8). The only configurations that we ran twice are the I/O-only baselines, and they show how large the run-to-run variation is on this node. Across 44 pairs, the two runs differ by a median of 8%, by more than 23% for a quarter of them, and by up to 62%. This variation is mostly due to the different state of the page cache and of write-back between runs. Table 7 in the appendix maps every mechanism described in this section to its implementation.

4.6 Metrics

We use the following metrics, where n is the number of steps:

- **Run time relative to the sleep prediction.** This metric directly measures how much longer the application runs than a sleep-based evaluation predicts. For a surrogate k, the sleep run predicts an asynchronous run time of $T_{async}(\texttt{sleep}) + n\,(t^{alone}_{comp}(k) - t^{alone}_{comp}(\texttt{sleep}))$, which corrects for small differences in the calibrated compute length. We divide the observed asynchronous run time with k by this prediction (Listing 18, lines 222-245). Because it compares asynchronous runs only, this metric does not depend on the synchronous runs, whose I/O time varies between runs.
- **End-to-end speedup** $S = T_{sync}/T_{async}$ over all timesteps. This is the metric most prior work reports.
- **Cost attribution.** We split the asynchronous run time into a compute part $n\,t^{\text{async}}_{comp}$ and an exposed-I/O part $T_{async} - n\,t^{\text{async}}_{comp}$. The exposed-I/O part includes the waits for background I/O, the issue overhead, and the final drain. We further split the compute part into the compute time of the synchronous run, $n\,t^{\text{sync}}_{comp}$, and the inflation, $n\,(t^{\text{async}}_{comp} - t^{\text{sync}}_{comp})$. With perfect overlap and no interference, T_{async} would equal $n\,t^{\text{sync}}_{comp}$ plus one step of I/O. Any excess is charged to the compute side (inflation) or to the I/O side (exposed I/O).
- **Compute slowdown** $t^{\text{async}}_{comp}/t^{\text{sync}}_{comp}$. This is the compute phase of the asynchronous run relative to that of the synchronous run with the same configuration, in which no I/O runs during the computation. We prefer this metric to the standalone baseline t^{alone}_{comp}, because the turbo frequency of the cores depends on whether they idle between compute phases, which shifts the standalone time by up to about 6% in our measurements.
- **Exposed I/O**, which is the I/O time the application still observes per step in asynchronous mode, relative to the synchronous I/O time.

The per-step timings are reduced over ranks in Listing 2, lines 189-197, and the metrics are computed from the resulting CSV records in Listing 18, lines 88-113.

4.7 Results

In this section, we present our measurements. We first characterize the I/O cost of each backend without computation. We then answer RQ1 to RQ4, and lastly we examine how sensitive the results are to the compute-to-I/O ratio and to the output size.

4.7.1 I/O-only baselines

Before adding computation, we measure each backend writing 5 timesteps with no compute phase at all. Table 5 shows the results for 48 ranks, with 2 runs per configuration. These runs determine the I/O cost that the compute surrogates later try to hide, and they show that the backends differ by an order of magnitude.

Cache-absorbed and write-back regimes. At 0.75 GiB per step, POSIX and ADIOS2 write 10.9 and 9.8 GB/s, respectively. This is memory speed, because the whole run fits in the page cache. At 48 GiB per step (240 GiB per run), their bandwidth drops to 6.3 and 6.5 GB/s, which is the rate at which the RAID array absorbs write-back. With 10 instead of 5 steps at 12 GiB per step, POSIX sustains only about 1.6 to 2.0 GB/s in our spot checks, because a larger fraction of each run waits for write-back. For this reason, every later experiment measures its own I/O-only baseline with the same size and number of steps (Listing 15, lines 101-106) instead of reusing these numbers.

Collective I/O limits MPI-IO and HDF5. MPI-IO and HDF5 both write through the collective buffering of ROMIO with 8

Backend	GiB/step	Sync (s)	Async (s)	Async/sync	Sync GB/s
POSIX	0.75	0.07	0.08	1.06	10.9
	3	0.29	0.26	0.89	10.9
	12	1.26	1.77	1.41	10.3
	48	8.23	4.28	0.52	6.3
MPI-IO	0.75	0.44	0.79	1.80	1.8
	3	1.77	3.65	2.06	1.8
	12	8.69	16.25	1.87	1.5
	48	36.87	68.85	1.87	1.4
HDF5	0.75	0.38	0.42	1.10	2.1
	3	2.40	1.80	0.75	1.3
	12	10.69	5.84	0.55	1.2
	48	45.54	30.43	0.67	1.1
ADIOS2	0.75	0.08	0.10	1.24	9.8
	3	0.37	0.32	0.87	8.7
	12	1.85	2.61	1.40	6.9
	48	7.99	8.25	1.03	6.5

Table 5. Measured I/O-only time per step (48 ranks, 5 steps, no computation; mean of the repetitions). Async/sync below 1 means the asynchronous mode is faster even without computation to overlap.

aggregators. They reach only 1.5 and 1.2 GB/s at 12 GiB per step, which is roughly a seventh of the bandwidth of POSIX at the same size. In addition, HDF5 has the largest fixed cost per step, 0.38 s at 0.75 GiB compared to 0.07 s for POSIX. This cost is dominated by the collective creation of datasets and by metadata operations.

Asynchronous mode without computation. With nothing to overlap, an ideal asynchronous mode would match the synchronous one. However, POSIX AIO and the HDF5 asynchronous VOL are faster at large sizes, with an async/sync ratio of 0.52 and 0.55, respectively, because the write of one step overlaps the buffer copy and setup of the next. The ratio of ADIOS2 ranges from 0.87 to 1.40. In contrast, nonblocking MPI-IO is 1.80 to 2.06× *slower* than blocking MPI-IO at every size. This is consistent with contention between the asynchronous progress thread of MPICH and the application thread, which share a hardware thread under our default binding. The core-allocation experiment in Section 4.7.7 supports this explanation.

4.7.2 Run time compared to the sleep prediction (RQ1)

In Fig. 2, we compare the observed run time of each asynchronous run with the run time that the `sleep` run predicts for the same compute length (Section 4.6). In 21 of the 24 configurations, the application runs at least 5% longer than `sleep` predicts, and up to 81% longer. The effect is largest for PDC, which runs 26% to 81% longer with every working surrogate, even though its server runs on a reserved core (Section 4.7.7). Among the backends whose engine runs inside the application, the effect is largest for the HDF5 asynchronous VOL, which runs 20% to 52% longer with every working surrogate. ADIOS2 runs 5% to 26% longer, and MPI-IO up to 14% longer. POSIX AIO is the least affected. With spin and DGEMM, its run time matches the `sleep` prediction, while with STREAM, heat, and halo it runs 16%, 12%, and 26%

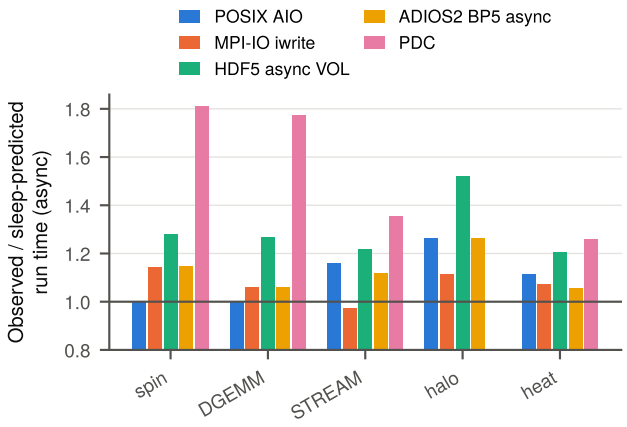


Figure 2. Observed run time of the asynchronous run with each compute surrogate, divided by the run time that the `sleep` run predicts for the same compute length (120 GiB per run, 48 ranks; PDC: 47 clients and 1 server). Values above 1 mean the application runs longer than a `sleep`-based evaluation predicts. PDC has no halo result, because its asynchronous halo run did not complete.

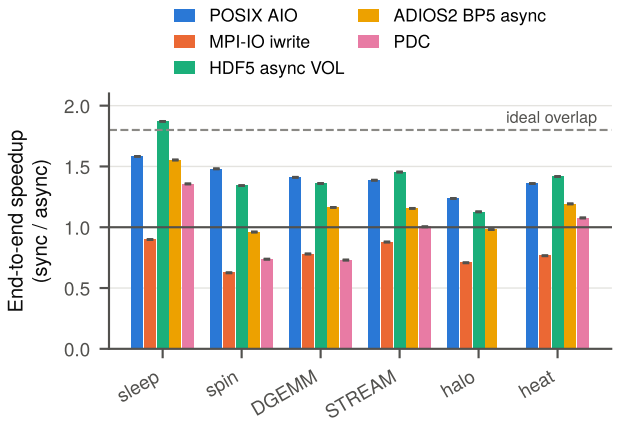


Figure 3. End-to-end speedup of asynchronous over synchronous I/O for each compute surrogate (same runs as Fig. 2). The dashed line is the ideal $(t_{comp} + t_{io})/\max(t_{comp}, t_{io})$ for $t_{comp} = 1.25\, t_{io}$.

longer. It is important to note that each configuration was run once, so differences of a few percent are within the run-to-run variation of this node (Section 4.5).

Fig. 3 shows the same runs as end-to-end speedups, which is the metric most prior work reports. For the three backends where asynchronous I/O helps, `sleep` yields the largest speedup of all surrogates: 1.58× for POSIX AIO, 1.87× for the HDF5 asynchronous VOL, and 1.55× for ADIOS2. Every surrogate that performs work lowers this benefit. The speedup of POSIX ranges from 1.48× with spin to 1.24× with halo, the speedup of HDF5 from 1.45× to 1.13×, and the speedup of ADIOS2 from 1.19× to 0.96×. In addition, MPI-IO is slower than its blocking mode in every configuration, from 0.90× with `sleep` to 0.63× with spin. However, speedups divide two separate runs, and they inherit the variation of the synchronous run. In our main experiment, the synchronous run with `sleep` had the slowest synchronous I/O of all surrogates for every backend (Section 4.8). This inflates the `sleep`

speedups, which is why we base RQ1 on the comparison of asynchronous run times in Fig. 2.

4.7.3 Where the cost appears (RQ2)

In Figs. 4 and 5, we attribute the lost benefit to the compute side or to the I/O side. The backends fall into two groups.

Compute-side interference (POSIX, HDF5, ADIOS2, and PDC). For these backends, little I/O remains exposed, and the loss appears as a longer compute phase. The size of the effect depends on the design of the background engine:

- **POSIX AIO** is the least intrusive. Spin, DGEMM, and STREAM slow down by at most 2%, which is consistent with its single glibc helper thread spending most of its time blocked on write-back rather than running. Only the communicating surrogates are affected, with a slowdown of 1.28× for halo and 1.11× for heat.
- **The HDF5 asynchronous VOL** slows down *every* working surrogate, from 1.23× for heat to 1.60× for halo. This includes the spin loop, which only uses the arithmetic units (1.34×). The Argobots thread executes HDF5 operations on the hardware thread of the rank and competes for it directly.
- **ADIOS2 BP5** in Guided mode behaves similarly, with a slowdown of 1.26× for spin and 1.65× for halo. In this mode, ADIOS2 deliberately writes during the computation blocks announced by the application, which is exactly when the computation needs the core.
- **PDC** slows down every working surrogate the most, from 1.29× for heat to 1.90× for spin, although its server runs on a separate core. The cause is on the client side: the progress thread of each client rank polls the network on the hardware thread of the rank. As a result, the speedup with spin and DGEMM drops below 1 (0.74× and 0.73×), compared to 1.36× with `sleep`.

For POSIX, HDF5, and ADIOS2, the halo exchange is the most sensitive surrogate. Therefore, contention reaches the application through MPI and shared-memory traffic as well as through the CPU.

I/O-side interference (MPI-IO). With non-blocking collective MPI-IO, the compute phase is undisturbed, with a slowdown between 0.96 and 1.04. However, the background write barely advances while the application computes. The application then waits for it at the next step, and the exposed I/O per step exceeds the synchronous I/O time (Fig. 5b). Over the run, the exposed I/O is 117 s with `sleep` and 146 s with spin.

In both groups, the run time that the application observes is longer than a `sleep`-based evaluation predicts. A study that reports only the observed I/O time would see the POSIX and HDF5 results as nearly complete hiding (Fig. 5b) and would miss the inflated compute phase entirely.

		Speedup			Longer than sleep predicts	
Backend	t_{comp}/t_{io}	Model	Sleep	Heat	Heat	Worst
POSIX	1.72	1.58	1.58	1.36	+12%	+26% (halo)
MPI-IO	1.12	1.89	0.90	0.77	+7%	+14% (spin)
HDF5	1.10	1.91	1.87	1.42	+20%	+52% (halo)
ADIOS2	0.99	1.99	1.55	1.19	+5%	+26% (halo)
PDC	2.28	1.44	1.36	1.08	+26%	+81% (spin)

Table 6. Speedup predicted by the overlap model of Ravi et al. [20] for each backend's measured ratio t_{comp}/t_{io} (no overhead), and measured with `sleep` and with the heat mini-app. The last two columns give how much longer the asynchronous run with the heat mini-app, and with the least favourable surrogate, takes than the `sleep` run predicts for the same compute length (Section 4.6).

Summary #2

With real computation, the application runs longer than a `sleep`-based evaluation predicts in 21 of 24 configurations, by up to 81%. For POSIX AIO, the HDF5 asynchronous VOL, and ADIOS2, the cost appears as a slower computation, and the halo exchange is the most sensitive surrogate. For non-blocking MPI-IO, the computation is undisturbed, but the I/O does not progress during it and is exposed later.

4.7.4 Sleep-based estimates and a mini-application (RQ3)

In Table 6, we compare the benefit predicted by the analytical model of Ravi et al. and by `sleep` with the benefit that the heat mini-app obtains. The heat mini-app is the closest stand-in for a real simulation in our set. The model is evaluated for the measured compute-to-I/O ratio of each backend, which ranges from 0.99 for ADIOS2 to 1.72 for POSIX. For POSIX and HDF5, the `sleep` speedup is close to the model (1.58 versus 1.58, and 1.87 versus 1.91). This means that a `sleep`-based evaluation mostly validates the model against itself. In contrast, the heat mini-app obtains a lower speedup with every backend, and its asynchronous run takes 12% (POSIX), 7% (MPI-IO), 20% (HDF5), and 5% (ADIOS2) longer than the `sleep` run predicts. The least favourable surrogate raises this gap to up to 52%. This mirrors, at kernel scale, the gap between the 4.5 to 4.9× I/O-time speedups reported with `sleep` [25] and the 1.5× end-to-end speedup observed in Flash-X [10].

4.7.5 Sensitivity to the compute-to-I/O ratio

Sleep-based studies choose t_{comp} so that it just covers the I/O. A single ratio could therefore favor or penalize either kind of surrogate. For this reason, we sweep t_{comp}/t_{io} from 0.25 to 3 (Listing 15, lines 155-163), as shown in Fig. 6. In the regime used by prior work ($t_{comp} \geq t_{io}$), `sleep` exceeds DGEMM and STREAM at every ratio for POSIX, HDF5, and ADIOS2. On average, the `sleep` speedup is higher by 13% and 13% for POSIX, 30% and 29% for HDF5, and 21% and 31% for ADIOS2. For ADIOS2, the speedup with DGEMM and STREAM stays near 1× across the whole range, so no choice of compute length recovers the benefit that `sleep` shows. MPI-IO stays below 1× everywhere and approaches break-even only for `sleep` at the largest ratios. Therefore,

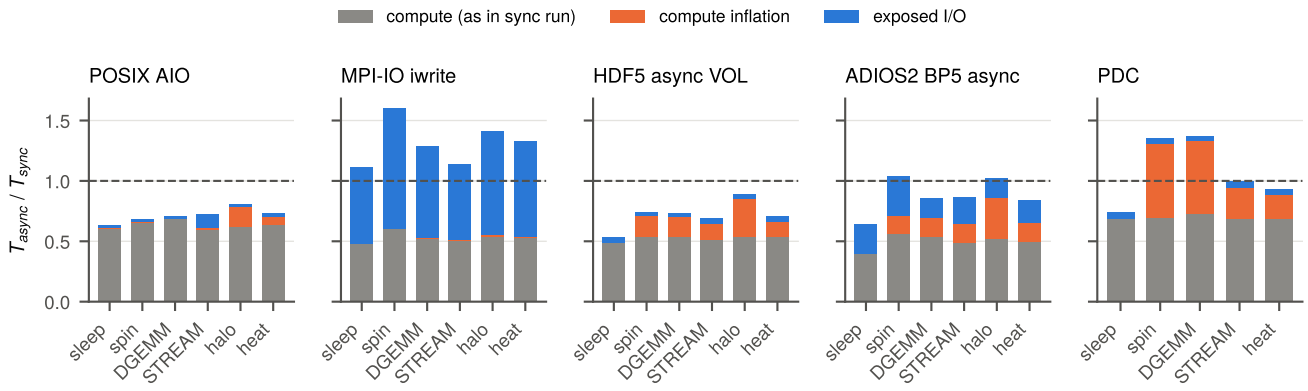


Figure 4. Breakdown of the time of an asynchronous run, relative to the synchronous run with the same surrogate. The dashed line marks the synchronous run time, and values below 1 mean that asynchronous I/O helps. Gray: compute time as in the synchronous run. Orange: compute inflation caused by background I/O. Blue: exposed I/O. PDC has no halo result.

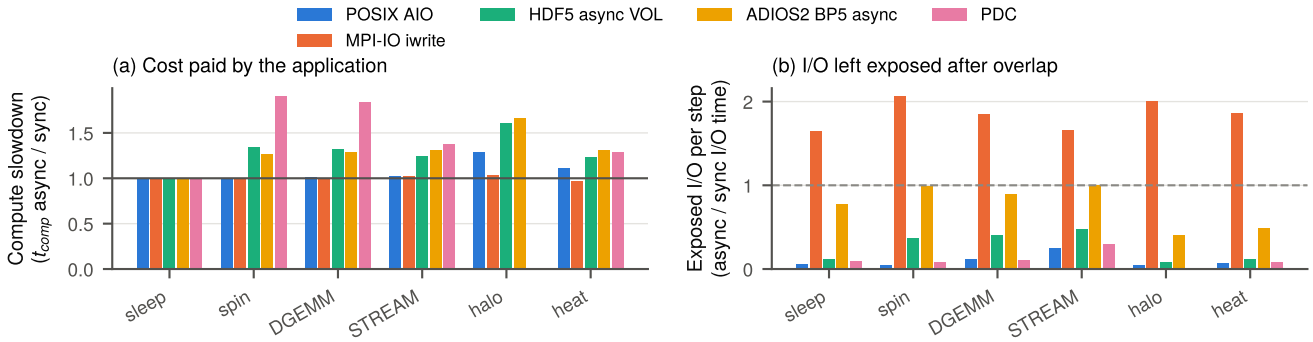


Figure 5. (a) Slowdown of the compute phase of the asynchronous run relative to the synchronous run. (b) I/O still exposed per step in the asynchronous run, relative to the synchronous I/O time per step (dashed line: no hiding at all).

the gap between `sleep` and working surrogates is not an artifact of the ratio used in the main experiment. At small ratios, the curves are noisier because the runs are short, and HDF5 and POSIX can exceed the ideal, because their asynchronous modes also pipeline consecutive writes (Table 5).

For PDC, we measure the ratios 0.25, 0.5, and 1 (Listing 15, lines 300-308) and take the point at 1.25 from the main experiment. Each PDC run writes 120 GiB through a single server, and ratios above 1.25 would take more than 10 minutes per run. The PDC curves show how the cost of the interference depends on the ratio. The compute slowdown with DGEMM stays at about 1.85× at every ratio, but its effect on the run time does not. At 0.25 and 0.5, the compute phase is shorter than the I/O even when it is slowed down, so the slowdown is hidden behind the I/O. In this case, DGEMM reaches its best speedup of 1.09× at a ratio of 0.50, close to the `sleep` speedup at the same ratio. Once the slowed compute phase is longer than the I/O, it lies on the critical path, and the speedup with DGEMM drops below 1×, while `sleep` reaches its peak of 1.38× at a ratio of 1.00. At ratios of 1 and 1.25, the `sleep` speedup is higher than the speedup with DGEMM by 83% and higher than with STREAM by 35%, and STREAM never exceeds 1.02×. Therefore, for PDC, the regime that `sleep`-based studies choose, where the computation covers the I/O, is exactly the regime in which the interference becomes visible in the run time.

4.7.6 Effect of output size

In Fig. 7, we vary the output per step from 0.75 GiB (7.5 GiB per run) to 12 GiB (120 GiB per run), as defined in Listing 15, lines 135-142. At 0.75 GiB, the output fits in the page cache, the I/O and compute phases are short, and the results are noisy. In this case, POSIX AIO is even slower than its blocking mode with `sleep` (0.96×). Once the I/O is substantial, the speedup with `sleep` increases with the size. For POSIX, it increases from 1.66× at 3 GiB to 1.78× at 12 GiB, and for HDF5 from 1.47× to 1.93×. However, the working surrogates do not keep pace. At 12 GiB, the heat mini-app reaches 1.34× with POSIX and 1.37× with HDF5. For HDF5, the gap between `sleep` and the heat mini-app grows with the output size, while for POSIX it is present from 3 GiB on. ADIOS2 varies without a clear trend, and MPI-IO stays below 1× at every size.

At the largest size, 48 GiB per step (480 GiB per run), which we measured for POSIX and ADIOS2, the gap persists. For POSIX, `sleep` reaches 1.77×, while DGEMM, STREAM, and the heat mini-app reach 1.60, 1.32, and 1.40×. For ADIOS2, `sleep` reaches 1.44×, while the working surrogates reach only 1.02 to 1.08×, so asynchronous I/O provides almost no benefit to them. Therefore, the overstatement of `sleep` does not disappear when the I/O dominates the run time.

PDC (Listing 15, lines 288-297) shows the most consistent picture of all backends. The speedup with `sleep` is nearly

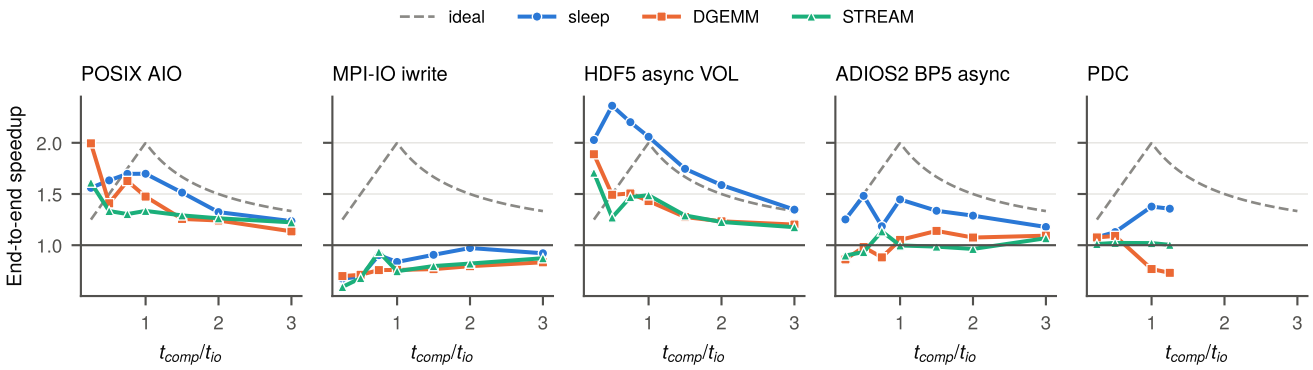


Figure 6. End-to-end speedup as a function of t_{comp}/t_{io} (12 GiB per step, 10 steps), where t_{io} is the synchronous I/O time per step of the same configuration. The dashed line is the ideal $(r+1)/\max(r,1)$. PDC (47 clients and 1 server) is measured from 0.25 to 1.25 only.

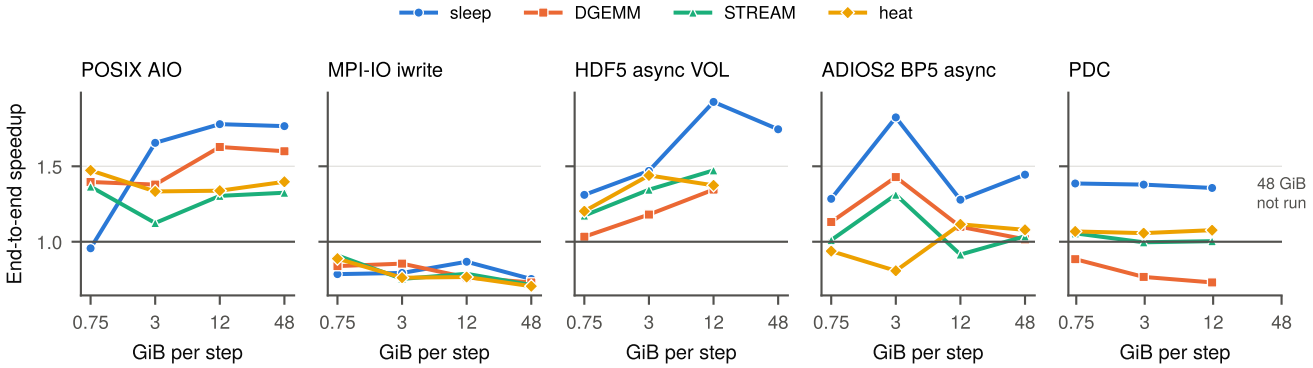


Figure 7. End-to-end speedup as a function of the output size per step (48 ranks, 10 steps, and $t_{comp} = 1.25\,t_{io}$ for each size). The 48 GiB point (480 GiB per run) is measured for POSIX and ADIOS2 only. PDC uses 47 clients, so its sizes are 0.73, 2.9, and 11.75 GiB per step.

constant, from 1.39× at 0.73 GiB to 1.36× at 11.75 GiB, which is consistent with a single server performing all writes at a similar rate for every size. The working surrogates stay well below it at every size, including the smallest one, where the other backends are noisy. STREAM reaches at most 1.06×, and the heat mini-app at most 1.08×. With DGEMM, the asynchronous run is slower than the synchronous run at every size, and the speedup falls from 0.88× to 0.77× and 0.73× as the size grows. We did not run PDC at 47 GiB per step, because a single run would take about 90 minutes through one server.

Summary #3

The advantage of `sleep` over working surrogates persists across compute-to-I/O ratios from 0.25 to 3 and across output sizes up to 480 GiB per run. For HDF5, it grows with the output size. For PDC, it is present at every size we measured, and it appears as soon as the computation is long enough to cover the I/O, which is the regime that `sleep`-based evaluations target.

4.7.7 Core allocation (RQ4)

In Fig. 8, we vary where the background engine can run (Listing 15, lines 166-177, with the bindings in Listing 14, lines 14-21). Sleep cannot respond to this choice, because a sleeping thread leaves its core idle anyway. In contrast, the working surrogates are affected by the placement:

- **HDF5.** Giving the Argobots thread a second hardware thread raises the speedup with DGEMM from 1.32× (fully subscribed) to 1.57× (SMT sibling) and 1.55× (core reserved), and the speedup with STREAM from 1.44× to 1.70×. The compute slowdown with DGEMM falls from 1.31 to 1.10 accordingly. However, the gap to `sleep` (1.89×) remains.
- **MPI-IO.** Reserving a core turns the loss into a small gain. The speedup with `sleep` rises from 0.93× to 1.13×, and with DGEMM from 0.75× to 1.06×. This supports the explanation that the progress thread, starved of CPU time when it shares the hardware thread of the rank, stalls nonblocking MPI-IO.
- **POSIX AIO** is largely insensitive to the three pinned placements, as expected from its low CPU demand.
- **ADIOS2** shows no consistent gain from extra hardware threads, and its `sleep` speedup is highest when fully subscribed. With a single run per configuration, we cannot tell whether this reflects a different aggregation behavior of ADIOS2 under other placements or run-to-run variation.
- **PDC.** The server of PDC already runs on a reserved core, but its clients do not. While an asynchronous transfer is in flight, each client runs a Mercury progress thread that polls the network with a 1 ms pause between polls, and we measured it at 40 to 50% of a core. When it shares the hardware thread of the rank, the compute phase with spin slows down by 1.89× and the run takes 613 s. When we give each client its SMT sibling (Listing 15, lines 250-268), the slow-

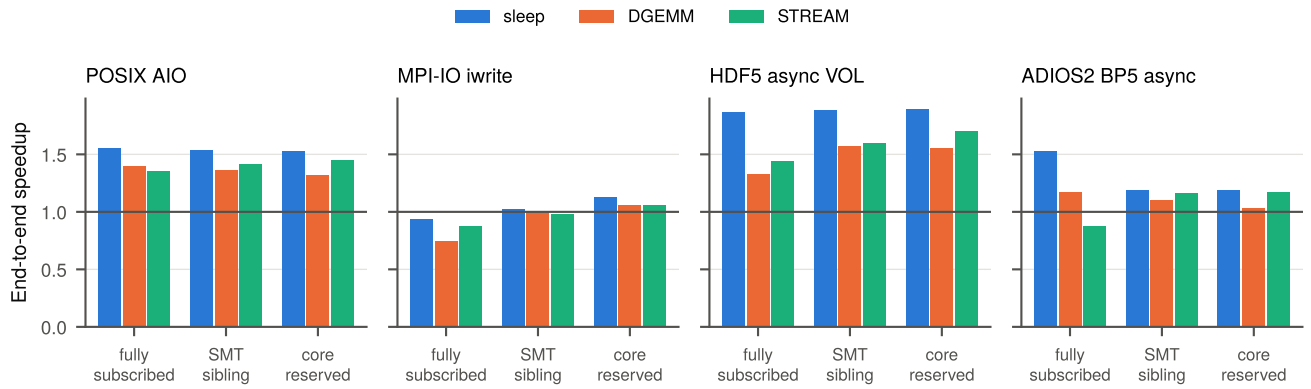


Figure 8. End-to-end speedup under three process placements (12 GiB per step): one hardware thread per rank shared with the background engine (fully subscribed), both hardware threads of the core (SMT sibling), and two physical cores per rank with 24 ranks (core reserved).

down falls to 1.02×, the run takes 348 s, and the run time is within 2% of the `sleep` prediction for the same placement. With DGEMM, the slowdown falls from 1.85× to 1.30×, but the run still takes 32% longer than `sleep` predicts, because the progress thread and DGEMM still share the core. Therefore, reserving cores for the server of a DMS does not remove the interference when the client library itself needs CPU time.

The advice of the HDF Group to reserve a core for the background thread [21] is supported by our results. However, it costs compute resources, 24 instead of 48 ranks in our configuration, and a `sleep`-based evaluation cannot show whether this trade-off is worthwhile.

4.7.8 GPU-resident computation (RQ4)

GPU-offloaded codes are where `sleep` is most often considered acceptable, because the host is assumed to be idle while the device computes [25]. In Fig. 9, we test this assumption with 4 ranks that share the GPU (Listing 15, lines 180-186, with the wait mode of the host selected in Listing 4, lines 32-40). Because the GPU idles at a low clock frequency, the GPU kernels are calibrated at their full length after a warm-up of several seconds (Listing 2, lines 71-89).

Our results largely support the assumption for the asynchronous run. Fig. 9b shows that, with a GPU kernel, the asynchronous run time of POSIX AIO matches the `sleep` prediction (0% with a blocking host and -1% with a spinning one), and HDF5 is within 7% of it. Non-blocking MPI-IO even runs faster than predicted. In contrast, the same 4 ranks running DGEMM on the CPU take 9% (POSIX) and 28% (HDF5) longer than the `sleep` prediction. Whether the host thread blocks or spins makes little difference in this configuration, because each rank is pinned to its own core and the background engine shares only the core of its rank. ADIOS2 is the exception, with run times 13% to 22% longer than predicted. However, its compute phase is short (about 0.7 s), so this result is the least certain.

However, the end-to-end speedups in Fig. 9a are still lower with GPU kernels than with `sleep`, for example 1.29× compared to 1.57× for POSIX and 1.24× compared to 1.45× for HDF5. This difference comes from the synchronous runs. After a `sleep` phase, the synchronous I/O took 2.64 s per step for POSIX, compared to 1.34 s after a GPU kernel, and 2.44 compared to 1.92 s for HDF5. This is the same effect we observed in the main experiment (Section 4.8). Therefore, for GPU-resident computation, `sleep` predicts the asynchronous run time well for most backends, but it still inflates the speedup by slowing down the synchronous baseline.

PDC (Listing 15, lines 278-285, with 1 server and 4 clients that share the GPU) confirms that its interference comes from the CPU. With a GPU kernel, the asynchronous run time is within -1% (blocking host) and 0% (spinning host) of the `sleep` prediction, and the compute phase slows down by only 0.96× and 1.02×. In contrast, the same 4 clients running DGEMM on the CPU slow down by 1.95× and take 81% longer than predicted, the largest excess in Fig. 9b. For PDC, the speedups with GPU kernels (1.38× and 1.33×) also match the `sleep` speedup (1.35×), unlike for POSIX and HDF5. The synchronous I/O takes 4.68 s per step after `sleep` and 4.54 s after a GPU kernel, so `sleep` barely slows down the synchronous baseline of PDC. With DGEMM, the speedup falls to 0.74×.

> **Summary #4**
>
> Reserving hardware threads for the background engine recovers part of the lost benefit for HDF5 and turns non-blocking MPI-IO from a loss into a small gain, at the cost of fewer compute ranks. For PDC, a reserved server core is not enough, because the client progress thread competes with the computation. With GPU-resident computation and pinned ranks, the host is mostly idle, and `sleep` predicts the asynchronous run time within a few percent for POSIX, HDF5, MPI-IO, and PDC, but not for ADIOS2. The same PDC clients running DGEMM on the CPU take 81% longer than predicted. Even in this case, `sleep` inflates the speedup by slowing down the synchronous baseline.

4.8 Discussion

In this section, we discuss when `sleep` is an acceptable stand-in for computation, the practical implications of our results, and the threats to their validity.

Our measurements show that `sleep` is a fair stand-in for computation only under narrow conditions. The background engine must have a low CPU demand, as POSIX AIO does. In addition, the computation must neither communicate nor com-

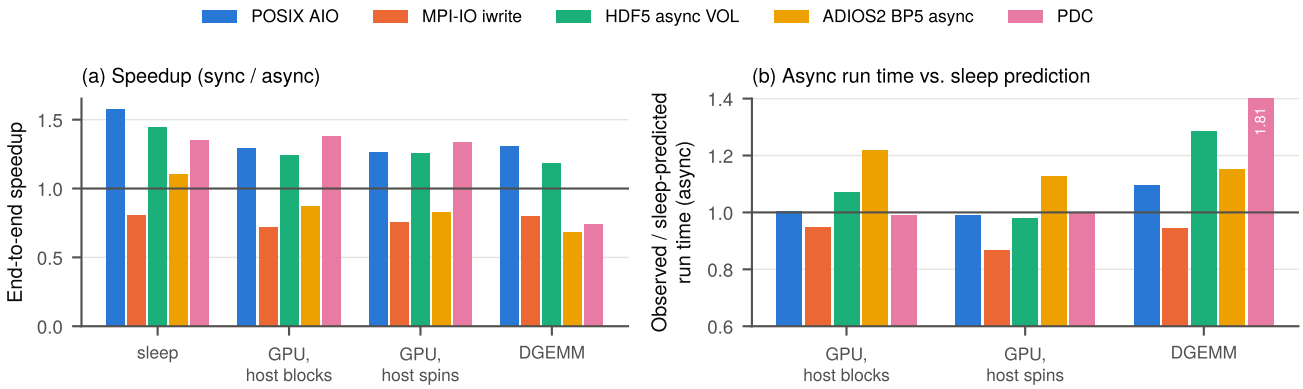


Figure 9. GPU-resident computation (4 ranks sharing one GPU, 4 GiB per step, 10 steps). "Host blocks" uses blocking synchronization, and "host spins" uses the spin-wait of CUDA. DGEMM runs on the CPU with the same 4 ranks. (a) End-to-end speedup. (b) Observed asynchronous run time divided by the run time that the `sleep` run predicts.

pete for the hardware thread of the engine, and the host thread must actually be idle. Even under these conditions, the halo exchange and the heat mini-app make the compute phase of POSIX AIO 1.28 and 1.11× longer. For engines that perform real work on the hardware thread of the application, such as the HDF5 asynchronous VOL and ADIOS2 in Guided mode, the application runs up to 52% and 26% longer than `sleep` predicts. The same holds for engines that need CPU time to progress, such as non-blocking MPI-IO, which runs up to 14% longer. Moving the server of the DMS to a reserved core is not sufficient either: PDC runs up to 81% longer, because its clients poll for progress on the cores of the application. Furthermore, the advantage of `sleep` persists across compute-to-I/O ratios and across output sizes up to 480 GiB per run.

Three practical observations follow from these results. First, the same contention appears in different places for different engines. It appears in the compute phase for POSIX, HDF5, and ADIOS2, and as exposed I/O for MPI-IO. Reporting only the I/O time hides the former, and reporting only the compute time hides the latter, while the end-to-end time captures both. Second, process placement matters more for asynchronous I/O than the literature suggests. Reserving hardware threads for the background engine recovers part of the loss for HDF5 and turns non-blocking MPI-IO from a loss into a small gain, but it costs compute resources. Third, GPU-resident computation is the case in which `sleep` comes closest to reality. With pinned ranks, the asynchronous run time with a GPU kernel matched the `sleep` prediction within a few percent for POSIX, HDF5, and MPI-IO, while the same ranks running DGEMM on the CPU ran up to 28% longer. However, ADIOS2 still ran longer than predicted, and `sleep` still inflated the speedups by slowing down the synchronous baseline.

Threats to validity. Our platform is a single node with local NVMe storage. It reproduces contention for cores and memory bandwidth, but not contention for the network links that MPI traffic and parallel file system traffic share. Our halo surrogate approximates this contention only with intra-node shared-memory transfers. In addition, we emulate a small client cache with a cgroup memory limit rather than with the client of a real parallel file system, and 4 ranks share one GPU in the GPU experiments, whereas production GPU nodes typically have one GPU per rank.

Every configuration was run once, and the I/O time on this node varies considerably between runs, with a median difference of 8% and up to 62% between two I/O-only runs (Section 4.5). It is important to note one systematic case. In the main experiment, the synchronous run with `sleep` had the slowest synchronous I/O of all surrogates for every backend: 9.9 s per step for POSIX, compared to 7.0 to 10.2 s for the other surrogates, and 3.0 s for ADIOS2, compared to 1.5 to 2.1 s. Because this run was the first one after the I/O-only baseline, we repeated the synchronous runs with spin and `sleep` in the opposite order, with spin first. The result did not change. The synchronous I/O after a `sleep` phase was again slower in every backend, by 17% for POSIX, 50% for MPI-IO, 21% for HDF5, and 98% for ADIOS2. Therefore, this is not an ordering effect. Instead, synchronous I/O appears to run slower after the cores have been idle, possibly because of the power states and clock frequency of idle cores, which we cannot inspect without administrator access. This is a second way in which `sleep` distorts an evaluation: besides hiding contention in the asynchronous run, it slows down the synchronous baseline and thereby inflates the speedup. For this reason, we base our main comparison on asynchronous run times, which do not depend on the synchronous runs (Section 4.6). Individual values of this comparison are also uncertain, and effects of a few percent, such as 5% for ADIOS2 with the heat mini-app, are within the run-to-run variation. Therefore, our conclusions rest on the pattern rather than on individual values. The application runs longer than `sleep` predicts in 21 of 24 configurations, and the difference has the same sign across backends, ratios, sizes, and placements, which run-to-run variation would not produce systematically.

Future work. We see two kinds of future work. The first strengthens the measurements in this paper:

- **Repeated runs.** Repeating each configuration would provide confidence intervals for every value and settle the single-run anomalies we note, such as ADIOS2 under different placements.
- **Idle-core effects.** Measuring core frequencies and power

states would identify why synchronous I/O runs slower after a `sleep` phase.

- **Multi-node systems and larger outputs.** Running on a multi-node system with a parallel file system would expose contention for the network links that MPI and file system traffic share, which a single node cannot. In addition, outputs at the terabyte scale of production checkpoints would show whether the effect continues to grow with the output size.
- **PDC with more servers and larger outputs.** We measured PDC with one server, in the main, ratio, size, and GPU experiments, but not at 47 GiB per step or at ratios above 1.25, because each run writes through a single server. Varying the number of servers, which would also make these runs affordable, and finding the cause of the hang with the halo exchange would complete its picture. In addition, a blocking or event-driven progress thread in the PDC client would likely remove most of the interference we measured.

The second kind opens new research directions:

- **Collective, independent, and single-rank I/O.** The interference may depend on how the data is written. Comparing collective I/O, independent I/O, and I/O funneled through a single rank would show which access strategy interferes least with each type of computation.
- **Sensitivity of data management systems to the computational kernel.** Our results suggest that each data management system (DMS) is sensitive to a different resource. For example, the HDF5 asynchronous VOL slowed down every working surrogate, while POSIX AIO mainly affected communicating ones. A systematic study of this sensitivity across compute-bound, memory-bound, and communication-bound kernels would characterize each DMS by the workloads it suits.
- **GPU data movement.** In our GPU experiments, the output is copied from the device to the host before it is written. The cost of this copy, and its interference with the background I/O, may depend on whether the GPU kernel is compute-bound or memory-bound, and on whether the data is moved to or from a running GPU kernel. In addition, GPUDirect Storage (GDS) writes directly from GPU memory to storage, which removes the host copy but may introduce its own interference with the GPU kernel and with the PCIe traffic of the application.

These directions lead to a practical question that clients of HPC systems ask, and to a research question that follows from it.

- **Client question.** *My workload is network-bound, compute-bound, or memory-bound, and I want to overlap its I/O. Which DMS will give me the largest speedup?* Our results show that the answer depends on the workload: for a communication-heavy code, POSIX AIO lost less benefit than the HDF5 asynchronous VOL and ADIOS2, while non-blocking MPI-IO required a reserved core to help at all.
- **Research question.** *Can a DMS take the characteristics of the client workload into account to improve the speedup of asynchronous I/O?* For example, a DMS could schedule background I/O when the computation is not using the resource the I/O needs, select its threading and aggregation strategy based on whether the workload is network-bound, compute-bound, or memory-bound, or reserve hardware threads only when the expected gain exceeds the loss of compute resources.

5 Conclusions and Recommendations

Sleep-based compute emulation answers a narrow question: *if the application left every node resource idle during its computation phase, how much of the I/O could be hidden?* This is a useful upper bound, but it is not the question practitioners need answered. In this paper, we surveyed 17 evaluations and benchmarks of asynchronous I/O and found that `sleep`, an unspecified compute phase, or no compute phase at all is the norm. We then measured five asynchronous I/O implementations with compute surrogates that consume resources. With these surrogates, the application ran longer than a `sleep`-based evaluation predicts in 21 of 24 configurations, by up to 81%. Depending on the implementation, the cost appeared as a slower computation or as I/O that did not progress during the computation. Based on these results and on the surveyed literature, we provide the following recommendations.

For authors evaluating asynchronous I/O.

1. **Treat sleep as an upper bound, and label it as such.** Results obtained with `sleep()` should be presented as the best-case overlap, not as the expected benefit for an application.
2. **Use compute surrogates that consume resources.** At a minimum, include a CPU-bound and a memory-bandwidth-bound surrogate calibrated to the same duration, as overlap benchmarks such as IMB-IO already do [9]. In addition, include a communication-bound surrogate, which was the most sensitive surrogate in our measurements.
3. **Report the end-to-end time and attribute it.** The figure of merit is the run time that the application observes. Report it, and break the difference from the ideal into compute-phase slowdown and exposed I/O. An I/O time that drops to near zero while the compute phase grows is not an improvement, and neither is an undisturbed compute phase followed by a long wait for I/O that never progressed. A sleep phase can reveal neither case.
4. **Report the core allocation explicitly.** State whether cores were reserved for background threads and how many ranks and threads ran per node. In addition, evaluate both fully subscribed and reserved configurations, because the placement changed our results considerably.
5. **Validate on at least one real application.** Kernel results should be checked against a full application. The gap between sleep-based I/O-time speedups and end-to-end speedups in real codes, such as Flash-X [10], shows why.
6. **Do not tune the compute phase to hide the I/O.** Choosing the emulated compute length so that the overlap is

complete guarantees the outcome. Instead, derive t_{comp} from measured application profiles, and sweep it to show where the benefit begins and ends.

7. **Repeat the measurements, and check the synchronous baseline.** I/O time can vary considerably between runs, as it did on our node, and a single synchronous run can inflate or deflate a speedup. In addition, `sleep` itself slowed down the synchronous I/O in our measurements, by 17% to 98%, which inflates `sleep`-based speedups further. Comparing asynchronous runs directly, as we do, reduces this sensitivity.
8. **Benchmark suites should offer more than sleep.** Tools such as h5bench [13] could provide selectable compute surrogates alongside the existing emulated-compute parameter.

For readers interpreting existing results.

1. **Read sleep-based speedups as upper bounds.** They show the overhead of the asynchronous mechanism without contention and the maximum attainable hiding, which is valuable, but they do not predict the speedup of an application.
2. **Check what is being measured.** Many studies report the observed I/O time with the emulated computation excluded. Such numbers cannot reveal a slower compute phase or I/O that stalls behind the computation, and they should not be equated with an end-to-end speedup.
3. **Consider the resource profile of the workload.** Sleep is a closer approximation when the host CPU is actually idle during the computation, as for GPU-offloaded codes [25]. In our measurements with pinned ranks, `sleep` predicted the asynchronous run time with a GPU kernel within a few percent for POSIX, HDF5, and MPI-IO, but not for ADIOS2. Even in this case, the speedup should be computed against a synchronous run with the real computation, because `sleep` slows down the synchronous baseline. Sleep is a poor approximation for CPU-bound, memory-bandwidth-bound, or communication-heavy codes running on fully subscribed nodes.
4. **Look for stated caveats.** Statements such as "in the absence of node-level oversubscription" or recommendations to reserve a core mark the boundary of the validity of a result.

Asynchronous I/O remains a promising technique, and the studies surveyed in this paper show that its mechanisms can be made efficient. Our argument is about the evidence, not about the technique. Ultimately, to know how much asynchronous I/O helps a real application, its evaluation must include a computation phase that actually computes.

References

[1] ADIOS2 Developers. adios_iotest: ADIOS2 I/O skeleton test utility. ADIOS2 v2.12.1 source, `source/utils/adios_iotest`, 2026.

[2] Argonne Leadership Computing Facility. DLIO benchmark source code. https://github.com/argonne-lcf/dlio_benchmark. Accessed October 2026.

[3] Alexandre Denis and François Trahay. MPI overlap: Benchmark and analysis. In *45th International Conference on Parallel Processing (ICPP)*, 2016.

[4] Hariharan Devarajan, Huihuo Zheng, Anthony Kougkas, Xian-He Sun, and Venkatram Vishwanath. DLIO: A data-centric benchmark for scientific deep learning applications. In *21st IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2021.

[5] Matthieu Dorier et al. Damaris: Addressing performance variability in data management for post-petascale simulations. *ACM Transactions on Parallel Computing*, 3(3):1-43, 2016.

[6] William F. Godoy, Norbert Podhorszki, Ruonan Wang, et al. ADIOS 2: The adaptable input output system. a framework for high-performance data management. *SoftwareX*, 12:100561, 2020.

[7] Mikaila J. Gossman, Bogdan Nicolae, Jon C. Calhoun, Franck Cappello, and Melissa C. Smith. Towards aggregated asynchronous checkpointing. In *Second International Symposium on Checkpointing for Supercomputing (SuperCheck'21)*, 2021. arXiv:2112.02289.

[8] HDF Group and LBNL. HDF5 asynchronous I/O VOL connector: Best practices. https://hdf5-vol-async.readthedocs.io/en/latest/bestpractice.html, 2024.

[9] Intel Corporation. Intel MPI Benchmarks user guide: IMB-IO non-blocking benchmarks. https://www.intel.com/content/www/us/en/docs/mpi-library/user-guide-benchmarks/2021-8/imb-io-non-blocking-benchmarks.html, 2021.

[10] Rajeev Jain, Houjun Tang, Akash Dhruv, Austin Harris, and Suren Byna. Accelerating Flash-X simulations with asynchronous I/O. In *2022 IEEE/ACM Seventh International Parallel Data Systems Workshop (PDSW)*, 2022.

[11] Anthony Kougkas, Hariharan Devarajan, Xian-He Sun, and Jay Lofstead. Harmonia: An interference-aware dynamic I/O scheduler for shared non-volatile burst buffers. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, page 290-301, 2018.

[12] William Lawry, Christopher Wilson, Arthur B. Maccabe, and Ron Brightwell. COMB: A portable benchmark suite for assessing MPI overlap. In *Proceedings of the IEEE International Conference on Cluster Computing (CLUSTER)*, page 472, 2002.

[13] Tonglin Li, Suren Byna, Quincey Koziol, Houjun Tang, Jean Luca Bez, and Qiao Kang. h5bench: HDF5 I/O kernel suite for exercising HPC I/O patterns. In *Proceedings of the Cray User Group (CUG)*, 2021.

[14] Jeremy Logan, Scott Klasky, Jay Lofstead, Hasan Abbasi, Stéphane Ethier, Ray Grout, Seung-Hoe Ku, Qing Liu, Xiaosong Ma, Manish Parashar, Norbert Podhorszki, Karsten Schwan, and Matthew Wolf. Skel: Generative software for producing skeletal I/O applications. In *7th IEEE International Conference on e-Science Workshops*, 2011.

[15] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. In *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, page 19-25, 1995.

[16] Bogdan Nicolae, Adam Moody, Elsa Gonsiorowski, Kathryn Mohror, and Franck Cappello. VeloC: Towards high performance adaptive asynchronous checkpointing at large scale. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, page 911-920, 2019.

[17] openPMD-api Developers. openPMD-api documentation: ADIOS2 backend. https://openpmd-api.readthedocs.io/en/0.15.2/backends/adios2.html, 2023.

[18] Christina M. Patrick, Seung Woo Son, and Mahmut Kandemir. Comparative evaluation of overlap strategies with study of I/O overlap in MPI-IO. In *ACM SIGOPS Operating Systems Review*, volume 42, page 43-49, 2008.

[19] Norbert Podhorszki et al. ADIOS 2: A framework for extreme scale I/O and in situ processing. OLCF User Training, https://www.olcf.ornl.gov/wp-content/uploads/ADIOS_OLCF_July2024.pdf, 2024.

[20] John Ravi, Suren Byna, Quincey Koziol, Houjun Tang, and Michela Becchi. Evaluating asynchronous parallel I/O on HPC systems. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, page 211-221, 2023.

[21] Houjun Tang. Accelerating HPC applications with asynchronous I/O. SC22 HDF5 BoF presentation, The HDF Group, 2022.

[22] Houjun Tang, Suren Byna, Stephen Bailey, Zarija Lukić, Jialin Liu, Quincey Koziol, and Bin Dong. Tuning object-centric data management systems for large scale scientific applications. In *26th IEEE International Conference on High Performance Computing, Data, and Analytics (HiPC)*, page 103-112, 2019.

[23] Houjun Tang, Suren Byna, François Tessier, Teng Wang, Bin Dong, Jingqing Mu, Quincey Koziol, Jerome Soumagne, Venkatram Vishwanath, Jialin Liu, and Richard Warren. Toward scalable and asynchronous object-centric data management for HPC. In *18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid)*, page 113-122, 2018.

[24] Houjun Tang, Quincey Koziol, Suren Byna, John Mainzer, and Tonglin Li. Enabling transparent asynchronous I/O using background threads. In *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, page 11-19, 2019.

[25] Houjun Tang, Quincey Koziol, John Ravi, and Suren Byna. Transparent asynchronous parallel I/O using background threads. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):891-902, 2022.

[26] Ahmad Tarraf, Javier Fernandez Muñoz, David E. Singh, Taylan Özden, Jesus Carretero, and Felix Wolf. I/O behind the scenes: Bandwidth requirements of HPC applications with asynchronous I/O. In *2024 IEEE International Conference on Cluster Computing (CLUSTER)*, page 426-439, 2024.

[27] Shu-Mei Tseng, Bogdan Nicolae, Franck Cappello, and Aparna Chandramowlishwaran. Demystifying asynchronous I/O interference in HPC applications. *The International Journal of High Performance Computing Applications*, 35(4):391-412, 2021.

A Benchmark Source Code

This appendix reproduces the complete source of the benchmark used in Section 4. It is built with `make` in `src/` against the software stack in Table 3, which `src/scripts/build_deps.sh` builds from source. Table 7 lists the code segments referred to in the paper, what each does, and where it is discussed.

Table 7. Code map: where each mechanism described in the paper is implemented. Line numbers refer to the listings in this appendix.

Segment	Listing	Lines	Description	Section
Driver				
`HeaderBackend`	1 (`aio_bench.h`)	25-37	Backend interface: init, issue, wait, fini, compute hints	§4.3
`ThreadInit`	2 (`aio_bench.c`)	104	MPI initialized with `MPI_THREAD_MULTIPLE` for every backend	§4.3
`TimedMax`	2 (`aio_bench.c`)	53-63	Concurrent kernel run timed as the maximum over ranks	§4.4
`Calibrate`	2 (`aio_bench.c`)	71-89	Calibration of fixed work to the target t_{comp}	§4.4
`Baseline`	2 (`aio_bench.c`)	148-155	Calibration call and standalone baseline t_{comp}^{alone}	§4.4
`Loop`	2 (`aio_bench.c`)	158-187	Timed epoch loop: compute, then issue (sync or async) I/O	§4.5
`Reduce`	2 (`aio_bench.c`)	189-197	Per-step maxima over ranks and end-to-end time	§4.6
Compute surrogates				
`Sleep`	3 (`kernels.c`)	189-192	`sleep`: `nanosleep` for t_{comp}	§4.4
`Spin`	3 (`kernels.c`)	25-32	spin: register-only xorshift loop	§4.4
`Dgemm`	3 (`kernels.c`)	38-44	DGEMM: 256^2 OpenBLAS calls (L2-resident)	§4.4
`Stream`	3 (`kernels.c`)	50-58	STREAM triad over 3×32 MiB arrays	§4.4
`Cart`	3 (`kernels.c`)	64-71	3-D periodic Cartesian decomposition	§4.4
`Exchange`	3 (`kernels.c`)	77-85	Six-neighbour non-blocking face exchange	§4.4
`Halo`	3 (`kernels.c`)	87-96	halo: 512 KiB face exchange plus allreduce	§4.4
`HeatHalo`	3 (`kernels.c`)	104-128	heat: pack, exchange, and unpack ghost faces	§4.4
`Heat`	3 (`kernels.c`)	130-145	heat: 7-point Jacobi sweep on 128^3 cells	§4.4
`GpuKernel`	4 (`gpu_kernels.cu`)	16-24	GPU FMA kernel	§4.4
`GpuSetup`	4 (`gpu_kernels.cu`)	32-40	Host wait mode: spin vs. blocking synchronization	§4.7.7
`GpuRun`	4 (`gpu_kernels.cu`)	43-50	GPU work units and stream synchronization	§4.4
I/O backends				
`PosixInit`	6 (`io_posix.c`)	31-42	POSIX AIO with one glibc helper thread	§4.3
`PosixWait`	6 (`io_posix.c`)	44-57	Completion of the in-flight `aio_write`	§4.3
`PosixIssue`	6 (`io_posix.c`)	59-90	Blocking `pwrite` vs. staged `aio_write`	§4.3
`MpiioIssue`	7 (`io_mpiio.c`)	60-77	Collective `MPI_File_write_at_all` vs. `iwrite_at_all`	§4.3
`HdfInit`	8 (`io_hdf5.c`)	19-42	MPI-IO VFD, collective transfer, event set, VOL check	§4.3
`HdfWait`	8 (`io_hdf5.c`)	44-51	`H5ESwait` on the event set	§4.3
`HdfIssue`	8 (`io_hdf5.c`)	53-81	Per-step dataset create/write/close via the `_async` API	§4.3
`BpInit`	9 (`io_adios2.c`)	27-61	BP5 setup: `AsyncWrite` mode and aggregation parameters	§4.3
`BpIssue`	9 (`io_adios2.c`)	63-73	One ADIOS2 step per timestep (deferred `Put`)	§4.3
`BpHints`	9 (`io_adios2.c`)	85-97	Computation-block hints for Guided mode	§4.3
`PdcIssue`	10 (`io_pdc.c`)	48-62	PDC region transfer (implemented, not evaluated)	§4.3
Experiment scripts				
`Binding`	14 (`run_one.sh`)	14-21	Rank binding: fully subscribed, SMT sibling, core reserved	§4.2
`AsyncEnv`	14 (`run_one.sh`)	27-36	Async VOL selection, `EXE_DCLOSE`, collective HDF5	§4.3
`Cgroup`	14 (`run_one.sh`)	40-44	`sync`, cgroup memory limit, and launch	§4.2
`MatrixEnv`	15 (`run_matrix.py`)	27-32	Library settings shared by all runs	§4.3
`MemMax`	15 (`run_matrix.py`)	42-45	Per-run cgroup limit: footprint + 24 GiB	§4.2
`IoBaseline`	15 (`run_matrix.py`)	101-106	Per-phase I/O-only baseline that sizes t_{comp}	§4.5
`TioSync`	15 (`run_matrix.py`)	80-98	Synchronous I/O time per step from the baseline runs	§4.5
`RunOne`	15 (`run_matrix.py`)	60-77	Resumable execution of one configuration	§4.5
`PhaseMain`	15 (`run_matrix.py`)	125-132	Main experiment: all CPU surrogates, one run each	§4.7.2
`PhaseSize`	15 (`run_matrix.py`)	135-142	Size sweep, 0.75 to 48 GiB per step	§4.5
`PhaseSizeLarge`	15 (`run_matrix.py`)	189-196	48 GiB per step for POSIX and ADIOS2	§4.5
`PhaseSweep`	15 (`run_matrix.py`)	155-163	t_{comp}/t_{io} sweep	§4.7.3
`PhaseBind`	15 (`run_matrix.py`)	166-177	Core-allocation experiment	§4.7.7
`PhaseGpu`	15 (`run_matrix.py`)	180-186	GPU experiment (4 ranks)	§4.7.7
`RunPdc`	15 (`run_matrix.py`)	212-244	One PDC run, killed as a group after 30 minutes	§4.3
`PhasePdc`	15 (`run_matrix.py`)	250-268	PDC main experiment and SMT-sibling placement	§4.7.7

Segment	Listing	Lines	Description	Section
`PhasePdcSize`	15 (`run_matrix.py`)	288-297	PDC size sweep, 0.73 and 2.9 GiB per step	§4.7.6
`PhasePdcSweep`	15 (`run_matrix.py`)	300-308	PDC t_{comp}/t_{io} sweep, 0.25 to 1	§4.7.5
`PhasePdcGpu`	15 (`run_matrix.py`)	278-285	PDC GPU experiment (1 server, 4 clients)	§4.7.8
Analysis				
`Metrics`	18 (`plots_measured.py`)	88-113	Speedup, compute slowdown, and exposed I/O	§4.6
`Excess`	18 (`plots_measured.py`)	222-245	Run time relative to the sleep prediction	§4.6
Build and patches				
`BuildHdf`	12 (`build_deps.sh`)	22-30	Parallel, thread-safe HDF5 build	§4.2
`BuildVol`	12 (`build_deps.sh`)	32-41	Async VOL build with the wait patch	§4.3
`BuildAdios`	12 (`build_deps.sh`)	43-56	ADIOS2 build options	§4.2
`VolPatch`	16 (`vol-async-wait.patch`)	1-22	Async VOL wait back-off: 1 s and 100 ms → 1 ms	§4.3
`AdiosPatch`	17 (`adios2-fsync.patch`)	1-56	Opt-in per-step `fdatasync` for BP5 (inactive)	§4.2

A.1 Benchmark driver

Listing 1. `aio_bench.h`

```c
/* aio_bench: compute/I-O epoch benchmark for evaluating asynchronous I/O
 * under different compute surrogates.  Shared declarations. */
#ifndef AIO_BENCH_H
#define AIO_BENCH_H

#include <mpi.h>
#include <stddef.h>
#include <stdint.h>

double now(void);
/* MPI_Info with ROMIO hints from AIO_ROMIO_HINTS="key=val,key=val" (or MPI_INFO_NULL). */
MPI_Info romio_hints(void);

/* ------------------------------------------------------------------ I/O -- */
typedef struct io_ctx {
    MPI_Comm comm;
    int rank, nranks;
    size_t bytes;      /* bytes written per rank per step */
    int async;         /* 0 = blocking, 1 = asynchronous */
    int durable;       /* 1 = a step completes only when its data is on the device */
    const char *dir;   /* output directory */
    void *priv;        /* backend state */
} io_ctx;

typedef struct io_backend {
    const char *name;
    int  (*init)(io_ctx *c);
    /* Write one step.  Blocking backends return when the data is written;
     * asynchronous backends may return as soon as `buf` can be reused. */
    void (*issue)(io_ctx *c, int step, const void *buf);
    /* Wait for all outstanding operations issued so far. */
    void (*wait)(io_ctx *c);
    void (*fini)(io_ctx *c);
    /* Optional hints bracketing the compute phase (ADIOS2 "Guided" mode). */
    void (*compute_begin)(io_ctx *c);
    void (*compute_end)(io_ctx *c);
} io_backend;

extern const io_backend io_posix, io_mpiio, io_hdf5, io_adios2, io_pdc;

/* ------------------------------------------------------------- kernels -- */
typedef enum {
    K_NONE,      /* no compute: I/O only */
    K_SLEEP,     /* nanosleep: yields core, no resources */
    K_SPIN,      /* integer ALU loop: one core, no memory traffic */
    K_DGEMM,     /* cache-resident DGEMM: CPU + caches */
    K_STREAM,    /* STREAM triad: memory bandwidth */
    K_HALO,      /* 3-D halo exchange + allreduce: MPI / interconnect */
    K_HEAT,      /* 3-D Jacobi stencil + halo exchange: mini-app */
    K_GPU_SPIN,  /* GPU FMA kernel, host spin-waits (CUDA default) */
    K_GPU_BLOCK, /* GPU FMA kernel, host blocks in the driver */
    K_NKERNELS
} kernel_t;

extern const char *kernel_names[K_NKERNELS];

int    kernel_parse(const char *s);
void   kernel_setup(kernel_t k, MPI_Comm comm, size_t io_bytes);
/* Run `units` of work of kernel k.  For K_SLEEP, units is microseconds. */
void   kernel_run(kernel_t k, double units);
```

```
/* Make the latest results available in host buffer `out` (GPU: D2H copy). */
void   kernel_output(kernel_t k, void *out, size_t bytes);
void   kernel_teardown(kernel_t k);

/* GPU entry points (gpu_kernels.cu, or stubs when built without CUDA). */
int  gpu_available(void);
void gpu_setup(int blocking_sync, size_t bytes);
void gpu_run(double units);
void gpu_output(void *out, size_t bytes);
void gpu_teardown(void);

#endif
```

Listing 2. `aio_bench.c`

```
/* aio_bench: alternate a compute phase with an I/O phase for a number of
 * steps, using a selectable compute surrogate and I/O backend.
 *
 *   for each step:  compute(t_comp)  ->  write step (blocking or async)  ->  barrier
 *   async:          issuing step i first waits for step i-1 (one in flight)
 *
 * The compute surrogate is calibrated, with all ranks running concurrently
 * and no I/O, to a fixed amount of work whose standalone duration is
 * --tcomp seconds.  Any growth of the compute phase in the timed loop is
 * therefore interference from the I/O path.
 *
 * Output: one CSV row (rank 0) with per-step means of the max over ranks. */
#define _GNU_SOURCE
#include "aio_bench.h"
#include <getopt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

double now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + 1e-9 * ts.tv_nsec;
}

MPI_Info romio_hints(void)
{
    const char *env = getenv("AIO_ROMIO_HINTS");
    if (!env || !*env) return MPI_INFO_NULL;
    MPI_Info info;
    char *s = strdup(env), *save, *kv;
    MPI_Info_create(&info);
    for (kv = strtok_r(s, ",", &save); kv; kv = strtok_r(NULL, ",", &save)) {
        char *eq = strchr(kv, '=');
        if (eq) { *eq = 0; MPI_Info_set(info, kv, eq + 1); }
    }
    free(s);
    return info;
}

static const io_backend *find_backend(const char *s)
{
    const io_backend *all[] = {&io_posix, &io_mpiio, &io_hdf5, &io_adios2, &io_pdc};
    for (size_t i = 0; i < sizeof all / sizeof *all; i++)
        if (!strcmp(s, all[i]->name)) return all[i];
    return NULL;
}

static double timed_max(kernel_t k, double units, MPI_Comm comm, void *buf, size_t bytes)
{
    double t0, t, tmax;
    MPI_Barrier(comm);
    t0 = now();
    kernel_run(k, units);
    kernel_output(k, buf, bytes);
    t = now() - t0;
    MPI_Allreduce(&t, &tmax, 1, MPI_DOUBLE, MPI_MAX, comm);
    return tmax;
}

/* Find the number of work units whose concurrent standalone run takes tcomp.
 * Work is linear in units, so calibrate against a short target (at most
 * CAL_TARGET seconds) and scale; the standalone baseline measured afterwards
 * at the full length reports any residual error. */
#define CAL_TARGET 2.0
```

```c
#define GPU_WARMUP 5.0
static double calibrate(kernel_t k, double tcomp, MPI_Comm comm, void *buf, size_t bytes)
{
    if (k == K_SLEEP) return tcomp * 1e6;
    /* GPUs idle at a low clock and boost only under sustained load, so a short
     * calibration burst would underestimate their speed: warm the device up
     * for GPU_WARMUP seconds and calibrate GPU kernels at the full length. */
    int gpu = (k == K_GPU_SPIN || k == K_GPU_BLOCK);
    double target = (gpu || tcomp < CAL_TARGET) ? tcomp : CAL_TARGET;
    double u = 1, t;
    timed_max(k, u, comm, buf, bytes); /* warm up */
    if (gpu)
        for (double w = 0; w < GPU_WARMUP; u *= 2) w += timed_max(k, u, comm, buf, bytes);
    while ((t = timed_max(k, u, comm, buf, bytes)) < 0.1 * target) u *= 4;
    for (int it = 0; it < 2; it++) {
        u = u * target / t;
        t = timed_max(k, u, comm, buf, bytes);
    }
    return u * target / t * (tcomp / target);
}

static void usage(void)
{
    fprintf(stderr,
            "usage: aio_bench --io {posix|mpiio|hdf5|adios2|pdc|none} --mode {sync|async}\n"
            "                 --kernel {none|sleep|spin|dgemm|stream|halo|heat|gpu-spin|gpu-block}\n"
            "                 [--tcomp s=2] [--mb MiB/rank=64] [--steps n=5] [--dir path=.]\n"
            "                 [--units u] [--tag label] [--csv file] [--no-fsync]\n"
            "                 [--alone-reps n=3]\n");
}

int main(int argc, char **argv)
{
    int provided, rank, nranks;
    MPI_Init_thread(&argc, &argv, MPI_THREAD_MULTIPLE, &provided);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nranks);
    /* MPI file errors return silently by default; make them abort instead
     * (also covers the MPI-IO calls HDF5 makes underneath). */
    MPI_File_set_errhandler(MPI_FILE_NULL, MPI_ERRORS_ARE_FATAL);

    const char *io_name = "none", *mode = "sync", *kname = "sleep", *dir = ".", *tag = "", *csv = NULL;
    double tcomp = 2.0, units = -1, mb = 64;
    int steps = 5, durable = 1, alone_reps = 3;
    static struct option opts[] = {
        {"io", 1, 0, 'i'}, {"mode", 1, 0, 'm'}, {"kernel", 1, 0, 'k'}, {"tcomp", 1, 0, 't'},
        {"mb", 1, 0, 'b'}, {"steps", 1, 0, 's'}, {"dir", 1, 0, 'd'}, {"units", 1, 0, 'u'},
        {"tag", 1, 0, 'g'}, {"csv", 1, 0, 'c'}, {"no-fsync", 0, 0, 'n'},
        {"alone-reps", 1, 0, 'a'}, {0, 0, 0, 0}};
    for (int o; (o = getopt_long(argc, argv, "", opts, NULL)) != -1;) {
        switch (o) {
        case 'i': io_name = optarg; break;
        case 'm': mode = optarg; break;
        case 'k': kname = optarg; break;
        case 't': tcomp = atof(optarg); break;
        case 'b': mb = atof(optarg); break;
        case 's': steps = atoi(optarg); break;
        case 'd': dir = optarg; break;
        case 'u': units = atof(optarg); break;
        case 'g': tag = optarg; break;
        case 'c': csv = optarg; break;
        case 'n': durable = 0; break;
        case 'a': alone_reps = atoi(optarg); break;
        default: if (!rank) usage(); MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }
    int k = kernel_parse(kname);
    const io_backend *be = strcmp(io_name, "none") ? find_backend(io_name) : NULL;
    if (k < 0 || (strcmp(io_name, "none") && !be)) { if (!rank) usage(); MPI_Abort(MPI_COMM_WORLD, 1); }
    if (provided < MPI_THREAD_MULTIPLE && !rank) fprintf(stderr, "warning: MPI_THREAD_MULTIPLE not provided\n");

    io_ctx ctx = {MPI_COMM_WORLD, rank, nranks, (size_t)(mb * 1048576.0) / 8 * 8,
                  !strcmp(mode, "async"), durable, dir, NULL};
    void *buf;
    if (posix_memalign(&buf, 4096, ctx.bytes)) MPI_Abort(MPI_COMM_WORLD, 1);
    for (size_t i = 0; i < ctx.bytes / sizeof(double); i++) ((double *)buf)[i] = rank + 1e-6 * i;
    if (!rank) mkdir(dir, 0755);

    /* ---- calibration and standalone baseline (no I/O) ---- */
    kernel_setup(k, MPI_COMM_WORLD, ctx.bytes);
    double t_alone = 0;
    if (k != K_NONE) {
        if (units < 0) units = calibrate(k, tcomp, MPI_COMM_WORLD, buf, ctx.bytes);
        MPI_Bcast(&units, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
        for (int r = 0; r < alone_reps; r++)
```

```
            t_alone += timed_max(k, units, MPI_COMM_WORLD, buf, ctx.bytes) / alone_reps;
    }

    /* ---- timed epoch loop ---- */
    double t_init = now();
    if (be && be->init(&ctx)) {
        if (!rank) fprintf(stderr, "backend %s unavailable\n", io_name);
        MPI_Abort(MPI_COMM_WORLD, 6);
    }
    t_init = now() - t_init;

    double *tc = calloc(steps, sizeof *tc), *ti = calloc(steps, sizeof *ti);
    MPI_Barrier(MPI_COMM_WORLD);
    double t0 = now();
    for (int s = 0; s < steps; s++) {
        double a = now();
        if (be && be->compute_begin) be->compute_begin(&ctx);
        if (k != K_NONE) { kernel_run(k, units); kernel_output(k, buf, ctx.bytes); }
        if (be && be->compute_end) be->compute_end(&ctx);
        double b = now();
        if (be) be->issue(&ctx, s, buf);
        /* The I/O phase ends when every rank has finished it; without this
         * barrier, ranks that finish early would charge the imbalance to the
         * next compute phase (e.g., waiting in a halo exchange). */
        MPI_Barrier(MPI_COMM_WORLD);
        tc[s] = b - a;
        ti[s] = now() - b;
    }
    double tf = now();
    if (be) { be->wait(&ctx); be->fini(&ctx); }
    double t_end = now();
    tf = t_end - tf;
    double total = t_end - t0;

    /* ---- reduce: per-step max over ranks, then mean over steps ---- */
    double *tcm = calloc(steps, sizeof *tcm), *tim = calloc(steps, sizeof *tim);
    double io_rank = tf, io_max, tf_max, total_max;
    for (int s = 0; s < steps; s++) io_rank += ti[s];
    MPI_Reduce(tc, tcm, steps, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(ti, tim, steps, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&io_rank, &io_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&tf, &tf_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&total, &total_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    if (!rank) {
        double comp = 0, io = 0;
        for (int s = 0; s < steps; s++) { comp += tcm[s] / steps; io += tim[s] / steps; }
        double gb = (double)ctx.bytes * nranks * steps / 1e9;
        FILE *f = csv ? fopen(csv, "a") : stdout;
        if (f && csv && ftell(f) == 0)
            fprintf(f, "io,mode,kernel,tag,durable,nranks,mib_per_rank,steps,tcomp_target,units,"
                       "t_alone,t_comp,slowdown,t_io_step,t_final,t_io_obs,t_total,t_init,gb\n");
        fprintf(f, "%s,%s,%s,%s,%d,%d,%.1f,%d,%.3f,%.6g,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f,%.3f\n",
                io_name, mode, kname, tag, durable, nranks, mb, steps, tcomp, units, t_alone, comp,
                t_alone > 0 ? comp / t_alone : 0, io, tf_max, io_max, total_max, t_init, gb);
        if (csv) fclose(f);
        fprintf(stderr, "%s/%s/%s[%s] P=%d alone=%.3f comp=%.3f (x%.2f) io/step=%.3f final=%.3f total=%.3f\n",
                io_name, mode, kname, tag, nranks, t_alone, comp, t_alone > 0 ? comp / t_alone : 0, io,
                tf_max, total_max);
    }
    kernel_teardown(k);
    free(buf);
    MPI_Finalize();
    return 0;
}
```

A.2 Compute surrogate kernels

Listing 3. `kernels.c`

```
/* Compute surrogates.  Each kernel performs a fixed amount of work per unit,
 * so that interference shows up as a longer compute phase (a time-based
 * busy-wait or sleep cannot be slowed down). */
#define _GNU_SOURCE
#include "aio_bench.h"
#include <cblas.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

const char *kernel_names[K_NKERNELS] = {
    "none", "sleep", "spin", "dgemm", "stream", "halo", "heat", "gpu-spin", "gpu-block"};

```

```
int kernel_parse(const char *s)
{
    for (int k = 0; k < K_NKERNELS; k++)
        if (!strcmp(s, kernel_names[k])) return k;
    return -1;
}

/* ---- spin ---- */
static volatile uint64_t spin_sink;

static void run_spin(double units)
{
    uint64_t x = 88172645463325252ULL, n = (uint64_t)(units * 1e6);
    for (uint64_t i = 0; i < n; i++) { /* xorshift: register-only ALU work */
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
    }
    spin_sink = x;
}

/* ---- dgemm: N=256 matrices (1.5 MiB total) stay in the private L2 ---- */
#define GEMM_N 256
static double *ga, *gb, *gc;

static void run_dgemm(double units)
{
    long n = (long)(units + 0.5);
    for (long i = 0; i < n; i++)
        cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, GEMM_N, GEMM_N, GEMM_N,
                    1.0, ga, GEMM_N, gb, GEMM_N, 0.5, gc, GEMM_N);
}

/* ---- STREAM triad: 3 x 32 MiB arrays per rank, far beyond the LLC share ---- */
#define STREAM_N (4L << 20)
static double *sa, *sb, *sc;

static void run_stream(double units)
{
    long n = (long)(units + 0.5);
    const double s = 3.0;
    for (long r = 0; r < n; r++) {
        for (long i = 0; i < STREAM_N; i++) sa[i] = sb[i] + s * sc[i];
        __asm__ volatile("" ::: "memory");
    }
}

/* ---- 3-D Cartesian neighbours shared by halo and heat ---- */
static MPI_Comm cart;
static int nbr[6]; /* -x +x -y +y -z +z */

static void setup_cart(MPI_Comm comm)
{
    int n, dims[3] = {0, 0, 0}, periods[3] = {1, 1, 1};
    MPI_Comm_size(comm, &n);
    MPI_Dims_create(n, 3, dims);
    MPI_Cart_create(comm, 3, dims, periods, 0, &cart);
    for (int d = 0; d < 3; d++) MPI_Cart_shift(cart, d, 1, &nbr[2 * d], &nbr[2 * d + 1]);
}

/* ---- halo: 6 faces of 256x256 doubles (512 KiB each) + 8-byte allreduce ---- */
#define HALO_FACE (256 * 256)
static double *hsend, *hrecv;

static void exchange(double *send, double *recv, int count)
{
    MPI_Request rq[12];
    for (int f = 0; f < 6; f++) {
        MPI_Irecv(recv + (size_t)f * count, count, MPI_DOUBLE, nbr[f], f ^ 1, cart, &rq[f]);
        MPI_Isend(send + (size_t)f * count, count, MPI_DOUBLE, nbr[f], f, cart, &rq[6 + f]);
    }
    MPI_Waitall(12, rq, MPI_STATUSES_IGNORE);
}

static void run_halo(double units)
{
    long n = (long)(units + 0.5);
    for (long i = 0; i < n; i++) {
        exchange(hsend, hrecv, HALO_FACE);
        double v = hrecv[i % HALO_FACE], s;
        MPI_Allreduce(&v, &s, 1, MPI_DOUBLE, MPI_SUM, cart);
        hsend[0] = s * 1e-9;
    }
}

/* ---- heat: 7-point Jacobi on a 128^3 block with ghost cells ---- */
#define HN 128
```

```
#define HS (HN + 2)
#define IDX(i, j, k) (((size_t)(i) * HS + (j)) * HS + (k))
static double *u0, *u1, *fsend, *frecv;

static void heat_halo(double *u)
{
    const int F = HN * HN;
    for (int a = 0; a < HN; a++)
        for (int b = 0; b < HN; b++) {
            int o = a * HN + b;
            fsend[0 * F + o] = u[IDX(1, a + 1, b + 1)];
            fsend[1 * F + o] = u[IDX(HN, a + 1, b + 1)];
            fsend[2 * F + o] = u[IDX(a + 1, 1, b + 1)];
            fsend[3 * F + o] = u[IDX(a + 1, HN, b + 1)];
            fsend[4 * F + o] = u[IDX(a + 1, b + 1, 1)];
            fsend[5 * F + o] = u[IDX(a + 1, b + 1, HN)];
        }
    exchange(fsend, frecv, F);
    for (int a = 0; a < HN; a++)
        for (int b = 0; b < HN; b++) {
            int o = a * HN + b;
            u[IDX(0, a + 1, b + 1)]      = frecv[0 * F + o];
            u[IDX(HN + 1, a + 1, b + 1)] = frecv[1 * F + o];
            u[IDX(a + 1, 0, b + 1)]      = frecv[2 * F + o];
            u[IDX(a + 1, HN + 1, b + 1)] = frecv[3 * F + o];
            u[IDX(a + 1, b + 1, 0)]      = frecv[4 * F + o];
            u[IDX(a + 1, b + 1, HN + 1)] = frecv[5 * F + o];
        }
}

static void run_heat(double units)
{
    long n = (long)(units + 0.5);
    const double c = 1.0 / 7.0;
    for (long it = 0; it < n; it++) {
        heat_halo(u0);
        for (int i = 1; i <= HN; i++)
            for (int j = 1; j <= HN; j++)
                for (int k = 1; k <= HN; k++)
                    u1[IDX(i, j, k)] = c * (u0[IDX(i, j, k)] + u0[IDX(i - 1, j, k)] +
                                            u0[IDX(i + 1, j, k)] + u0[IDX(i, j - 1, k)] +
                                            u0[IDX(i, j + 1, k)] + u0[IDX(i, j, k - 1)] +
                                            u0[IDX(i, j, k + 1)]);
        double *t = u0; u0 = u1; u1 = t;
    }
}

static double *xalloc(size_t n)
{
    double *p;
    if (posix_memalign((void **)&p, 64, n * sizeof(double))) MPI_Abort(MPI_COMM_WORLD, 1);
    for (size_t i = 0; i < n; i++) p[i] = 1.0 + (double)(i % 97) * 1e-3;
    return p;
}

void kernel_setup(kernel_t k, MPI_Comm comm, size_t io_bytes)
{
    switch (k) {
    case K_DGEMM:
        ga = xalloc(GEMM_N * GEMM_N); gb = xalloc(GEMM_N * GEMM_N); gc = xalloc(GEMM_N * GEMM_N);
        break;
    case K_STREAM:
        sa = xalloc(STREAM_N); sb = xalloc(STREAM_N); sc = xalloc(STREAM_N);
        break;
    case K_HALO:
        setup_cart(comm);
        hsend = xalloc(6 * HALO_FACE); hrecv = xalloc(6 * HALO_FACE);
        break;
    case K_HEAT:
        setup_cart(comm);
        u0 = xalloc((size_t)HS * HS * HS); u1 = xalloc((size_t)HS * HS * HS);
        fsend = xalloc(6 * HN * HN); frecv = xalloc(6 * HN * HN);
        break;
    case K_GPU_SPIN:
    case K_GPU_BLOCK:
        if (!gpu_available()) {
            fprintf(stderr, "aio_bench: built without CUDA or no GPU found\n");
            MPI_Abort(MPI_COMM_WORLD, 2);
        }
        gpu_setup(k == K_GPU_BLOCK, io_bytes);
        break;
    default:
        break;
    }
}
```

```
void kernel_run(kernel_t k, double units)
{
    switch (k) {
    case K_SLEEP: {
        struct timespec ts = {(time_t)(units / 1e6), (long)(((uint64_t)units % 1000000) * 1000)};
        while (nanosleep(&ts, &ts)) ;
        break;
    }
    case K_SPIN:   run_spin(units); break;
    case K_DGEMM:  run_dgemm(units); break;
    case K_STREAM: run_stream(units); break;
    case K_HALO:   run_halo(units); break;
    case K_HEAT:   run_heat(units); break;
    case K_GPU_SPIN:
    case K_GPU_BLOCK: gpu_run(units); break;
    default: break;
    }
}

void kernel_output(kernel_t k, void *out, size_t bytes)
{
    if (k == K_GPU_SPIN || k == K_GPU_BLOCK) gpu_output(out, bytes);
    else ((volatile char *)out)[0] ^= 1; /* host data is already in place */
}

void kernel_teardown(kernel_t k)
{
    if (k == K_GPU_SPIN || k == K_GPU_BLOCK) gpu_teardown();
}
```

Listing 4. `gpu_kernels.cu`

```
// GPU compute surrogate: an FMA-bound kernel that keeps the device busy while
// the host either spin-waits (CUDA's default for few contexts per core) or
// blocks in the driver (cudaDeviceScheduleBlockingSync).  The output field
// lives on the device and is copied to the host before every I/O phase.
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>

#define CK(x) do { cudaError_t e = (x); if (e != cudaSuccess) { \
    fprintf(stderr, "CUDA %s:%d %s\n", __FILE__, __LINE__, cudaGetErrorString(e)); abort(); } } while (0)

static float *d_field;
static size_t d_n;
static cudaStream_t stream;

__global__ void fma_kernel(float *x, size_t n, int iters)
{
    for (size_t i = blockIdx.x * (size_t)blockDim.x + threadIdx.x; i < n;
         i += (size_t)gridDim.x * blockDim.x) {
        float a = x[i], b = 1.0001f, c = 0.9999f;
        for (int k = 0; k < iters; k++) { a = fmaf(a, b, c); b = fmaf(b, c, a) * 1e-3f; }
        x[i] = a + b;
    }
}

extern "C" int gpu_available(void)
{
    int n = 0;
    return cudaGetDeviceCount(&n) == cudaSuccess && n > 0;
}

extern "C" void gpu_setup(int blocking_sync, size_t bytes)
{
    CK(cudaSetDeviceFlags(blocking_sync ? cudaDeviceScheduleBlockingSync : cudaDeviceScheduleSpin));
    CK(cudaSetDevice(0));
    d_n = bytes / sizeof(float);
    CK(cudaMalloc(&d_field, d_n * sizeof(float)));
    CK(cudaMemset(d_field, 0, d_n * sizeof(float)));
    CK(cudaStreamCreate(&stream));
}

// One unit = one launch over a 1M-element slice with 4096 FMA pairs per element.
extern "C" void gpu_run(double units)
{
    long n = (long)(units + 0.5);
    size_t slice = d_n < (1u << 20) ? d_n : (1u << 20);
    for (long i = 0; i < n; i++)
        fma_kernel<<<1024, 256, 0, stream>>>(d_field + (i * slice) % (d_n - slice + 1), slice, 4096);
    CK(cudaStreamSynchronize(stream));
}
```

```
extern "C" void gpu_output(void *out, size_t bytes)
{
    CK(cudaMemcpyAsync(out, d_field, bytes, cudaMemcpyDeviceToHost, stream));
    CK(cudaStreamSynchronize(stream));
}

extern "C" void gpu_teardown(void)
{
    cudaFree(d_field);
    cudaStreamDestroy(stream);
}
```

Listing 5. `gpu_stub.c`

```
/* Stubs used when aio_bench is built without CUDA. */
#include "aio_bench.h"

int  gpu_available(void) { return 0; }
void gpu_setup(int blocking_sync, size_t bytes) { (void)blocking_sync; (void)bytes; }
void gpu_run(double units) { (void)units; }
void gpu_output(void *out, size_t bytes) { (void)out; (void)bytes; }
void gpu_teardown(void) {}
```

A.3 I/O backends

Listing 6. `io_posix.c`

```
/* POSIX backend.  Each step is one shared file; rank r writes its block at
 * offset r*bytes.  Blocking mode uses pwrite(); asynchronous mode copies the
 * block to a staging buffer and submits it with POSIX AIO (aio_write), which
 * glibc services from a background thread.  When durable, blocking mode
 * calls fdatasync() and asynchronous mode queues aio_fsync(O_DSYNC). */
#define _GNU_SOURCE
#include "aio_bench.h"
#include <aio.h>
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

typedef struct {
    void *stage;
    struct aiocb cb, sync_cb;
    int fd, pending;
} posix_state;

static int open_step(io_ctx *c, int step)
{
    char path[4096];
    snprintf(path, sizeof path, "%s/posix_step%03d.dat", c->dir, step);
    int fd = open(path, O_CREAT | O_WRONLY, 0644);
    if (fd < 0) { perror(path); MPI_Abort(c->comm, 3); }
    return fd;
}

static int posix_init(io_ctx *c)
{
    posix_state *s = calloc(1, sizeof *s);
    if (c->async) {
        struct aioinit ai = {.aio_threads = 1, .aio_num = 4, .aio_idle_time = 1};
        aio_init(&ai);
        if (posix_memalign(&s->stage, 4096, c->bytes)) return -1;
        memset(s->stage, 0, c->bytes);
    }
    c->priv = s;
    return 0;
}

static void posix_wait(io_ctx *c)
{
    posix_state *s = c->priv;
    if (!s->pending) return;
    const struct aiocb *list[2] = {&s->cb, c->durable ? &s->sync_cb : NULL};
    while (aio_error(&s->cb) == EINPROGRESS) aio_suspend(list, 2, NULL);
    if (aio_return(&s->cb) != (ssize_t)c->bytes) { fprintf(stderr, "aio_write failed\n"); MPI_Abort(c->comm, 3); }
    if (c->durable) {
        while (aio_error(&s->sync_cb) == EINPROGRESS) aio_suspend(list + 1, 1, NULL);
        if (aio_return(&s->sync_cb)) { fprintf(stderr, "aio_fsync failed\n"); MPI_Abort(c->comm, 3); }
    }
    close(s->fd);
```

```
    s->pending = 0;
}

static void posix_issue(io_ctx *c, int step, const void *buf)
{
    posix_state *s = c->priv;
    off_t off = (off_t)c->rank * c->bytes;
    if (!c->async) {
        int fd = open_step(c, step);
        const char *p = buf;
        for (size_t done = 0; done < c->bytes;) {
            ssize_t n = pwrite(fd, p + done, c->bytes - done, off + done);
            if (n < 0) { perror("pwrite"); MPI_Abort(c->comm, 3); }
            done += n;
        }
        if (c->durable) fdatasync(fd);
        close(fd);
        return;
    }
    posix_wait(c); /* staging buffer is single-buffered */
    memcpy(s->stage, buf, c->bytes);
    s->fd = open_step(c, step);
    memset(&s->cb, 0, sizeof s->cb);
    s->cb.aio_fildes = s->fd;
    s->cb.aio_buf = s->stage;
    s->cb.aio_nbytes = c->bytes;
    s->cb.aio_offset = off;
    if (aio_write(&s->cb)) { perror("aio_write"); MPI_Abort(c->comm, 3); }
    if (c->durable) {
        memset(&s->sync_cb, 0, sizeof s->sync_cb);
        s->sync_cb.aio_fildes = s->fd;
        if (aio_fsync(O_DSYNC, &s->sync_cb)) { perror("aio_fsync"); MPI_Abort(c->comm, 3); }
    }
    s->pending = 1;
}

static void posix_fini(io_ctx *c)
{
    posix_state *s = c->priv;
    posix_wait(c);
    free(s->stage);
    free(s);
}

const io_backend io_posix = {"posix", posix_init, posix_issue, posix_wait, posix_fini, NULL, NULL};
```

Listing 7. `io_mpiio.c`

```
/* MPI-IO backend.  One shared file per step, collective writes at rank
 * offsets.  Asynchronous mode uses MPI_File_iwrite_at_all on a staging copy;
 * whether it progresses in the background depends on the MPI library (for
 * MPICH, MPIR_CVAR_ASYNC_PROGRESS=1 starts a progress thread). */
#include "aio_bench.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    void *stage;
    MPI_File fh;
    MPI_Request req;
    int pending;
} mpiio_state;

static MPI_File open_step(io_ctx *c, int step)
{
    char path[4096];
    MPI_File fh;
    snprintf(path, sizeof path, "%s/mpiio_step%03d.dat", c->dir, step);
    if (MPI_File_open(c->comm, path, MPI_MODE_CREATE | MPI_MODE_WRONLY, romio_hints(), &fh)) {
        fprintf(stderr, "MPI_File_open %s failed\n", path);
        MPI_Abort(c->comm, 3);
    }
    if (step == 0 && c->rank == 0 && getenv("AIO_DEBUG")) {
        MPI_Info info;
        char key[MPI_MAX_INFO_KEY + 1], val[256];
        int n, flag;
        MPI_File_get_info(fh, &info);
        MPI_Info_get_nkeys(info, &n);
        for (int i = 0; i < n; i++) {
            MPI_Info_get_nthkey(info, i, key);
            MPI_Info_get(info, key, sizeof val - 1, val, &flag);
            fprintf(stderr, "[mpiio] %s=%s\n", key, val);
        }
```

```
        MPI_Info_free(&info);
    }
    return fh;
}

static int mpiio_init(io_ctx *c)
{
    mpiio_state *s = calloc(1, sizeof *s);
    if (c->async) { s->stage = malloc(c->bytes); memset(s->stage, 0, c->bytes); }
    c->priv = s;
    return 0;
}

static void mpiio_wait(io_ctx *c)
{
    mpiio_state *s = c->priv;
    if (!s->pending) return;
    MPI_Wait(&s->req, MPI_STATUS_IGNORE);
    if (c->durable) MPI_File_sync(s->fh); /* MPI has no non-blocking sync */
    MPI_File_close(&s->fh);
    s->pending = 0;
}

static void mpiio_issue(io_ctx *c, int step, const void *buf)
{
    mpiio_state *s = c->priv;
    MPI_Offset off = (MPI_Offset)c->rank * c->bytes;
    int count = (int)(c->bytes / sizeof(double));
    if (!c->async) {
        MPI_File fh = open_step(c, step);
        MPI_File_write_at_all(fh, off, buf, count, MPI_DOUBLE, MPI_STATUS_IGNORE);
        if (c->durable) MPI_File_sync(fh);
        MPI_File_close(&fh);
        return;
    }
    mpiio_wait(c);
    memcpy(s->stage, buf, c->bytes);
    s->fh = open_step(c, step);
    MPI_File_iwrite_at_all(s->fh, off, s->stage, count, MPI_DOUBLE, &s->req);
    s->pending = 1;
}

static void mpiio_fini(io_ctx *c)
{
    mpiio_state *s = c->priv;
    mpiio_wait(c);
    free(s->stage);
    free(s);
}

const io_backend io_mpiio = {"mpiio", mpiio_init, mpiio_issue, mpiio_wait, mpiio_fini, NULL, NULL};
```

Listing 8. `io_hdf5.c`

```
/* HDF5 backend (parallel HDF5, MPI-IO VFD).  One file for the run; each step
 * writes a new dataset "stepNNN" of nranks*bytes with a collective write.
 * Both modes use the *_async API with an event set: under the native VOL
 * these calls complete synchronously; when the asynchronous VOL connector is
 * selected (HDF5_VOL_CONNECTOR="async under_vol=0;under_info={}") they are
 * queued and executed by its Argobots background thread.  When durable,
 * each step ends with H5Fflush, which the MPI-IO VFD turns into
 * MPI_File_sync (fsync). */
#include "aio_bench.h"
#include <hdf5.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    hid_t file, fapl, dxpl, es;
} hdf5_state;

static int hdf5_init(io_ctx *c)
{
    hdf5_state *s = calloc(1, sizeof *s);
    char path[4096];
    snprintf(path, sizeof path, "%s/hdf5_out.h5", c->dir);
    s->es = H5EScreate();
    s->fapl = H5Pcreate(H5P_FILE_ACCESS);
    H5Pset_fapl_mpio(s->fapl, c->comm, romio_hints());
    /* AIO_HDF5_INDEP=1: independent raw-data I/O and metadata writes. */
    int indep = getenv("AIO_HDF5_INDEP") && atoi(getenv("AIO_HDF5_INDEP"));
    H5Pset_coll_metadata_write(s->fapl, !indep);
    s->dxpl = H5Pcreate(H5P_DATASET_XFER);
```

```
    H5Pset_dxpl_mpio(s->dxpl, indep ? H5FD_MPIO_INDEPENDENT : H5FD_MPIO_COLLECTIVE);
    s->file = H5Fcreate_async(path, H5F_ACC_TRUNC, H5P_DEFAULT, s->fapl, s->es);
    if (s->file < 0) return -1;
    if (c->rank == 0) {
        char name[64] = "";
        H5VLget_connector_name(s->file, name, sizeof name);
        fprintf(stderr, "[hdf5] VOL connector: %s (async=%d)\n", name, c->async);
        if (c->async && strcmp(name, "async")) fprintf(stderr, "[hdf5] WARNING: async VOL not active\n");
    }
    c->priv = s;
    return 0;
}

static void hdf5_wait(io_ctx *c)
{
    hdf5_state *s = c->priv;
    size_t ninprog;
    hbool_t failed;
    H5ESwait(s->es, H5ES_WAIT_FOREVER, &ninprog, &failed);
    if (failed) { fprintf(stderr, "[hdf5] async operation failed\n"); MPI_Abort(c->comm, 4); }
}

static void hdf5_issue(io_ctx *c, int step, const void *buf)
{
    hdf5_state *s = c->priv;
    char name[32];
    hsize_t n = c->bytes / sizeof(double), dims = n * c->nranks, start = n * c->rank;
    double t[6];
    snprintf(name, sizeof name, "step%03d", step);
    t[0] = now();
    if (c->async) hdf5_wait(c); /* at most one step in flight, as in h5bench */
    t[1] = now();
    hid_t fspace = H5Screate_simple(1, &dims, NULL);
    hid_t mspace = H5Screate_simple(1, &n, NULL);
    hid_t dset = H5Dcreate_async(s->file, name, H5T_NATIVE_DOUBLE, fspace, H5P_DEFAULT,
                                 H5P_DEFAULT, H5P_DEFAULT, s->es);
    t[2] = now();
    H5Sselect_hyperslab(fspace, H5S_SELECT_SET, &start, NULL, &n, NULL);
    H5Dwrite_async(dset, H5T_NATIVE_DOUBLE, mspace, fspace, s->dxpl, buf, s->es);
    t[3] = now();
    H5Dclose_async(dset, s->es);
    t[4] = now();
    if (c->durable) H5Fflush_async(s->file, H5F_SCOPE_LOCAL, s->es);
    H5Sclose(mspace);
    H5Sclose(fspace);
    t[5] = now();
    if (!c->async) hdf5_wait(c);
    if (c->rank == 0 && getenv("AIO_DEBUG"))
        fprintf(stderr, "[hdf5] step %d wait=%.3f create=%.3f write=%.3f close=%.3f flush=%.3f end=%.3f\n",
                step, t[1] - t[0], t[2] - t[1], t[3] - t[2], t[4] - t[3], t[5] - t[4], now() - t[5]);
}

static void hdf5_fini(io_ctx *c)
{
    hdf5_state *s = c->priv;
    double t0 = now();
    hdf5_wait(c);
    double t1 = now();
    H5Fclose_async(s->file, s->es);
    double t2 = now();
    hdf5_wait(c);
    if (c->rank == 0 && getenv("AIO_DEBUG"))
        fprintf(stderr, "[hdf5] fini drain=%.3f fclose=%.3f wait=%.3f\n", t1 - t0, t2 - t1, now() - t2);
    H5ESclose(s->es);
    H5Pclose(s->dxpl);
    H5Pclose(s->fapl);
    free(s);
}

const io_backend io_hdf5 = {"hdf5", hdf5_init, hdf5_issue, hdf5_wait, hdf5_fini, NULL, NULL};
```

Listing 9. `io_adios2.c`

```
/* ADIOS2 backend (BP5 engine).  One output stream; each step is an ADIOS
 * step holding a global 1-D array.  Asynchronous mode sets the BP5
 * AsyncWrite parameter (default "Guided", overridable with
 * AIO_ADIOS2_ASYNC_MODE) so that buffered data is written by a background
 * thread after EndStep; in Guided mode the compute phase is bracketed with
 * Enter/ExitComputationBlock hints.  Durability (fdatasync of every step)
 * requires the patched ADIOS2; ADIOS2_FSYNC is set here from --no-fsync. */
#define _GNU_SOURCE
#include "aio_bench.h"
#define ADIOS2_USE_MPI 1
#include <adios2_c.h>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    adios2_adios *adios;
    adios2_io *io;
    adios2_variable *var;
    adios2_engine *eng;
    int guided;
} bp5_state;

#define CHK(x) do { if ((x) != adios2_error_none) { \
    fprintf(stderr, "[adios2] %s failed\n", #x); MPI_Abort(c->comm, 5); } } while (0)

static int bp5_init(io_ctx *c)
{
    bp5_state *s = calloc(1, sizeof *s);
    char path[4096];
    const char *mode = getenv("AIO_ADIOS2_ASYNC_MODE");
    if (!mode) mode = "Guided";
    /* The patched FilePOSIX::Flush reads ADIOS2_FSYNC on first use. */
    setenv("ADIOS2_FSYNC", c->durable ? "1" : "0", 1);
    s->adios = adios2_init_mpi(c->comm);
    s->io = adios2_declare_io(s->adios, "out");
    CHK(adios2_set_engine(s->io, "BP5"));
    CHK(adios2_set_parameter(s->io, "AsyncWrite", c->async ? mode : "false"));
    s->guided = c->async && !strcmp(mode, "Guided");
    /* Extra BP5 parameters, AIO_ADIOS2_PARAMS="Key=Value;Key=Value". */
    const char *extra = getenv("AIO_ADIOS2_PARAMS");
    if (extra && *extra) {
        char *dup = strdup(extra), *save, *kv;
        for (kv = strtok_r(dup, ";", &save); kv; kv = strtok_r(NULL, ";", &save)) {
            char *eq = strchr(kv, '=');
            if (eq) { *eq = 0; CHK(adios2_set_parameter(s->io, kv, eq + 1)); }
        }
        free(dup);
    }
    size_t n = c->bytes / sizeof(double);
    size_t shape = n * c->nranks, start = n * c->rank, count = n;
    s->var = adios2_define_variable(s->io, "field", adios2_type_double, 1, &shape, &start, &count,
                                    adios2_constant_dims_true);
    snprintf(path, sizeof path, "%s/adios2_out.bp", c->dir);
    s->eng = adios2_open(s->io, path, adios2_mode_write);
    if (!s->eng || !s->var) return -1;
    if (c->rank == 0)
        fprintf(stderr, "[adios2] BP5 AsyncWrite=%s %s\n", c->async ? mode : "false", extra ? extra : "");
    c->priv = s;
    return 0;
}

static void bp5_issue(io_ctx *c, int step, const void *buf)
{
    bp5_state *s = c->priv;
    adios2_step_status st;
    (void)step;
    /* In async mode, BeginStep/EndStep block only if the previous step's
     * background write is still running. */
    CHK(adios2_begin_step(s->eng, adios2_step_mode_append, -1.0f, &st));
    CHK(adios2_put(s->eng, s->var, buf, adios2_mode_deferred));
    CHK(adios2_end_step(s->eng));
}

static void bp5_wait(io_ctx *c) { (void)c; /* completion is only observable at Close */ }

static void bp5_fini(io_ctx *c)
{
    bp5_state *s = c->priv;
    CHK(adios2_close(s->eng));
    adios2_finalize(s->adios);
    free(s);
}

static void bp5_cbegin(io_ctx *c)
{
    bp5_state *s = c->priv;
    if (s->guided) adios2_enter_computation_block(s->adios);
}

static void bp5_cend(io_ctx *c)
{
    bp5_state *s = c->priv;
    if (s->guided) adios2_exit_computation_block(s->adios);
}

```

```
const io_backend io_adios2 = {"adios2", bp5_init, bp5_issue, bp5_wait, bp5_fini,
                              bp5_cbegin, bp5_cend};
```

Listing 10. `io_pdc.c`

```
/* PDC (Proactive Data Containers) backend.  Each step is a PDC object of
 * nranks*bytes; every rank transfers its region with the region-transfer
 * API.  Asynchronous mode starts the transfer and waits for it before the
 * next step; blocking mode waits immediately.  Data movement to storage is
 * performed by the PDC servers (launched separately, see scripts/run_pdc.sh).
 *
 * Compiled only with -DHAVE_PDC (make pdc builds aio_bench_pdc against an
 * existing PDC install); without PDC the backend reports itself unavailable. */
#include "aio_bench.h"
#include <stdio.h>
#include <stdlib.h>

#ifdef HAVE_PDC
#include <pdc.h>

typedef struct {
    pdcid_t pdc, cont_prop, cont, obj_prop, obj, local, remote, xfer;
    int pending;
} pdc_state;

static int pdc_init(io_ctx *c)
{
    pdc_state *s = calloc(1, sizeof *s);
    uint64_t dims = (uint64_t)(c->bytes / sizeof(double)) * c->nranks;
    s->pdc = PDCinit("pdc");
    s->cont_prop = PDCprop_create(PDC_CONT_CREATE, s->pdc);
    s->cont = PDCcont_create_col("aio_bench", s->cont_prop);
    s->obj_prop = PDCprop_create(PDC_OBJ_CREATE, s->pdc);
    PDCprop_set_obj_type(s->obj_prop, PDC_DOUBLE);
    PDCprop_set_obj_dims(s->obj_prop, 1, &dims);
    PDCprop_set_obj_transfer_region_type(s->obj_prop, PDC_REGION_STATIC);
    c->priv = s;
    return 0;
}

static void pdc_wait(io_ctx *c)
{
    pdc_state *s = c->priv;
    if (!s->pending) return;
    PDCregion_transfer_wait(s->xfer);
    PDCregion_transfer_close(s->xfer);
    PDCregion_close(s->local);
    PDCregion_close(s->remote);
    PDCobj_close(s->obj);
    s->pending = 0;
}

static void pdc_issue(io_ctx *c, int step, const void *buf)
{
    pdc_state *s = c->priv;
    char name[32];
    uint64_t n = c->bytes / sizeof(double), zero = 0, off = n * c->rank;
    pdc_wait(c);
    snprintf(name, sizeof name, "step%03d", step);
    s->obj = PDCobj_create_mpi(s->cont, name, s->obj_prop, 0, c->comm);
    s->local = PDCregion_create(1, &zero, &n);
    s->remote = PDCregion_create(1, &off, &n);
    s->xfer = PDCregion_transfer_create((void *)buf, PDC_WRITE, s->obj, s->local, s->remote);
    PDCregion_transfer_start(s->xfer);
    s->pending = 1;
    if (!c->async) pdc_wait(c);
}

static void pdc_fini(io_ctx *c)
{
    pdc_state *s = c->priv;
    pdc_wait(c);
    /* The PDC server caches regions in memory and flushes them later; flush
     * before the timed run ends so that, as for the other backends, all data
     * has been handed to the storage layer when the run completes. */
    MPI_Barrier(c->comm);
    if (c->rank == 0) PDCobj_flush_all_start();
    MPI_Barrier(c->comm);
    PDCprop_close(s->obj_prop);
    PDCcont_close(s->cont);
    PDCprop_close(s->cont_prop);
    PDCclose(s->pdc);
    free(s);
}
```

```
#else
static int pdc_init(io_ctx *c)
{
    if (c->rank == 0) fprintf(stderr, "[pdc] aio_bench was built without PDC (-DHAVE_PDC)\n");
    return -1;
}
static void pdc_issue(io_ctx *c, int step, const void *buf) { (void)c; (void)step; (void)buf; }
static void pdc_wait(io_ctx *c) { (void)c; }
static void pdc_fini(io_ctx *c) { (void)c; }
#endif

const io_backend io_pdc = {"pdc", pdc_init, pdc_issue, pdc_wait, pdc_fini, NULL, NULL};
```

A.4 Build and run scripts

Listing 11. `Makefile`

```
# Build aio_bench against the stack installed by scripts/build_deps.sh.
#   make            CPU + GPU (needs nvcc)
#   make CUDA=0     CPU only
#   make PDC=1      also build the PDC backend (needs PDC in $(DEPS))
DEPS   ?= $(abspath ../deps/install)
CUDA   ?= 1
PDC    ?= 0
NVCC   ?= /usr/local/cuda/bin/nvcc
CC      = mpicc
CFLAGS  = -O2 -march=native -Wall -Wextra -Wno-unused-parameter -I$(DEPS)/include
LDFLAGS = -L$(DEPS)/lib -Wl,-rpath,$(DEPS)/lib
LIBS    = -lhdf5 -ladios2_c_mpi -ladios2_c -lopenblas -lrt -lpthread -lm

OBJS = aio_bench.o kernels.o io_posix.o io_mpiio.o io_hdf5.o io_adios2.o io_pdc.o
ifeq ($(CUDA),1)
  OBJS += gpu_kernels.o
  LIBS += -L/usr/local/cuda/lib64 -Wl,-rpath,/usr/local/cuda/lib64 -lcudart -lstdc++
else
  OBJS += gpu_stub.o
endif
ifeq ($(PDC),1)
  CFLAGS += -DHAVE_PDC
  LIBS   += -lpdc -lmercury
endif

aio_bench: $(OBJS)
    $(CC) -o $@ $^ $(LDFLAGS) $(LIBS)

# PDC-enabled build as a separate binary, against an existing PDC install
# (see scripts/pdc_env.sh).  make pdc PDC_DIR=... MERCURY_DIR=...
PDC_DIR     ?= /mnt/fast/nlewis/workspace/install/pdc
MERCURY_DIR ?= /mnt/fast/nlewis/workspace/install/mercury
PDC_OBJS = $(filter-out io_pdc.o,$(OBJS)) io_pdc_on.o
io_pdc_on.o: io_pdc.c aio_bench.h
    $(CC) $(CFLAGS) -DHAVE_PDC -I$(PDC_DIR)/include -I$(MERCURY_DIR)/include -c $< -o $@
pdc: $(PDC_OBJS)
    $(CC) -o aio_bench_pdc $^ $(LDFLAGS) $(LIBS) -Wl,--disable-new-dtags -L$(PDC_DIR)/lib -Wl,-rpath,$(PDC_DIR)/lib \
      -L$(MERCURY_DIR)/lib -Wl,-rpath,$(MERCURY_DIR)/lib -lpdc -lpdc_commons -lmercury

%.o: %.c aio_bench.h
    $(CC) $(CFLAGS) -c $<

gpu_kernels.o: gpu_kernels.cu
    $(NVCC) -O2 -arch=native -c $< -o $@

clean:
    rm -f *.o aio_bench aio_bench_pdc

.PHONY: clean pdc
```

Listing 12. `scripts/build_deps.sh`

```
#!/usr/bin/env bash
# Builds the I/O software stack used by the benchmarks into deps/install.
# Usage: src/scripts/build_deps.sh [component...]
#   components: argobots hdf5 volasync adios2 openblas mercury pdc tectonic (default: all)
set -euo pipefail
ROOT=$(cd "$(dirname "$0")/../.." && pwd)
SRC=$ROOT/deps/src
PREFIX=$ROOT/deps/install
J=${J:-48}
export CC=mpicc CXX=mpicxx
mkdir -p "$SRC" "$PREFIX/bin"
cd "$SRC"

```

```
fetch() { [ -e "$2" ] || curl -sL -o "$2" "$1"; }

build_argobots() {
  fetch https://github.com/pmodels/argobots/releases/download/v1.2/argobots-1.2.tar.gz argobots-1.2.tar.gz
  rm -rf argobots-1.2 && tar xf argobots-1.2.tar.gz && cd argobots-1.2
  CC=gcc ./configure --prefix="$PREFIX" && make -j$J && make install
}

build_hdf5() {   # parallel + thread-safe, required by the async VOL
  fetch https://github.com/HDFGroup/hdf5/releases/download/hdf5_1.14.6/hdf5-1.14.6.tar.gz hdf5-1.14.6.tar.gz
  rm -rf hdf5-1.14.6 hdf5-build && tar xf hdf5-1.14.6.tar.gz
  cmake -S hdf5-1.14.6 -B hdf5-build -DCMAKE_INSTALL_PREFIX="$PREFIX" -DCMAKE_BUILD_TYPE=Release \
    -DHDF5_ENABLE_PARALLEL=ON -DHDF5_ENABLE_THREADSAFE=ON -DALLOW_UNSUPPORTED=ON \
    -DHDF5_BUILD_CPP_LIB=OFF -DHDF5_BUILD_FORTRAN=OFF -DHDF5_BUILD_JAVA=OFF \
    -DBUILD_TESTING=OFF -DHDF5_BUILD_EXAMPLES=OFF -DHDF5_ENABLE_Z_LIB_SUPPORT=OFF -DHDF5_ENABLE_SZIP_SUPPORT=OFF
  cmake --build hdf5-build -j$J && cmake --install hdf5-build
}

build_volasync() {
  [ -d vol-async ] || git clone -q https://github.com/hpc-io/vol-async.git
  # 1 s / 100 ms back-offs in H5VL_async_request_wait -> 1 ms (see paper)
  git -C vol-async apply --check "$ROOT/src/scripts/vol-async-wait.patch" 2>/dev/null &&
    git -C vol-async apply "$ROOT/src/scripts/vol-async-wait.patch"
  rm -rf volasync-build
  cmake -S vol-async -B volasync-build -DCMAKE_INSTALL_PREFIX="$PREFIX" -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH="$PREFIX" -DHDF5_DIR="$PREFIX/cmake" -DABT_DIR="$PREFIX" -DENABLE_WRITE_MEMCPY=ON
  cmake --build volasync-build -j$J && cmake --install volasync-build
}

build_adios2() {
  fetch https://github.com/ornladios/ADIOS2/archive/refs/tags/v2.12.1.tar.gz adios2-2.12.1.tar.gz
  rm -rf ADIOS2-2.12.1 adios2-build && tar xf adios2-2.12.1.tar.gz
  # opt-in per-step fdatasync (ADIOS2_FSYNC=1), see adios2-fsync.patch
  (cd ADIOS2-2.12.1/.. && patch -p1 -d ADIOS2-2.12.1 < "$ROOT/src/scripts/adios2-fsync.patch")
  cmake -S ADIOS2-2.12.1 -B adios2-build -DCMAKE_INSTALL_PREFIX="$PREFIX" -DCMAKE_BUILD_TYPE=Release \
    -DADIOS2_USE_MPI=ON -DADIOS2_USE_Fortran=OFF -DADIOS2_USE_Python=OFF -DADIOS2_USE_HDF5=OFF \
    -DADIOS2_USE_SST=OFF -DADIOS2_USE_CUDA=OFF -DADIOS2_USE_Kokkos=OFF -DADIOS2_USE_Campaign=OFF \
    -DADIOS2_USE_ZeroMQ=OFF -DADIOS2_USE_Blosc2=OFF -DADIOS2_USE_BZip2=OFF -DADIOS2_USE_ZFP=OFF \
    -DADIOS2_USE_SZ=OFF -DADIOS2_USE_MGARD=OFF -DADIOS2_USE_PNG=OFF -DADIOS2_USE_AWSSDK=OFF \
    -DADIOS2_USE_DataSpaces=OFF -DADIOS2_USE_DAOS=OFF -DADIOS2_USE_UCX=OFF \
    -DBUILD_TESTING=OFF -DADIOS2_BUILD_EXAMPLES=OFF
  cmake --build adios2-build -j$J && cmake --install adios2-build
}

build_openblas() {
  fetch https://github.com/OpenMathLib/OpenBLAS/releases/download/v0.3.34/OpenBLAS-0.3.34.tar.gz OpenBLAS-0.3.34.tar.
    ↪ gz
  rm -rf OpenBLAS-0.3.34 && tar xf OpenBLAS-0.3.34.tar.gz && cd OpenBLAS-0.3.34
  make CC=gcc FC=false NOFORTRAN=1 USE_OPENMP=0 USE_THREAD=0 TARGET=SAPPHIRERAPIDS -j$J
  make PREFIX="$PREFIX" NOFORTRAN=1 install
}

build_mercury() {  # shared-memory NA plugin only (single node)
  fetch https://github.com/mercury-hpc/mercury/releases/download/v2.3.1/mercury-2.3.1.tar.bz2 mercury-2.3.1.tar.bz2
  rm -rf mercury-2.3.1 mercury-build && tar xf mercury-2.3.1.tar.bz2
  CC=gcc CXX=g++ cmake -S mercury-2.3.1 -B mercury-build -DCMAKE_INSTALL_PREFIX="$PREFIX" -DCMAKE_BUILD_TYPE=Release
    ↪ \
    -DBUILD_SHARED_LIBS=ON -DNA_USE_SM=ON -DNA_USE_OFI=OFF -DNA_USE_UCX=OFF -DMERCURY_USE_BOOST_PP=ON \
    -DBUILD_TESTING=OFF -DMERCURY_USE_CHECKSUMS=OFF
  cmake --build mercury-build -j$J && cmake --install mercury-build
}

build_pdc() {
  [ -d pdc ] || git clone -q https://github.com/hpc-io/pdc.git
  rm -rf pdc-build
  cmake -S pdc -B pdc-build -DCMAKE_INSTALL_PREFIX="$PREFIX" -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_PREFIX_PATH="$PREFIX" -DMERCURY_DIR="$PREFIX/share/cmake/mercury" \
    -DPDC_ENABLE_MPI=ON -DBUILD_TESTING=OFF -DPDC_ENABLE_FASTBIT=OFF -DBUILD_MPI_TESTING=OFF \
    -DPDC_SERVER_CACHE=ON -DCMAKE_C_FLAGS="-Wno-error" -DCMAKE_POLICY_VERSION_MINIMUM=3.5
  cmake --build pdc-build -j$J && cmake --install pdc-build
}

build_tectonic() {
  fetch "https://github.com/tectonic-typesetting/tectonic/releases/download/tectonic%400.17.0/tectonic-0.17.0-x86_64-
    ↪ unknown-linux-musl.tar.gz" tectonic.tar.gz
  tar xf tectonic.tar.gz -C "$PREFIX/bin"
}

COMPONENTS=${*:-argobots hdf5 volasync adios2 openblas mercury pdc tectonic}
for c in $COMPONENTS; do
  echo "=== building $c"; (cd "$SRC" && build_$c) > "$SRC/build_$c.log" 2>&1 && echo "=== $c OK" || { echo "=== $c
    ↪ FAILED (see $SRC/build_$c.log)"; exit 1; }
done
```

Listing 13. scripts/env.sh

```
# Source to set up the environment for aio_bench.
AIO_ROOT=$(cd "$(dirname "${BASH_SOURCE[0]}")/../.." && pwd)
export AIO_ROOT
export AIO_DEPS=$AIO_ROOT/deps/install
export PATH=$AIO_DEPS/bin:/usr/local/cuda/bin:$PATH
export LD_LIBRARY_PATH=$AIO_DEPS/lib:/usr/local/cuda/lib64:${LD_LIBRARY_PATH:-}
export HDF5_PLUGIN_PATH=$AIO_DEPS/lib
export AIO_BENCH=$AIO_ROOT/src/aio_bench
export AIO_SCRATCH=${AIO_SCRATCH:-/mnt/fast/nlewis/aio_scratch}
```

Listing 14. scripts/run_one.sh

```
#!/usr/bin/env bash
# Run one aio_bench configuration.
#   run_one.sh <binding> <nranks> <io> <mode> <kernel> [extra aio_bench args...]
# binding: full     one hardware thread per rank (background threads share it)
#          smt      rank owns its core's two hardware threads
#          reserved rank owns two physical cores (one for background threads)
#          none     no binding (OS scheduler free to migrate)
# Environment: AIO_MEMMAX (cgroup memory cap, default 24G), AIO_CSV,
#   AIO_HDF5_EXE_DCLOSE (async VOL trigger, default 1).
set -euo pipefail
source "$(dirname "$0")/env.sh"
bind=$1 np=$2 io=$3 mode=$4 kernel=$5; shift 5

case $bind in
  full)     list=$(seq -s, 0 $((np - 1))) ;;
  smt)      list=$(for ((r = 0; r < np; r++)); do printf "%d+%d," $r $((r + 48)); done) ;;
  # two physical cores on the same socket (even CPUs: socket 0, odd: socket 1)
  reserved) list=$(for ((r = 0; r < np; r++)); do s=$((r % 2)) i=$((r / 2)); printf "%d+%d," $((4 * i + s)) $((4 * i
     ↪ + 2 + s)); done) ;;
  none)     list= ;;
  *) echo "unknown binding $bind" >&2; exit 1 ;;
esac
bindopt=()
[ -n "$list" ] && bindopt=(-bind-to "user:${list%,}")

envs=()
if [ "$io" = hdf5 ] && [ "$mode" = async ]; then
  envs+=(-genv HDF5_VOL_CONNECTOR "async under_vol=0;under_info={}" -genv HDF5_PLUGIN_PATH "$HDF5_PLUGIN_PATH")
  # Start background execution at dataset close, as the vol-async docs
  # recommend for periodic checkpoint output (default: idle detection).
  envs+=(-genv HDF5_ASYNC_EXE_DCLOSE "${AIO_HDF5_EXE_DCLOSE:-1}")
fi
if [ "$io" = mpiio ] && [ "$mode" = async ]; then
  envs+=(-genv MPIR_CVAR_ASYNC_PROGRESS 1)
fi

# Collective HDF5 I/O by default (AIO_HDF5_INDEP=1 for independent).
envs+=(-genv AIO_HDF5_INDEP "${AIO_HDF5_INDEP:-0}")
dir=$AIO_SCRATCH/run_$$
rm -rf "$dir"; mkdir -p "$dir"
sync  # start every run with no dirty pages from earlier runs
systemd-run --user --scope --quiet -p MemoryMax=${AIO_MEMMAX:-24G} -p MemorySwapMax=0 \
  mpiexec -n "$np" "${bindopt[@]}" "${envs[@]}" \
  "$AIO_BENCH" --io "$io" --mode "$mode" --kernel "$kernel" --dir "$dir" --tag "$bind" \
  ${AIO_CSV:+--csv "$AIO_CSV"} "$@"
rc=$?
rm -rf "$dir"
exit $rc
```

Listing 15. scripts/run_matrix.py

```
#!/usr/bin/env python3
"""Run the experiment matrix for the paper, phase by phase.

    run_matrix.py [phase ...]      phases: io0 main size tera sweep bind gpu
                                     (default: all; breadth-first over repetitions)

Each phase appends to results/<phase>.csv and skips configurations already
present there, so the script can be interrupted and restarted.  The compute
length t_comp of a configuration is derived from the synchronous I/O-only
time measured in phase io0 (t_comp = ratio * t_io_sync per step).
"""
import csv
import os
import subprocess
import sys
import time

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
```

```
RES = os.path.join(ROOT, "results")
RUN = os.path.join(ROOT, "src", "scripts", "run_one.sh")
LOG = os.path.join(RES, "run_matrix.log")

BACKENDS = ["posix", "mpiio", "hdf5", "adios2"]
CPU_KERNELS = ["sleep", "spin", "dgemm", "stream", "halo", "heat"]
RATIO = 1.25          # default t_comp / t_io_sync
NP = 48               # one rank per physical core
ENV = {
    "AIO_ADIOS2_PARAMS": "AggregationType=TwoLevelShm;NumAggregators=8",
    "AIO_HDF5_INDEP": "0",
    "AIO_HDF5_EXE_DCLOSE": "1",
    "AIO_ROMIO_HINTS": "cb_config_list=*:8,cb_nodes=8",
}


def log(msg):
    line = time.strftime("%H:%M:%S ") + msg
    print(line, flush=True)
    with open(LOG, "a") as f:
        f.write(line + "\n")


def memmax(mb, np_):
    # buffer + staging/library copy + kernel arrays per rank, plus page cache
    anon_gb = np_ * (2.2 * mb + 120) / 1024
    return f"{int(min(230, anon_gb + 24))}G"


def done_keys(path):
    if not os.path.exists(path):
        return set()
    with open(path) as f:
        return {key_of(r) for r in csv.DictReader(f)}


def key_of(r):
    return (r["io"], r["mode"], r["kernel"], r["tag"], r["nranks"],
            f'{float(r["mib_per_rank"]):.1f}', r["steps"], f'{float(r["tcomp_target"]):.3f}')


def run(phase, bind, np_, io, mode, kernel, mb, steps, tcomp, extra=(), env=None, rep=0):
    tcomp = round(tcomp, 2) + rep * 1e-3  # repetition index encoded in the key
    path = os.path.join(RES, f"{phase}.csv")
    key = (io, mode, kernel, bind, str(np_), f"{float(mb):.1f}", str(steps), f"{float(tcomp):.3f}")
    if key in done_keys(path):
        return
    e = dict(os.environ, **ENV, AIO_CSV=path, AIO_MEMMAX=memmax(mb, np_))
    if env:
        e.update(env)
    cmd = [RUN, bind, str(np_), io, mode, kernel, "--mb", str(mb), "--steps", str(steps),
           "--tcomp", f"{tcomp:.3f}", "--no-fsync", "--alone-reps", "2", *extra]
    t0 = time.time()
    p = subprocess.run(cmd, env=e, capture_output=True, text=True, timeout=7200)
    tail = (p.stderr.strip().splitlines() or ["?"])[-1]
    log(f"[{phase}] rc={p.returncode} {time.time() - t0:6.1f}s {tail}")
    if p.returncode:
        with open(LOG, "a") as f:
            f.write(p.stdout[-3000:] + p.stderr[-3000:] + "\n")


def tio_sync(io, mb, np_=NP, phase="io0", steps=None, tag="full", rep=None):
    """Per-step synchronous I/O time (end-to-end / steps) of the I/O-only
    run with the same backend, size, ranks and step count, taken from
    `phase` when present there (repetition `rep` only, so t_comp is
    deterministic across restarts), otherwise from io0."""
    for ph in (phase, "io0"):
        path = os.path.join(RES, f"{ph}.csv")
        if not os.path.exists(path):
            continue
        with open(path) as f:
            rows = [r for r in csv.DictReader(f) if r["io"] == io and r["mode"] == "sync"
                    and r["kernel"] == "none" and int(r["nranks"]) == np_ and r["tag"] == tag
                    and abs(float(r["mib_per_rank"]) - mb) < 0.1
                    and (steps is None or ph == "io0" or int(r["steps"]) == steps)
                    and (rep is None or ph == "io0"
                         or round(float(r["tcomp_target"]) * 1000) == rep)]
        if rows:
            return sum(float(r["t_total"]) / int(r["steps"]) for r in rows) / len(rows)
    raise SystemExit(f"no I/O-only baseline for {io} {mb} MiB P={np_}")


def baseline(phase, bind, np_, io, mb, steps, rep=0):
    """I/O-only sync and async runs of this exact configuration; returns the
    per-step synchronous I/O time used to size t_comp."""
```

```
    for mode in ("sync", "async"):
        run(phase, bind, np_, io, mode, "none", mb, steps, 0, rep=rep)
    return tio_sync(io, mb, np_, phase=phase, steps=steps, tag=bind, rep=rep)


SIZES = [16, 64, 256, 1024]   # MiB per rank; x48 ranks = 0.75 .. 48 GiB per step


def phase_io0(rep):
    """Synchronous/asynchronous I/O-only baselines (5 steps) for every size."""
    for mb in SIZES:
        for io in BACKENDS:
            for mode in ("sync", "async"):
                run("io0", "full", NP, io, mode, "none", mb, 5, 0, rep=rep)
    for io in BACKENDS:                 # GPU configuration: 4 ranks
        for mode in ("sync", "async"):
            run("io0", "full", 4, io, mode, "none", 1024, 5, 0, rep=rep)
    for io in BACKENDS:                 # reserved-core configuration: 24 ranks
        run("io0", "reserved", 24, io, "sync", "none", 512, 5, 0, rep=rep)


def phase_main(rep):
    """RQ1/RQ2: every CPU surrogate, 12 GiB/step x 10 steps = 120 GiB."""
    mb, steps = 256, 10
    for io in BACKENDS:
        tc = RATIO * baseline("main", "full", NP, io, mb, steps, rep)
        for k in CPU_KERNELS:
            for mode in ("sync", "async"):
                run("main", "full", NP, io, mode, k, mb, steps, tc, rep=rep)


def phase_size(rep):
    """Size sweep: 0.75 .. 48 GiB per step, 10 steps (7.5 GiB .. 480 GiB)."""
    for mb in SIZES:
        for io in BACKENDS:
            tc = RATIO * baseline("size", "full", NP, io, mb, 10, rep)
            for k in ["sleep", "dgemm", "stream", "heat"]:
                for mode in ("sync", "async"):
                    run("size", "full", NP, io, mode, k, mb, 10, tc, rep=rep)


def phase_tera(rep):
    """~1 TB of output: 48 GiB/step x 20 steps = 960 GiB."""
    mb = 1024
    for io in BACKENDS:
        tc = RATIO * baseline("tera", "full", NP, io, mb, 20, rep)
        for k in ["sleep", "stream", "heat"]:
            for mode in ("sync", "async"):
                run("tera", "full", NP, io, mode, k, mb, 20, tc, rep=rep)


def phase_sweep(rep):
    """t_comp / t_io sweep, 12 GiB/step; t_io from the main phase (rep 0)."""
    mb = 256
    for io in BACKENDS:
        t = tio_sync(io, mb, phase="main", steps=10, rep=0)
        for r in (0.25, 0.5, 0.75, 1.0, 1.5, 2.0, 3.0):
            for k in ["sleep", "dgemm", "stream"]:
                for mode in ("sync", "async"):
                    run("sweep", "full", NP, io, mode, k, mb, 10, r * t, rep=rep)


def phase_bind(rep):
    """RQ4a: core allocation.  Same 12 GiB per step in every configuration."""
    for io in BACKENDS:
        tc = RATIO * tio_sync(io, 256, phase="main", steps=10, rep=0)
        for bind in ("smt", "none"):
            baseline("bind", bind, NP, io, 256, 10, rep)
        baseline("bind", "reserved", 24, io, 512, 10, rep)
        for k in ["sleep", "dgemm", "stream"]:
            for mode in ("sync", "async"):
                for bind in ("full", "smt", "none"):
                    run("bind", bind, NP, io, mode, k, 256, 10, tc, rep=rep)
                run("bind", "reserved", 24, io, mode, k, 512, 10, tc, rep=rep)


def phase_gpu(rep):
    """RQ4b: GPU-resident compute, 4 ranks sharing the GPU, 4 GiB/step."""
    for io in BACKENDS:
        tc = RATIO * baseline("gpu", "full", 4, io, 1024, 10, rep)
        for k in ["sleep", "gpu-block", "gpu-spin", "dgemm"]:
            for mode in ("sync", "async"):
                run("gpu", "full", 4, io, mode, k, 1024, 10, tc, rep=rep)


```

```
def phase_sizelarge(rep):
    """48 GiB per step (480 GiB per run).  POSIX and ADIOS2 ran first; MPI-IO
    and HDF5 (30 to 40 minutes per run at this size) were added afterwards."""
    for io in ["posix", "adios2", "mpiio", "hdf5"]:
        tc = RATIO * baseline("size", "full", NP, io, 1024, 10, rep)
        for k in ["sleep", "dgemm", "stream", "heat"]:
            for mode in ("sync", "async"):
                run("size", "full", NP, io, mode, k, 1024, 10, tc, rep=rep)


def phase_recheck(rep):
    """Re-run the main configuration's synchronous sleep and spin runs, with
    spin first, to test whether the sleep run's slower synchronous I/O in the
    main phase was caused by its position right after the baselines."""
    for io in BACKENDS:
        tc = RATIO * tio_sync(io, 256, phase="main", steps=10, rep=0)
        for k in ("spin", "sleep"):
            run("recheck", "full", NP, io, "sync", k, 256, 10, tc, rep=rep)


RUN_PDC = os.path.join(ROOT, "src", "scripts", "run_pdc.sh")


def run_pdc(phase, ns, nc, mode, kernel, mb, steps, tcomp, rep=0, smt=False):
    """Like run(), but through run_pdc.sh (PDC servers on reserved cores).
    smt=True also gives every client its SMT sibling (tag pdc<ns>smt)."""
    tcomp = round(tcomp, 2) + rep * 1e-3
    path = os.path.join(RES, f"{phase}.csv")
    tag = f"pdc{ns}" + ("smt" if smt else "")
    key = ("pdc", mode, kernel, tag, str(nc), f"{float(mb):.1f}", str(steps), f"{float(tcomp):.3f}")
    if key in done_keys(path):
        return
    e = dict(os.environ, AIO_CSV=path, AIO_MEMMAX=memmax(mb, nc), AIO_PDC_TAG=tag)
    if smt:
        e["AIO_PDC_CLIENT_CPUS"] = ",".join(f"{c}+{c + 48}" for c in range(ns, ns + nc))
    cmd = [RUN_PDC, str(ns), str(nc), mode, kernel, "--mb", str(mb), "--steps", str(steps),
           "--tcomp", f"{tcomp:.3f}", "--no-fsync", "--alone-reps", "2"]
    t0 = time.time()
    # Own process group, so a hung run (server, clients, mpiexec) can be
    # killed as a whole instead of leaving orphans that block later runs.
    pr = subprocess.Popen(cmd, env=e, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
                          text=True, start_new_session=True)
    try:
        out, err = pr.communicate(timeout=PDC_TIMEOUT)
        rc = pr.returncode
    except subprocess.TimeoutExpired:
        os.killpg(pr.pid, 9)
        out, err = pr.communicate()
        subprocess.run(["pkill", "-9", "-x", "pdc_server"])
        subprocess.run(["pkill", "-9", "-x", "aio_bench_pdc"])
        rc, err = "timeout", (err or "") + f"\nkilled after {PDC_TIMEOUT} s"
    tail = (err.strip().splitlines() or ["?"])[-1]
    log(f"[{phase}] rc={rc} {time.time() - t0:6.1f}s {tail}")
    if rc != 0:
        with open(LOG, "a") as f:
            f.write(out[-3000:] + err[-3000:] + "\n")


PDC_TIMEOUT = 1800  # seconds per PDC run


def phase_pdc(rep):
    """PDC in its deployment model: 1 server on a reserved core (and its SMT
    sibling), 47 clients on the remaining cores, 256 MiB per client per step,
    10 steps, every CPU surrogate."""
    ns, nc, mb, steps = 1, 47, 256, 10
    for mode in ("sync", "async"):
        run_pdc("pdc", ns, nc, mode, "none", mb, steps, 0, rep=rep)
    t = tio_sync("pdc", mb, nc, phase="pdc", steps=steps, tag=f"pdc{ns}", rep=rep)
    for k in CPU_KERNELS:
        for mode in ("sync", "async"):
            # Async halo with the default placement did not finish in 30
            # minutes in two attempts (time slicing with the client progress
            # thread); it is measured with the SMT placement below instead.
            if (k, mode) != ("halo", "async"):
                run_pdc("pdc", ns, nc, mode, k, mb, steps, RATIO * t, rep=rep)
    # The client's Mercury progress thread polls on the rank's core; with the
    # SMT sibling free for it, does the async slowdown go away?
    for k in ("sleep", "spin", "dgemm", "stream", "halo", "heat"):
        run_pdc("pdc", ns, nc, "async", k, mb, steps, RATIO * t, rep=rep, smt=True)


def pdc_baseline(phase, ns, nc, mb, steps, rep=0):
    """PDC I/O-only sync and async runs; returns the sync I/O time per step."""
    for mode in ("sync", "async"):
```

```
        run_pdc(phase, ns, nc, mode, "none", mb, steps, 0, rep=rep)
    return tio_sync("pdc", mb, nc, phase=phase, steps=steps, tag=f"pdc{ns}", rep=rep)


def phase_pdcgpu(rep):
    """PDC with GPU-resident compute: 1 server, 4 clients sharing the GPU,
    4 GiB per step, as in phase_gpu."""
    ns, nc, mb, steps = 1, 4, 1024, 10
    tc = RATIO * pdc_baseline("pdcgpu", ns, nc, mb, steps, rep)
    for k in ["sleep", "gpu-block", "gpu-spin", "dgemm"]:
        for mode in ("sync", "async"):
            run_pdc("pdcgpu", ns, nc, mode, k, mb, steps, tc, rep=rep)


def phase_pdcsize(rep):
    """PDC at 0.73 and 2.9 GiB per step (47 clients); 11.75 GiB is the main
    experiment.  47 GiB per step takes about 90 minutes per run through one
    server and is not run."""
    ns, nc, steps = 1, 47, 10
    for mb in (16, 64):
        tc = RATIO * pdc_baseline("pdcsize", ns, nc, mb, steps, rep)
        for k in ["sleep", "dgemm", "stream", "heat"]:
            for mode in ("sync", "async"):
                run_pdc("pdcsize", ns, nc, mode, k, mb, steps, tc, rep=rep)


def phase_pdcsweep(rep):
    """PDC t_comp / t_io sweep at 11.75 GiB per step; 1.25 is the main
    experiment."""
    ns, nc, mb, steps = 1, 47, 256, 10
    t = tio_sync("pdc", mb, nc, phase="pdc", steps=steps, tag=f"pdc{ns}", rep=0)
    for r in (0.25, 0.5, 1.0):
        for k in ["sleep", "dgemm", "stream"]:
            for mode in ("sync", "async"):
                run_pdc("pdcsweep", ns, nc, mode, k, mb, steps, r * t, rep=rep)


# Default order: cheaper phases first so every figure gets data early.
PHASES = {"io0": phase_io0, "main": phase_main, "gpu": phase_gpu, "bind": phase_bind,
          "sweep": phase_sweep, "size": phase_size, "tera": phase_tera,
          "sizelarge": phase_sizelarge, "recheck": phase_recheck, "pdc": phase_pdc,
          "pdcgpu": phase_pdcgpu, "pdcsize": phase_pdcsize, "pdcsweep": phase_pdcsweep}
# Repetitions per phase.  Execution is breadth-first: repetition 0 of every
# phase runs before repetition 1 of any, so every result gets data early.
# One run per configuration (repetitions are future work); io0 was run twice.
REPS_OF = {"io0": 2, "main": 1, "size": 1, "tera": 1, "sweep": 1, "bind": 1, "gpu": 1,
           "sizelarge": 1, "recheck": 1, "pdc": 1, "pdcgpu": 1, "pdcsize": 1,
           "pdcsweep": 1}

if __name__ == "__main__":
    os.makedirs(RES, exist_ok=True)
    order = sys.argv[1:] or list(PHASES)
    for rep in range(max(REPS_OF[p] for p in order)):
        for ph in order:
            if rep < REPS_OF[ph]:
                log(f"=== phase {ph} rep {rep}")
                PHASES[ph](rep)
    log("=== all requested phases done")
```

A.5 Library patches

Listing 16. `scripts/vol-async-wait.patch`

```
diff --git a/src/h5_async_vol.c b/src/h5_async_vol.c
index 938b00a..dcab80a 100644
--- a/src/h5_async_vol.c
+++ b/src/h5_async_vol.c
@@ -22800,7 +22800,7 @@ H5VL_async_request_wait(void *obj, uint64_t timeout, H5VL_request_status_t *stat
         }

         if (timeout > 0)
-            usleep(100000);
+            usleep(1000); /* aio_bench patch: was 100 ms */
         now_time = clock();
         elapsed  = (double)(now_time - start_time) / CLOCKS_PER_SEC;

@@ -22824,7 +22824,7 @@ done:
         func_log_uint64_1(__func__, "                    timeout", timeout);
         func_log_int1(__func__, "                  iter", tmp_cnt);
         if (tmp_cnt > 0)
-            usleep(1000000);
+            usleep(1000); /* aio_bench patch: was 1 s */
         tmp_cnt++;
     }
```

```
    func_log_int1(__func__, "acquired global lock, count", mutex_count);
```

Listing 17. scripts/adios2-fsync.patch

```
diff -ru adios2-orig/ADIOS2-2.12.1/source/adios2/engine/bp5/BP5Writer_EveryoneWrites_Async.cpp ADIOS2-2.12.1/source/adios2/engine/bp5/BP5Writer_EveryoneWrites_Async.cpp
--- a/source/adios2/engine/bp5/BP5Writer_EveryoneWrites_Async.cpp   2026-04-29 00:58:00.000000000 +0000
+++ b/source/adios2/engine/bp5/BP5Writer_EveryoneWrites_Async.cpp   2026-10-05 15:50:24.046495871 +0000
@@ -209,6 +209,7 @@
             info->tokenChain->RecvToken();
         }
     }
+    info->tm->FlushFiles(); /* aio_bench patch */
     info->tm->FinalizeSegment();
     delete info->Data;
     return 1;
diff -ru adios2-orig/ADIOS2-2.12.1/source/adios2/engine/bp5/BP5Writer_TwoLevelShm_Async.cpp ADIOS2-2.12.1/source/adios2/engine/bp5/BP5Writer_TwoLevelShm_Async.cpp
--- a/source/adios2/engine/bp5/BP5Writer_TwoLevelShm_Async.cpp  2026-04-29 00:58:00.000000000 +0000
+++ b/source/adios2/engine/bp5/BP5Writer_TwoLevelShm_Async.cpp  2026-10-05 15:50:34.154057010 +0000
@@ -145,6 +145,7 @@
         info->tokenChain->SendToken(nextWriterPos);
         AsyncWriteThread_TwoLevelShm_Aggregator(info);
         info->tokenChain->RecvToken();
+        info->tm->FlushFiles(); /* aio_bench patch */
         info->tm->FinalizeSegment();
     }
     else
diff -ru adios2-orig/ADIOS2-2.12.1/source/adios2/toolkit/transport/file/FilePOSIX.cpp ADIOS2-2.12.1/source/adios2/toolkit/transport/file/FilePOSIX.cpp
--- a/source/adios2/toolkit/transport/file/FilePOSIX.cpp    2026-04-29 00:58:00.000000000 +0000
+++ b/source/adios2/toolkit/transport/file/FilePOSIX.cpp    2026-10-05 15:50:24.046161613 +0000
@@ -10,6 +10,7 @@
  *
  */
 #include "FilePOSIX.h"
+#include <cstdlib>
 #include "adios2/helper/adiosLog.h"
 #include "adios2/helper/adiosString.h"

@@ -591,15 +592,13 @@

 void FilePOSIX::Flush()
 {
-    /* Turn this off now because BP3/BP4 calls manager Flush and this syncing
-     * slows down IO performance */
-#if 0
-#if (_POSIX_C_SOURCE >= 199309L || _XOPEN_SOURCE >= 500)
-    fdatasync(m_FileDescriptor);
-#else
-    fsync(m_FileDescriptor)
-#endif
-#endif
+    /* aio_bench patch: BP3/BP4/BP5 never sync data; with ADIOS2_FSYNC=1 make
+     * every transport flush durable so that a completed step is on the device. */
+    static const bool doSync = getenv("ADIOS2_FSYNC") && atoi(getenv("ADIOS2_FSYNC"));
+    if (doSync && m_FileDescriptor >= 0)
+    {
+        fdatasync(m_FileDescriptor);
+    }
 }

 void FilePOSIX::Close()
```

A.6 Analysis scripts

Listing 18. analysis/plots_measured.py

```
#!/usr/bin/env python3
"""Figures, tables and LaTeX macros from measured results (results/*.csv).

    python3 src/analysis/plots_measured.py

Writes paper/figures/m_*.pdf and paper/results_macros.tex.  Every number
quoted in the paper's results section should come from the macros file.

Metrics (per configuration; sync and async rows share io, kernel, binding,
ranks, size, steps and t_comp):
  speedup    S = T_sync / T_async                (end-to-end, all steps)
  slowdown   t_comp(async) / t_comp(sync)        (compute phase; also vs. alone)
  exposed    observed async I/O per step / sync I/O per step
  overlap    (t_io + t_comp - t_epoch) / min(t_io, t_comp)   (IMB-IO)
```

```
"""
import csv
import glob
import os
from collections import defaultdict

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.normpath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
OUT = os.path.join(ROOT, "paper", "figures")
MACROS = os.path.join(ROOT, "paper", "results_macros.tex")

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]
GRAY, INK, INK2 = "#8a8985", "#0b0b0b", "#52514e"
BACKENDS = ["posix", "mpiio", "hdf5", "adios2"]
BNAME = {"posix": "POSIX AIO", "hdf5": "HDF5 async VOL", "adios2": "ADIOS2 BP5 async",
         "mpiio": "MPI-IO iwrite", "pdc": "PDC"}
KERNELS = ["sleep", "spin", "dgemm", "stream", "halo", "heat"]
KNAME = {"sleep": "sleep", "spin": "spin", "dgemm": "DGEMM", "stream": "STREAM",
         "halo": "halo", "heat": "heat", "gpu-block": "GPU,\nhost blocks",
         "gpu-spin": "GPU,\nhost spins", "none": "none"}

plt.rcParams.update({
    "font.family": "sans-serif", "font.sans-serif": ["TeX Gyre Heros", "DejaVu Sans"],
    "mathtext.fontset": "custom", "mathtext.rm": "TeX Gyre Heros",
    "mathtext.it": "TeX Gyre Heros:italic", "mathtext.bf": "TeX Gyre Heros:bold",
    "font.size": 7.5, "axes.titlesize": 7.5,
    "axes.labelsize": 7.5, "legend.fontsize": 6.5, "xtick.labelsize": 7,
    "ytick.labelsize": 7, "axes.edgecolor": INK2, "axes.labelcolor": INK,
    "xtick.color": INK2, "ytick.color": INK2, "axes.spines.top": False,
    "axes.spines.right": False, "axes.grid": True, "axes.grid.axis": "y",
    "grid.color": "#e4e3df", "grid.linewidth": 0.5, "axes.axisbelow": True,
    "legend.frameon": False, "savefig.dpi": 300,
})


# ---------------------------------------------------------------- data --
def load(phase):
    path = os.path.join(RES, f"{phase}.csv")
    if not os.path.exists(path):
        return []
    rows = []
    with open(path) as f:
        for r in csv.DictReader(f):
            for k in ("nranks", "steps", "durable"):
                r[k] = int(r[k])
            for k in ("mib_per_rank", "tcomp_target", "units", "t_alone", "t_comp", "slowdown",
                      "t_io_step", "t_final", "t_io_obs", "t_total", "t_init", "gb"):
                r[k] = float(r[k])
            r["tc"] = round(r["tcomp_target"], 2)  # strip the repetition offset
            rows.append(r)
    return rows


def pairs(rows):
    """Group sync/async rows of the same configuration; returns dict
    key -> {"sync": [rows], "async": [rows]}."""
    g = defaultdict(lambda: {"sync": [], "async": []})
    for r in rows:
        key = (r["io"], r["kernel"], r["tag"], r["nranks"], r["mib_per_rank"], r["steps"], r["tc"])
        g[key][r["mode"]].append(r)
    return {k: v for k, v in g.items() if v["sync"] and v["async"]}


def mean(xs):
    return float(np.mean(xs)) if xs else float("nan")


def metrics(p, t_io_sync=None):
    ts = [r["t_total"] for r in p["sync"]]
    ta = [r["t_total"] for r in p["async"]]
    a = p["async"]
    steps = a[0]["steps"]
    m = {
        "S": mean(ts) / mean(ta),
        "S_lo": min(ts) / max(ta) if len(ts) > 1 else None,
        "S_hi": max(ts) / min(ta) if len(ts) > 1 else None,
        # primary: async compute phase vs. the same configuration's sync
        # compute phase (no background I/O during compute, same CPU frequency
        # regime); secondary: vs. the standalone calibration baseline
```

```
        "slow": (mean([r["t_comp"] for r in a]) / mean([r["t_comp"] for r in p["sync"]])
                 if a[0]["t_alone"] > 0 else 1.0),
        "slow_alone": mean([r["slowdown"] for r in a]) if a[0]["t_alone"] > 0 else 1.0,
        "T_sync": mean(ts), "T_async": mean(ta),
        "io_obs_async": mean([r["t_io_obs"] for r in a]) / steps,
        "t_alone": mean([r["t_alone"] for r in a]),
        "n": min(len(ts), len(ta)),
    }
    if t_io_sync:
        m["exposed"] = m["io_obs_async"] / t_io_sync
        t_epoch = m["T_async"] / steps
        tc = m["t_alone"] if m["t_alone"] > 0 else a[0]["tcomp_target"]
        m["overlap"] = (t_io_sync + tc - t_epoch) / min(t_io_sync, tc)
    return m


def io_sync_per_step(rows, io, mb, np_=48, tag="full"):
    xs = [r["t_total"] / r["steps"] for r in rows if r["io"] == io and r["mode"] == "sync"
          and r["kernel"] == "none" and r["nranks"] == np_ and abs(r["mib_per_rank"] - mb) < .1
          and r["tag"] == tag]
    return mean(xs) if xs else None


# ---------------------------------------------------------------- macros --
MAC = {}


def mac(name, value, fmt="{:.2f}"):
    v = fmt.format(value) if isinstance(value, (int, float, np.floating)) else value
    MAC[name] = "0" if v in ("-0", "-0.0", "-0.00") else v


def write_macros():
    with open(MACROS, "w") as f:
        f.write("% Generated by src/analysis/plots_measured.py ; do not edit.\n")
        for k in sorted(MAC):
            f.write(f"\\newcommand{{\\{k}}}{{{MAC[k]}\\xspace}}\n")


SYNC_IO = {}  # filled in main(): backend -> {kernel: sync I/O s/step}
SZ = {16: "Xs", 64: "Sm", 256: "Md", 512: "Hf", 1024: "Lg"}  # size codes (macro names are letters only)


def cname(*parts):
    """Macro-safe name: letters only."""
    s = "".join(p.replace("-", " ").title().replace(" ", "") for p in parts)
    return "".join(ch for ch in s if ch.isalpha())


# ---------------------------------------------------------------- figures --
def bars(ax, groups, series, vals, err=None, width=0.8):
    n = len(series)
    w = width / n
    x = np.arange(len(groups))
    for i, s in enumerate(series):
        v = np.array(vals[s], dtype=float)
        e = None
        if err and s in err:
            lo, hi = np.array(err[s][0], dtype=float), np.array(err[s][1], dtype=float)
            e = np.vstack([np.clip(v - lo, 0, None), np.clip(hi - v, 0, None)])
            e = np.nan_to_num(e)
        ax.bar(x + (i - (n - 1) / 2) * w, v, w * 0.88, color=C[i], label=BNAME.get(s, s),
               yerr=e, error_kw=dict(lw=0.6, capsize=1.2, ecolor=INK2))
    ax.set_xticks(x, groups, rotation=30, ha="right")


def fig_main(rows):
    P = pairs(rows)
    sp, lo, hi, slow, expo = ({b: [] for b in BACKENDS} for _ in range(5))
    for b in BACKENDS:
        tio = (io_sync_per_step(rows, b, 256, np_=47, tag="pdc1") if b == "pdc"
               else io_sync_per_step(rows, b, 256))
        for k in KERNELS:
            ks = [v for kk, v in P.items() if kk[0] == b and kk[1] == k]
            if not ks:
                for d in (sp, lo, hi, slow, expo):
                    d[b].append(np.nan)
                continue
            merged = {"sync": sum((v["sync"] for v in ks), []),
                      "async": sum((v["async"] for v in ks), [])}
            m = metrics(merged, tio)
            sp[b].append(m["S"]); slow[b].append(m["slow"]); expo[b].append(m.get("exposed", np.nan))
            S_each = [s["t_total"] / a["t_total"] for s, a in zip(merged["sync"], merged["async"])]
            lo[b].append(min(S_each)); hi[b].append(max(S_each))
            mac(cname("sp", b, k), m["S"]); mac(cname("slow", b, k), m["slow"])
```

```
            mac(cname("ovl", b, k), m.get("overlap", np.nan))
            mac(cname("expo", b, k), m.get("exposed", np.nan))
            mac(cname("reps", b, k), m["n"], "{:d}")
        if tio:
            mac(cname("tio", b), tio)
    ticks = [KNAME[k] for k in KERNELS]

    fig, ax = plt.subplots(figsize=(3.4, 2.4))
    bars(ax, ticks, BACKENDS, sp, {b: (lo[b], hi[b]) for b in BACKENDS})
    ax.axhline(1, color=INK2, lw=0.8)
    ax.axhline(2.25 / 1.25, color=GRAY, lw=0.8, ls="--")
    ax.text(len(ticks) - 0.5, 2.25 / 1.25 + 0.03, "ideal overlap", color=INK2,
            fontsize=6, ha="right", va="bottom")
    ax.set_ylabel("End-to-end speedup\n(sync / async)")
    ax.set_ylim(0, 2.1)
    ax.legend(ncol=2, loc="lower center", bbox_to_anchor=(0.5, 1.0), columnspacing=0.8,
              handlelength=1.2)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "m_speedup.pdf"))
    plt.close(fig)

    fig, (a, b2) = plt.subplots(1, 2, figsize=(7.0, 1.9))
    bars(a, ticks, BACKENDS, slow)
    a.axhline(1, color=INK2, lw=0.8)
    a.set_ylabel("Compute slowdown\n($t_{comp}$ async / sync)")
    a.set_title("(a) Cost paid by the application", loc="left")
    bars(b2, ticks, BACKENDS, expo)
    b2.axhline(1, color=GRAY, lw=0.8, ls="--")
    b2.set_ylabel("Exposed I/O per step\n(async / sync I/O time)")
    b2.set_title("(b) I/O left exposed after overlap", loc="left")
    h, l = a.get_legend_handles_labels()
    fig.legend(h, l, ncol=4, loc="upper center", bbox_to_anchor=(0.5, 1.02))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_breakdown.pdf"))
    plt.close(fig)


def excess(rows):
    """Observed asynchronous run time with surrogate k divided by the time the
    sleep run predicts for the same compute length:
        pred_k = T_async(sleep) + n * (t_alone_k - t_alone_sleep)
    This compares asynchronous runs only, so it does not depend on the
    (noisier) synchronous runs.  Returns {backend: {kernel: ratio}}."""
    P = pairs(rows)
    out = {}
    for b in BACKENDS:
        def asy(k):
            v = [x for kk, p in P.items() if kk[0] == b and kk[1] == k for x in p["async"]]
            return v
        sl = asy("sleep")
        if not sl:
            continue
        Ts, ta_s, n = mean([r["t_total"] for r in sl]), mean([r["t_alone"] for r in sl]), sl[0]["steps"]
        out[b] = {}
        for k in KERNELS:
            if k == "sleep" or not asy(k):
                continue
            a_ = asy(k)
            pred = Ts + n * (mean([r["t_alone"] for r in a_]) - ta_s)
            out[b][k] = mean([r["t_total"] for r in a_]) / pred
    return out


def fig_excess(rows):
    ex = excess(rows)
    ks = [k for k in KERNELS if k != "sleep"]
    vals = {b: [ex.get(b, {}).get(k, np.nan) for k in ks] for b in BACKENDS}
    fig, ax = plt.subplots(figsize=(3.4, 2.4))
    bars(ax, [KNAME[k] for k in ks], BACKENDS, vals)
    ax.axhline(1, color=INK2, lw=0.8)
    ax.set_ylim(0.8, max(1.6, np.nanmax([v for b in vals for v in vals[b]]) + 0.05))
    ax.set_ylabel("Observed / sleep-predicted\nrun time (async)")
    ax.legend(ncol=2, loc="lower center", bbox_to_anchor=(0.5, 1.0), columnspacing=0.8,
              handlelength=1.2)
    fig.tight_layout()
    fig.savefig(os.path.join(OUT, "m_excess.pdf"))
    plt.close(fig)
    allv = []
    for b in ex:
        v = list(ex[b].values())
        allv += v
        for k, x in ex[b].items():
            mac(cname("exc", b, k), x)
            mac(cname("excPct", b, k), (x - 1) * 100, "{:.0f}")
        mac(cname("excMinPct", b), (min(v) - 1) * 100, "{:.0f}")
```

```
        mac(cname("excMaxPct", b), (max(v) - 1) * 100, "{:.0f}")
    if allv:
        mac("ExcN", len(allv), "{:d}")
        mac("ExcLonger", sum(x > 1.0 for x in allv), "{:d}")
        mac("ExcLongerFive", sum(x >= 1.05 for x in allv), "{:d}")
        mac("ExcMaxPct", (max(allv) - 1) * 100, "{:.0f}")


def pdc_smt(rows):
    """PDC async runs whose clients also own their SMT sibling (tag pdc1smt),
    against the default placement (pdc1): compute slowdown and how much longer
    than the same placement's sleep run predicts."""
    def a(tag, k):
        return [r for r in rows if r["io"] == "pdc" and r["mode"] == "async"
                and r["tag"] == tag and r["kernel"] == k]
    sl = a("pdc1smt", "sleep")
    if not sl:
        return
    Ts, ta_s, n = sl[0]["t_total"], sl[0]["t_alone"], sl[0]["steps"]
    for k in ("spin", "dgemm"):
        x = a("pdc1smt", k)
        if not x:
            continue
        r = x[0]
        mac(cname("pdcSmtSlow", k), r["t_comp"] / r["t_alone"])
        mac(cname("pdcSmtExcPct", k), (r["t_total"] / (Ts + n * (r["t_alone"] - ta_s)) - 1) * 100, "{:.0f}")
        mac(cname("pdcSmtTotal", k), r["t_total"], "{:.0f}")
        d = a("pdc1", k)
        if d:
            mac(cname("pdcSlow", k), d[0]["t_comp"] / d[0]["t_alone"])
            mac(cname("pdcTotal", k), d[0]["t_total"], "{:.0f}")
    mac("pdcSmtSleepTotal", Ts, "{:.0f}")


def sync_io_per_step(rows):
    """Synchronous I/O time per step implied by each sync run:
    (T_sync - n * t_comp) / n."""
    P = pairs(rows)
    out = defaultdict(dict)
    for kk, p in P.items():
        if kk[1] in KERNELS:
            v = [(r["t_total"] - r["steps"] * r["t_comp"]) / r["steps"] for r in p["sync"]]
            out[kk[0]][kk[1]] = mean(v)
    for b, d in out.items():
        others = [v for k, v in d.items() if k != "sleep"]
        if "sleep" in d and others:
            mac(cname("syncIoSleep", b), d["sleep"], "{:.1f}")
            mac(cname("syncIoOtherMin", b), min(others), "{:.1f}")
            mac(cname("syncIoOtherMax", b), max(others), "{:.1f}")
    return out


def fig_decomp(rows):
    """Where the asynchronous run time goes, normalized to the sync run:
    sync compute + compute inflation + exposed I/O = T_async / T_sync."""
    P = pairs(rows)
    fig, axs = plt.subplots(1, len(BACKENDS), figsize=(7.0, 2.1), sharey=True)
    for ax, b in zip(axs, BACKENDS):
        base, infl, expo, xs = [], [], [], []
        for k in KERNELS:
            ks = [v for kk, v in P.items() if kk[0] == b and kk[1] == k]
            if not ks:
                continue
            sy = sum((v["sync"] for v in ks), [])
            asy = sum((v["async"] for v in ks), [])
            n = asy[0]["steps"]
            Ts, Ta = mean([r["t_total"] for r in sy]), mean([r["t_total"] for r in asy])
            cs, ca = mean([r["t_comp"] for r in sy]), mean([r["t_comp"] for r in asy])
            base.append(n * cs / Ts)
            infl.append(max(n * (ca - cs), 0) / Ts)
            expo.append((Ta - n * ca) / Ts)
            xs.append(KNAME[k])
            mac(cname("dcInfl", b, k), n * (ca - cs) / Ts)
            mac(cname("dcExpo", b, k), (Ta - n * ca) / Ts)
            mac(cname("dcExpoSec", b, k), (Ta - n * ca), "{:.0f}")
            mac(cname("dcInflSec", b, k), n * (ca - cs), "{:.0f}")
        x = np.arange(len(xs))
        ax.bar(x, base, 0.7, color=GRAY, label="compute (as in sync run)")
        ax.bar(x, infl, 0.7, bottom=base, color=C[1], label="compute inflation")
        ax.bar(x, expo, 0.7, bottom=np.add(base, infl), color=C[0], label="exposed I/O")
        ax.axhline(1, color=INK2, lw=0.8, ls="--")
        ax.set_xticks(x, xs, rotation=45, ha="right")
        ax.set_title(BNAME[b], loc="left")
    axs[0].set_ylabel("$T_{async}$ / $T_{sync}$")
    h, l = axs[0].get_legend_handles_labels()
```

```
    fig.legend(h, l, ncol=3, loc="upper center", bbox_to_anchor=(0.5, 1.03))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_decomp.pdf"))
    plt.close(fig)


def fig_size(rows):
    """Speedup vs. output size per step, one panel per backend."""
    P = pairs(rows)
    ks = ["sleep", "dgemm", "stream", "heat"]
    fig, axs = plt.subplots(1, len(BACKENDS), figsize=(7.0, 2.0), sharey=True)
    for ax, b in zip(axs, BACKENDS):
        for i, k in enumerate(ks):
            pts = sorted((kk[4] * kk[3] / 1024, metrics(v)["S"], int(kk[4])) for kk, v in P.items()
                         if kk[0] == b and kk[1] == k)
            if not pts:
                continue
            ax.plot([p_[0] for p_ in pts], [p_[1] for p_ in pts], marker="os^D"[i], ms=3.5,
                    lw=1.5, color=C[i], label=KNAME[k], markeredgecolor="white",
                    markeredgewidth=0.5)
            for gib, sp, mb in pts:
                mac(cname("size", b, k, SZ[mb]), sp)
        if not any(int(kk[4]) == 1024 for kk in P if kk[0] == b):
            ax.text(48, 0.5, "48 GiB\nnot run", transform=ax.get_xaxis_transform(),
                    ha="center", va="center", fontsize=6, color=INK2)
        ax.set_xscale("log")
        ax.set_xticks([0.75, 3, 12, 48], ["0.75", "3", "12", "48"])
        ax.minorticks_off()
        ax.axhline(1, color=INK2, lw=0.8)
        ax.set_title(BNAME[b], loc="left")
        ax.set_xlabel("GiB per step")
    axs[0].set_ylabel("End-to-end speedup")
    h, l = axs[0].get_legend_handles_labels()
    fig.legend(h, l, ncol=4, loc="upper center", bbox_to_anchor=(0.5, 1.03))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_size.pdf"))
    plt.close(fig)


def tio_main_rep0(main_rows, b):
    """Synchronous I/O time per step of the main phase's first I/O-only run:
    the value the sweep phase was sized from (run_matrix.phase_sweep)."""
    xs = [r["t_total"] / r["steps"] for r in main_rows if r["io"] == b and r["mode"] == "sync"
          and r["kernel"] == "none" and round(r["tcomp_target"] * 1000) == 0]
    return mean(xs) if xs else None


def fig_sweep(rows, main_rows):
    P = pairs(rows)
    fig, axs = plt.subplots(1, len(BACKENDS), figsize=(7.0, 2.0), sharey=True)
    for ax, b in zip(axs, BACKENDS):
        tio = tio_main_rep0(main_rows, b)
        if not tio:
            continue
        r = np.linspace(0.25, 3, 60)
        ax.plot(r, (r + 1) / np.maximum(r, 1), color=GRAY, ls="--", lw=1, label="ideal")
        for i, k in enumerate(["sleep", "dgemm", "stream"]):
            pts = sorted((round(kk[6] / tio, 2), metrics(v)["S"]) for kk, v in P.items()
                         if kk[0] == b and kk[1] == k)
            if pts:
                ax.plot(*zip(*pts), marker="os^"[i], ms=3.5, lw=1.5, color=C[i], label=KNAME[k],
                        markeredgecolor="white", markeredgewidth=0.5)
                best = max(pts, key=lambda t: t[1])
                mac(cname("peak", b, k), best[1])
                mac(cname("peakAt", b, k), best[0])
        # average excess of the sleep speedup over each working surrogate in
        # the regime used by sleep-based studies (t_comp >= t_io)
        sl = {round(kk[6] / tio, 2): metrics(v)["S"] for kk, v in P.items()
              if kk[0] == b and kk[1] == "sleep"}
        for k in ["dgemm", "stream"]:
            wk = {round(kk[6] / tio, 2): metrics(v)["S"] for kk, v in P.items()
                  if kk[0] == b and kk[1] == k}
            gaps = [(sl[r_] / wk[r_] - 1) * 100 for r_ in sl if r_ >= 0.99 and r_ in wk]
            if gaps:
                mac(cname("sweepGap", b, k), mean(gaps), "{:.0f}")
        ax.axhline(1, color=INK2, lw=0.8)
        ax.set_title(BNAME[b], loc="left")
        ax.set_xlabel("$t_{comp}/t_{io}$")
    axs[0].set_ylabel("End-to-end speedup")
    h, l = axs[0].get_legend_handles_labels()
    fig.legend(h, l, ncol=4, loc="upper center", bbox_to_anchor=(0.5, 1.03))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_sweep.pdf"))
    plt.close(fig)

```

```

def fig_bind(rows):
    """Core allocation: one panel per backend, bars = surrogate."""
    P = pairs(rows)
    # The unbound placement is excluded: without pinning, the OS placed
    # threads differently during calibration and measurement, so its
    # compute phases missed their target length (30 to 75%).
    order = [("full", 48), ("smt", 48), ("reserved", 24)]
    labels = ["fully\nsubscribed", "SMT\nsibling", "core\nreserved"]
    ks = ["sleep", "dgemm", "stream"]
    fig, axs = plt.subplots(1, len(BACKENDS), figsize=(7.0, 2.1), sharey=True)
    x = np.arange(len(order))
    for ax, b in zip(axs, BACKENDS):
        for i, k in enumerate(ks):
            v = []
            for tag, np_ in order:
                ms = [metrics(p)["S"] for kk, p in P.items()
                      if kk[0] == b and kk[1] == k and kk[2] == tag and kk[3] == np_]
                v.append(mean(ms))
                if ms:
                    mac(cname("bind", b, k, tag), mean(ms))
                sl = [metrics(p)["slow"] for kk, p in P.items()
                      if kk[0] == b and kk[1] == k and kk[2] == tag and kk[3] == np_]
                if sl:
                    mac(cname("bindslow", b, k, tag), mean(sl))
            ax.bar(x + (i - 1) * 0.27, v, 0.24, color=C[i], label=KNAME[k])
        ax.set_xticks(x, labels, fontsize=6.3)
        ax.axhline(1, color=INK2, lw=0.8)
        ax.set_title(BNAME[b], loc="left")
    axs[0].set_ylabel("End-to-end speedup")
    h, l = axs[0].get_legend_handles_labels()
    fig.legend(h, l, ncol=3, loc="upper center", bbox_to_anchor=(0.5, 1.03))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_bind.pdf"))
    plt.close(fig)


def fig_gpu(rows):
    """CPU vs. GPU-resident computation (4 ranks sharing one GPU):
    (a) end-to-end speedup, (b) observed async run time / sleep prediction."""
    G = pairs(rows)
    ks = ["sleep", "gpu-block", "gpu-spin", "dgemm"]

    def asy(b, k):
        return [x for kk, p in G.items() if kk[0] == b and kk[1] == k for x in p["async"]]
    gv = {b: [] for b in BACKENDS}
    ex = {b: [] for b in BACKENDS}
    for b in BACKENDS:
        sl = asy(b, "sleep")
        for k in ks:
            ms = [metrics(v)["S"] for kk, v in G.items() if kk[0] == b and kk[1] == k]
            gv[b].append(mean(ms))
            if ms:
                mac(cname("gpu", b, k), mean(ms))
            sw = [metrics(v)["slow"] for kk, v in G.items() if kk[0] == b and kk[1] == k]
            if sw:
                mac(cname("gpuslow", b, k), mean(sw))
            a_ = asy(b, k)
            if sl and a_ and k != "sleep":
                n = sl[0]["steps"]
                pred = mean([r["t_total"] for r in sl]) + n * (
                    mean([r["t_alone"] for r in a_]) - mean([r["t_alone"] for r in sl]))
                x = mean([r["t_total"] for r in a_]) / pred
                ex[b].append(x)
                mac(cname("gpuExc", b, k), x)
                mac(cname("gpuExcPct", b, k), (x - 1) * 100, "{:.0f}")
            elif k != "sleep":
                ex[b].append(np.nan)
        # synchronous I/O per step after sleep vs. after the GPU kernels
        sy = {k: [(r["t_total"] - r["steps"] * r["t_comp"]) / r["steps"]
                  for kk, p in G.items() if kk[0] == b and kk[1] == k for r in p["sync"]]
              for k in ks}
        if sy["sleep"] and sy["gpu-block"]:
            mac(cname("gpuSyncIoSleep", b), mean(sy["sleep"]))
            mac(cname("gpuSyncIoBlock", b), mean(sy["gpu-block"]))
    fig, (a, b2) = plt.subplots(1, 2, figsize=(7.0, 2.1))
    bars(a, [KNAME[k] for k in ks], BACKENDS, gv)
    a.set_xticks(np.arange(len(ks)), [KNAME[k] for k in ks], rotation=0, ha="center", fontsize=6.5)
    a.axhline(1, color=INK2, lw=0.8)
    a.set_ylabel("End-to-end speedup")
    a.set_title("(a) Speedup (sync / async)", loc="left")
    k2 = ks[1:]
    bars(b2, [KNAME[k] for k in k2], BACKENDS, ex)
    b2.set_xticks(np.arange(len(k2)), [KNAME[k] for k in k2], rotation=0, ha="center", fontsize=6.5)
    b2.axhline(1, color=INK2, lw=0.8)
```

```
    b2.set_ylim(0.6, 1.4)
    # label bars that run past the axis (e.g. PDC with DGEMM) with their value
    n_ = len(BACKENDS)
    for i, s in enumerate(BACKENDS):
        for j, v in enumerate(ex[s]):
            if np.isfinite(v) and v > 1.4:
                b2.text(j + (i - (n_ - 1) / 2) * 0.8 / n_, 1.37, f"{v:.2f}", ha="center",
                        va="top", rotation=90, fontsize=5.5, color="white")
    b2.set_ylabel("Observed / sleep-predicted\nrun time (async)")
    b2.set_title("(b) Async run time vs. sleep prediction", loc="left")
    h, l = a.get_legend_handles_labels()
    fig.legend(h, l, ncol=len(BACKENDS), loc="upper center", bbox_to_anchor=(0.5, 1.03))
    fig.tight_layout(rect=(0, 0, 1, 0.9))
    fig.savefig(os.path.join(OUT, "m_gpu.pdf"))
    plt.close(fig)


def tables(tera, mpiio):
    for b in BACKENDS:
        for kk, v in pairs(tera).items():
            if kk[0] == b:
                m = metrics(v)
                mac(cname("tera", b, kk[1]), m["S"])
                mac(cname("teraslow", b, kk[1]), m["slow"])
                mac(cname("teraT", b, kk[1]), m["T_sync"] / 60, "{:.1f}")
                mac(cname("teraGB", b), v["sync"][0]["gb"], "{:.0f}")
    for kk, v in pairs(mpiio).items():
        mac(cname("mpiio", kk[1]), metrics(v)["S"])


def table_rq3(rows):
    """Model, sleep, and heat speedups, plus how much longer the heat run and
    the worst surrogate take than sleep predicts: paper/table_rq3.tex."""
    P = pairs(rows)
    ex = excess(rows)

    def S(b, k):
        ks = [v for kk, v in P.items() if kk[0] == b and kk[1] == k]
        if not ks:
            return None
        return metrics({"sync": sum((v["sync"] for v in ks), []),
                        "async": sum((v["async"] for v in ks), [])})["S"]
    lines = ["% Generated by src/analysis/plots_measured.py; do not edit.",
             "\\begin{table}[t]", "\\centering", "\\small", "\\setlength{\\tabcolsep}{2.2pt}",
             "\\begin{tabular}{@{}l c c c c c c@{}}", "\\toprule",
             " & & \\multicolumn{3}{c}{Speedup} & \\multicolumn{2}{c}{Longer than sleep predicts} \\\\",
             "\\cmidrule(lr){3-5}\\cmidrule(l){6-7}",
             "Backend & $t_{comp}/t_{io}$ & Model & Sleep & Heat & Heat & Worst \\\\", "\\midrule"]
    for b in BACKENDS:
        ss, sh = S(b, "sleep"), S(b, "heat")
        if ss is None or sh is None or b not in ex:
            continue
        sio = SYNC_IO.get(b, {})
        tc = mean([r["t_comp"] for kk, p in P.items() if kk[0] == b and kk[1] in KERNELS
                   for r in p["sync"]])
        r_eff = tc / mean(list(sio.values())) if sio else float("nan")
        model = (r_eff + 1) / max(r_eff, 1)
        wk = max(ex[b], key=ex[b].get)
        eh, ew = (ex[b]["heat"] - 1) * 100, (ex[b][wk] - 1) * 100
        mac(cname("ratioEff", b), r_eff)
        mac(cname("model", b), model)
        mac(cname("worstExcKernel", b), KNAME[wk])
        lines.append(f"{BNAME[b].split()[0]} & {r_eff:.2f} & {model:.2f} & {ss:.2f} & {sh:.2f} & "
                     f"{eh:+.0f}\\% & {ew:+.0f}\\% ({KNAME[wk]}) \\\\")
    lines += ["\\bottomrule", "\\end{tabular}",
              "\\caption{Speedup predicted by the overlap model of Ravi et "
              "al.~\\cite{ravi2023evaluating} for each backend's measured ratio "
              "$t_{comp}/t_{io}$ (no overhead), and measured with \\texttt{sleep} and with "
              "the heat mini-app. The last two columns give how much longer the "
              "asynchronous run with the heat mini-app, and with the least favourable "
              "surrogate, takes than the \\texttt{sleep} run predicts for the same "
              "compute length (Section~\\ref{sec:metrics}).}",
              "\\label{tab:rq3}", "\\end{table}"]
    with open(os.path.join(ROOT, "paper", "table_rq3.tex"), "w") as f:
        f.write("\n".join(lines) + "\n")


def recheck():
    """Synchronous spin and sleep runs repeated in swapped order (spin first):
    synchronous I/O time per step after each kind of compute phase."""
    rows = load("recheck")
    if not rows:
        return
    by = defaultdict(dict)
    for r in rows:
```

```
        by[r["io"]][r["kernel"]] = (r["t_total"] - r["steps"] * r["t_comp"]) / r["steps"]
    ratios = []
    for b, d in by.items():
        if "spin" in d and "sleep" in d:
            mac(cname("recheckSpin", b), d["spin"])
            mac(cname("recheckSleep", b), d["sleep"])
            mac(cname("recheckPct", b), (d["sleep"] / d["spin"] - 1) * 100, "{:.0f}")
            ratios.append((d["sleep"] / d["spin"] - 1) * 100)
    if ratios:
        mac("RecheckMinPct", min(ratios), "{:.0f}")
        mac("RecheckMaxPct", max(ratios), "{:.0f}")


def variability(io0):
    """Run-to-run spread of the duplicated I/O-only baselines."""
    agg = defaultdict(list)
    for r in io0:
        agg[(r["io"], r["mode"], r["mib_per_rank"], r["nranks"], r["tag"])].append(r["t_total"])
    sp = sorted((max(v) - min(v)) / min(v) * 100 for v in agg.values() if len(v) >= 2)
    if sp:
        mac("VarPairs", len(sp), "{:d}")
        mac("VarMedian", float(np.median(sp)), "{:.0f}")
        mac("VarPseventyfive", sp[int(0.75 * len(sp))], "{:.0f}")
        mac("VarMax", sp[-1], "{:.0f}")


def table_io0(io0):
    """Measured I/O-only baselines (48 ranks, 5 steps): paper/table_io0.tex."""
    agg = defaultdict(list)
    for r in io0:
        if r["nranks"] == 48 and r["tag"] == "full" and r["kernel"] == "none":
            agg[(r["io"], r["mode"], r["mib_per_rank"])].append(r["t_total"] / r["steps"])
    lines = ["% Generated by src/analysis/plots_measured.py; do not edit.",
             "\\begin{table}[t]", "\\centering", "\\small", "\\setlength{\\tabcolsep}{3.5pt}",
             "\\begin{tabular}{@{}l r r r r r@{}}", "\\toprule",
             "Backend & GiB/step & Sync (s) & Async (s) & Async/sync & Sync GB/s \\\\",
             "\\midrule"]
    for b in BACKENDS:
        first = True
        for mb in (16, 64, 256, 1024):
            s_, a_ = agg.get((b, "sync", mb)), agg.get((b, "async", mb))
            if not s_ or not a_:
                continue
            gib = 48 * mb / 1024
            ts, ta = mean(s_), mean(a_)
            bw = gib * 1.073741824 / ts
            name = BNAME[b].split()[0] if first else ""
            first = False
            lines.append(f"{name} & {gib:g} & {ts:.2f} & {ta:.2f} & {ta / ts:.2f} & {bw:.1f} \\\\")
            tag = SZ[mb]
            mac(cname("ioSync", b, tag), ts)
            mac(cname("ioAsync", b, tag), ta)
            mac(cname("ioRatio", b, tag), ta / ts)
            mac(cname("ioBw", b, tag), bw, "{:.1f}")
            mac(cname("ioN", b, tag), min(len(s_), len(a_)), "{:d}")
        if b != BACKENDS[-1]:
            lines.append("\\addlinespace[2pt]")
    lines += ["\\bottomrule", "\\end{tabular}",
              "\\caption{Measured I/O-only time per step (48 ranks, 5 steps, no computation; "
              "mean of the repetitions). Async/sync below 1 means the asynchronous mode is "
              "faster even without computation to overlap.}",
              "\\label{tab:io0}", "\\end{table}"]
    with open(os.path.join(ROOT, "paper", "table_io0.tex"), "w") as f:
        f.write("\n".join(lines) + "\n")


def main():
    os.makedirs(OUT, exist_ok=True)
    io0 = load("io0")
    main_rows = load("main")
    if io0:
        table_io0(io0)
        variability(io0)
    recheck()
    if main_rows:
        # PDC (1 server on a reserved core, 47 clients) joins the main-experiment
        # figures as a fifth backend when its runs exist.
        global BACKENDS
        pdc_rows = load("pdc")
        four = BACKENDS
        if pdc_rows:
            main_rows = main_rows + pdc_rows
            BACKENDS = four + ["pdc"]
        SYNC_IO.update(sync_io_per_step(main_rows))
        fig_main(main_rows)
```

```
        fig_decomp(main_rows)
        fig_excess(main_rows)
        table_rq3(main_rows)
        pdc_smt(pdc_rows)
        BACKENDS = four
        # PDC also joins the size, sweep, and GPU figures when its phases
        # exist; its main-experiment runs (t_comp = 1.25 t_io, 11.75 GiB per
        # step) supply the matching point of the size and sweep panels.
        pdc_main = [r for r in pdc_rows if r["tag"] == "pdc1"]
        main_rows = [r for r in main_rows if r["io"] != "pdc"]
        five = four + ["pdc"]
        BACKENDS = five if load("pdcsize") else four
        if load("size"):
            fig_size(load("size") + (load("pdcsize") + pdc_main if load("pdcsize") else []))
        BACKENDS = five if load("pdcsweep") else four
        if load("sweep"):
            fig_sweep(load("sweep") + (load("pdcsweep") + pdc_main if load("pdcsweep") else []),
                      main_rows + pdc_main)
        BACKENDS = five if load("pdcgpu") else four
        if load("gpu"):
            fig_gpu(load("gpu") + load("pdcgpu"))
        BACKENDS = four
    if load("bind"):
        fig_bind(load("bind"))
    tables(load("tera"), load("mpiio"))
    write_macros()
    print(f"wrote {len(MAC)} macros to {MACROS}")


if __name__ == "__main__":
    main()
```